



03 大模型与智能体

3.1 人工智能与大模型

3.2 大模型发展历程

3.3 大模型相关概念

3.4 大语言模型 (LLM)

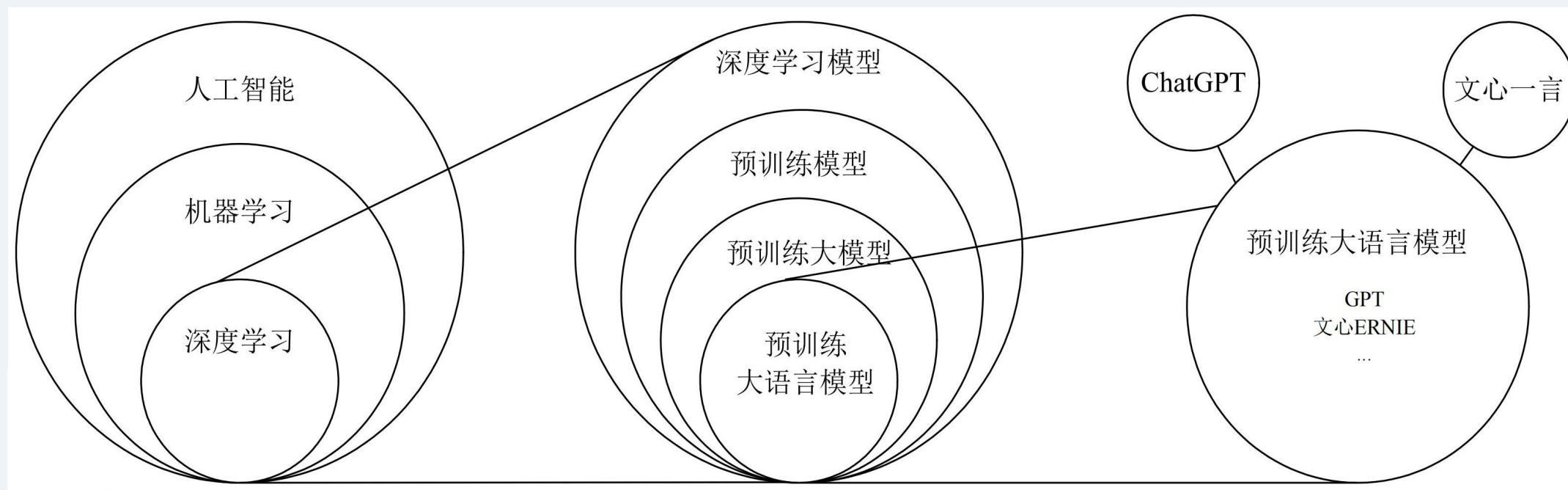
3.5 从LLM到AI Agents

3.6 从AI Agents到Agentic AI Systems

3.7 大模型智能体框架与协议

✧ 3.1 人工智能与大模型

- 人工智能包含了机器学习，机器学习包含了深度学习，深度学习可以采用不同的模型，其中一种模型是预训练模型，预训练模型包含了预训练大模型（可以简称为“大模型”），预训练大模型包含了预训练大语言模型（可以简称为“大语言模型”），预训练大语言模型的典型代表包括OpenAI的GPT和百度的文心ERNIE，ChatGPT是基于GPT开发的大模型产品，文心一言是基于文心ERNIE开发的大模型产品。



✧ 3.1 大模型发展历程

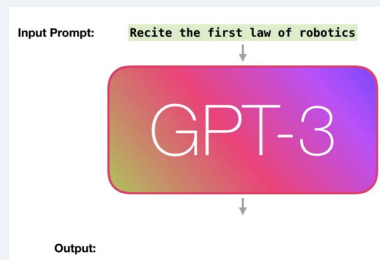
- 大模型通常指的是大规模的人工智能模型，是一种基于深度学习技术，具有海量参数、强大的学习能力和泛化能力，能够处理和生成多种类型数据的人工智能模型。

大模型的“大”的特点体现在

参数数据量庞大

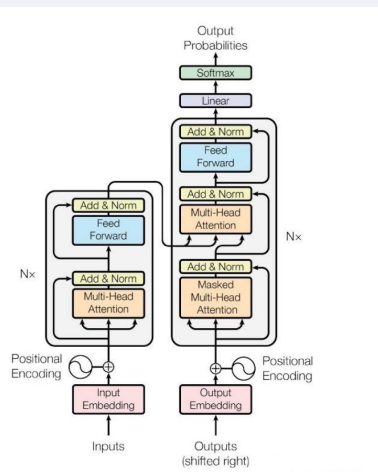
训练数据量大

计算资源需求高



模型架构：Transformer

2017 年，Vaswani 等提出 Transformer 架构，奠定了当前大模型领域主流的算法架构基础；Transformer 结构的提出，使深度学习模型参数达到了上亿的规模。

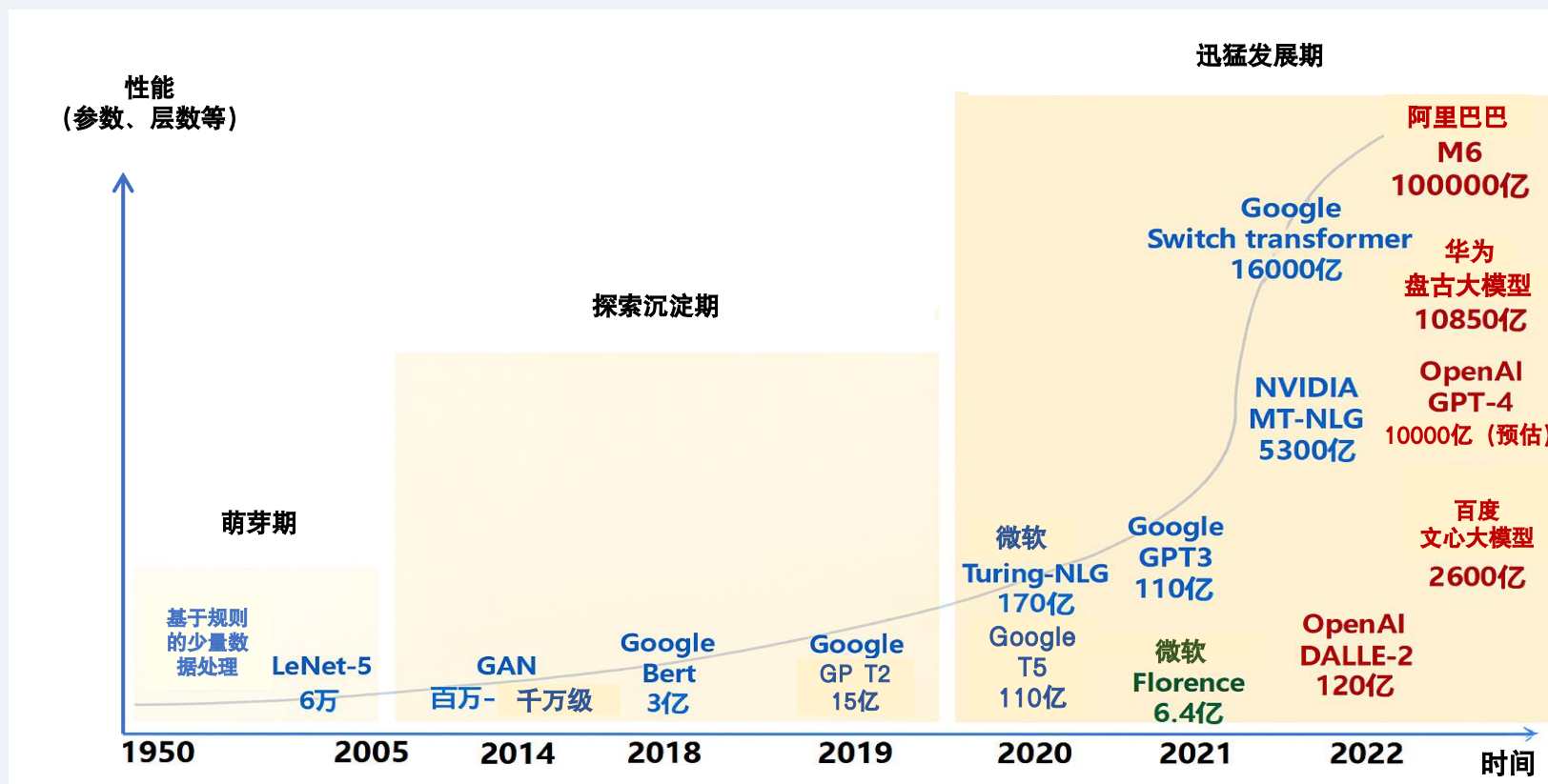


里程碑式预训练模型：GPT-3

2020年7月，OpenAI发布彼时全球规模最大的用于文本生成的预训练自然语言模型—GPT-3（31位作者 45TB数据 72页论文 1750亿个参数），GPT-3 在许多 NLP 数据集上均具有出色的性能，不仅包括翻译、问答和文本填空任务，也包括一些需要即时推理或领域适应的任务。

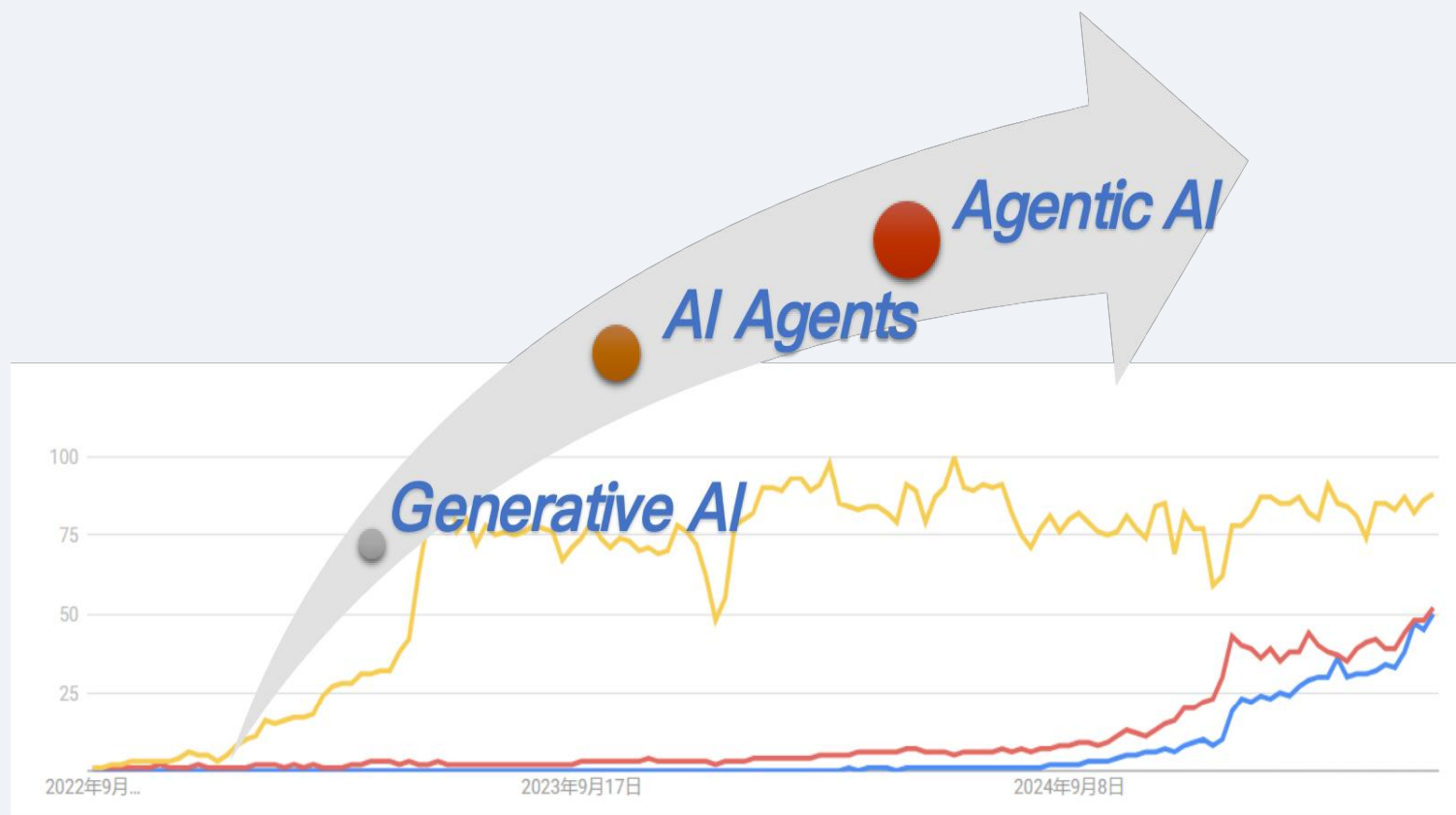
3.2 大模型发展历程

大模型发展历经三个阶段，分别是萌芽期、沉淀期和爆发期。



✧ 3.2 大模型发展历程

- 自2022年底ChatGPT开启大模型时代以来，人工智能系统从单点生成模型向更复杂的任务执行和系统协调迈进。
- 大模型从生成模型到工具增强智能体，从单体智能进一步迈向多智能体协作的 Agentic AI 系统。



✧ 3.3 大模型相关概念

- 大模型的设计和训练旨在提供更强大、更准确的模型性能，以应对更复杂、更庞大的数据集或任务。大模型通常能够学习到更细微的模式和规律，具有更强的泛化能力和表达能力。

上下文理解能力

大模型具有更强的上下文理解能力，能够理解更复杂的语意和语境。这使得它们能够产生更准确、更连贯的回答。



语言生成能力

大模型可以生成更自然、更流利的语言，减少了生成输出时呈现的错误或令人困惑的问题。



学习能力强

大模型可以从大量的数据中学习，并利用学到的知识和模式来提供更精准的答案和预测。这使得它们在解决复杂问题和应对新的场景时表现更加出色。



可迁移性高

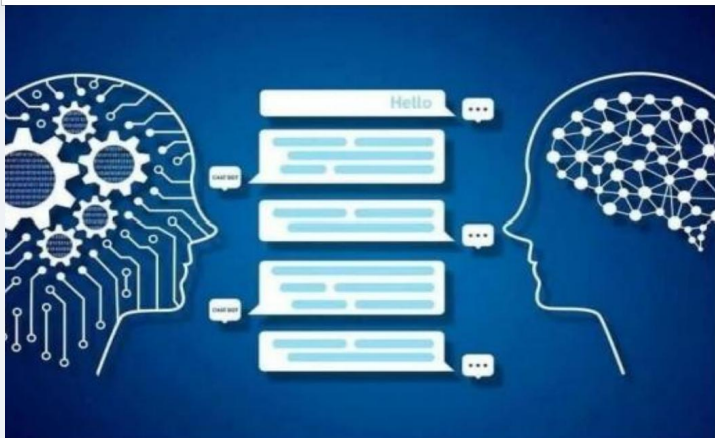
学习到的知识和能力可以在不同的任务和领域中迁移和应用。这意味着一次训练就可以将模型应用于多种任务，无需重新训练。



✧ 3.3 大模型相关概念

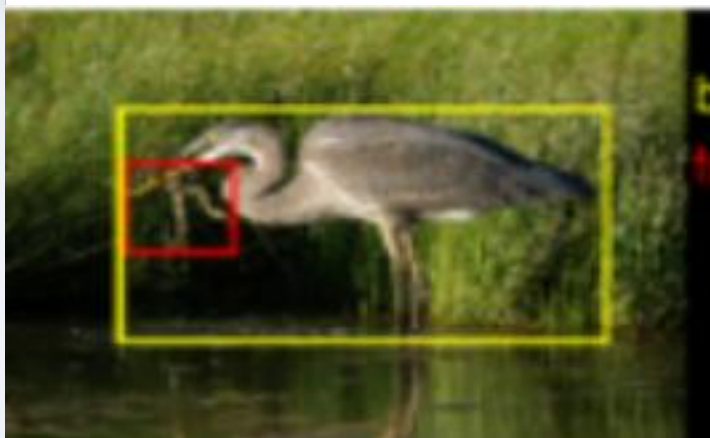
语言大模型

指在自然语言处理 (Natural Language Processing, NLP) 领域中一类大模型，通常用于处理文本数据和理解自然语言。这类大模型的主要特点是它们在大规模语料库上进行了训练，以学习自然语言的各种语法、语义和语境规则。代表性产品包括GPT 系列 (OpenAI)、Bard(Google)、DeepSeek、文心一言(百度)等。



视觉大模型

指在计算机视觉 (Computer Vision, CV) 领域中使用的大模型，通常用于图像处理和分析。这类模型通过在大规模图像数据上进行训练，可以实现各种视觉任务，如图像分类、目标检测、图像分割、姿态估计、人脸识别等。代表性产品包括VIT系列 (Google)、文心UFO、华为盘古 CV、INTERN (商汤)等。



多模态大模型

指能够处理多种不同类型数据的大模型，例如文本、图像、音频等多模态数据。这类模型结合了NLP 和 CV 的能力，以实现了对多模态信息的综合理解和分析，从而能够更全面地理解 and 处理复杂的数据。代表性产品包括DingoDB 多模向量数据库 (九章云极DataCanvas)、DALL-E (OpenAI)、悟空画画 (华为)、midjourney 等。



✧ 3.3 大模型相关概念

▣ 按照应用领域的不同，大模型主要可以分为L0、L1、L2三个层级。

通用大模型L0 (Large Model, General Purpose)

指可以在多个领域和任务上通用的大模型。它们利用大算力、使用海量的开放数据与具有巨量参数的深度学习算法，在大规模无标注数据上进行训练，以寻找特征并发现规律，进而形成可“举一反三”的强大泛化能力，可在不进行微调或少量微调的情况下完成多场景任务，相当于AI完成了“通识教育”

- 举例：GPT-4
- 描述：可以处理各种任务，如文本生成、翻译、问答等，适用于多领域、广泛使用。

行业大模型L1 (Industry-specific Large Model Level 1)

指那些针对特定行业或领域的大模型。它们通常使用行业相关的数据进行预训练或微调，以提高在该领域的性能和准确度，相当于AI成为“行业专家”

- 举例：医药行业的专用模型（如专门用在医学诊断或药物研发的模型）
- 描述：针对特定行业的专业知识与应用场景训练，能提供行业内更准确的建议和分析。

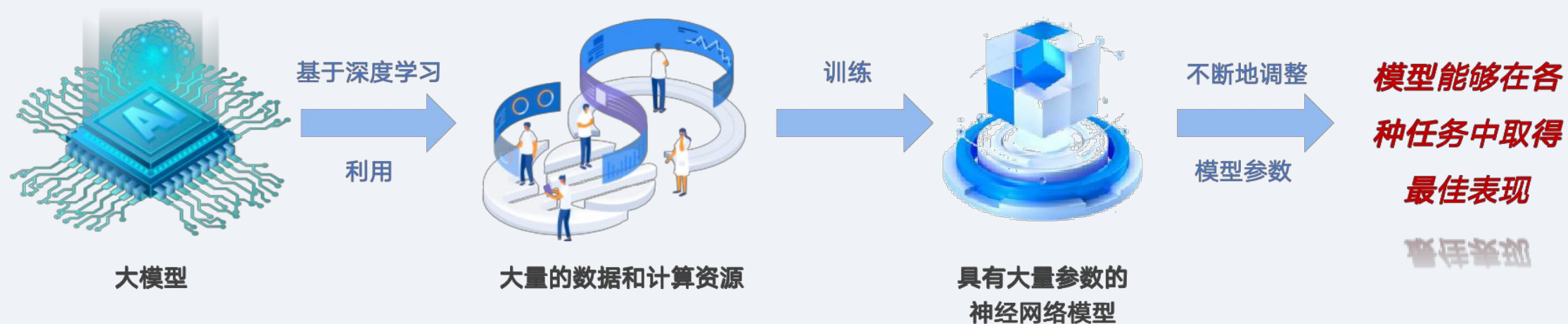
垂直大模型L2 (Vertical Large Model Level 2)

指那些针对特定任务或场景的大模型。它们通常使用任务相关的数据进行预训练或微调，以提高在该任务上的性能和效果

- 举例：金融行业的专业智能助手（如用于证券分析或投资顾问的模型）
- 描述：细化到特定任务或领域的应用，精通某一行业的具体知识和技能，提供更深层次的专业性能和效果。

✧ 3.4 大语言模型 (LLM)

- 大语言模型是基于Transformer架构的，这种架构是一种专门用于自然语言处理的“编码-解码器”架构。在训练过程中，大模型将输入的单词以向量的形式传递给神经网络，然后通过网络的编码解码以及自注意力机制，建立起每个单词之间联系的权重。大模型的核心能力在于将输入的每句话中的每个单词与已经编码在模型中的单词进行相关性的计算，并把相关性又编码叠加在每个单词中。这样，大模型能够更好地理解和生成自然文本，同时还能够表现出一定的逻辑思维和推理能力。



3.4.1 大语言模型的训练

- ❑ 语言模型 (Language model)：计算给定单词序列在句子中出现的概率

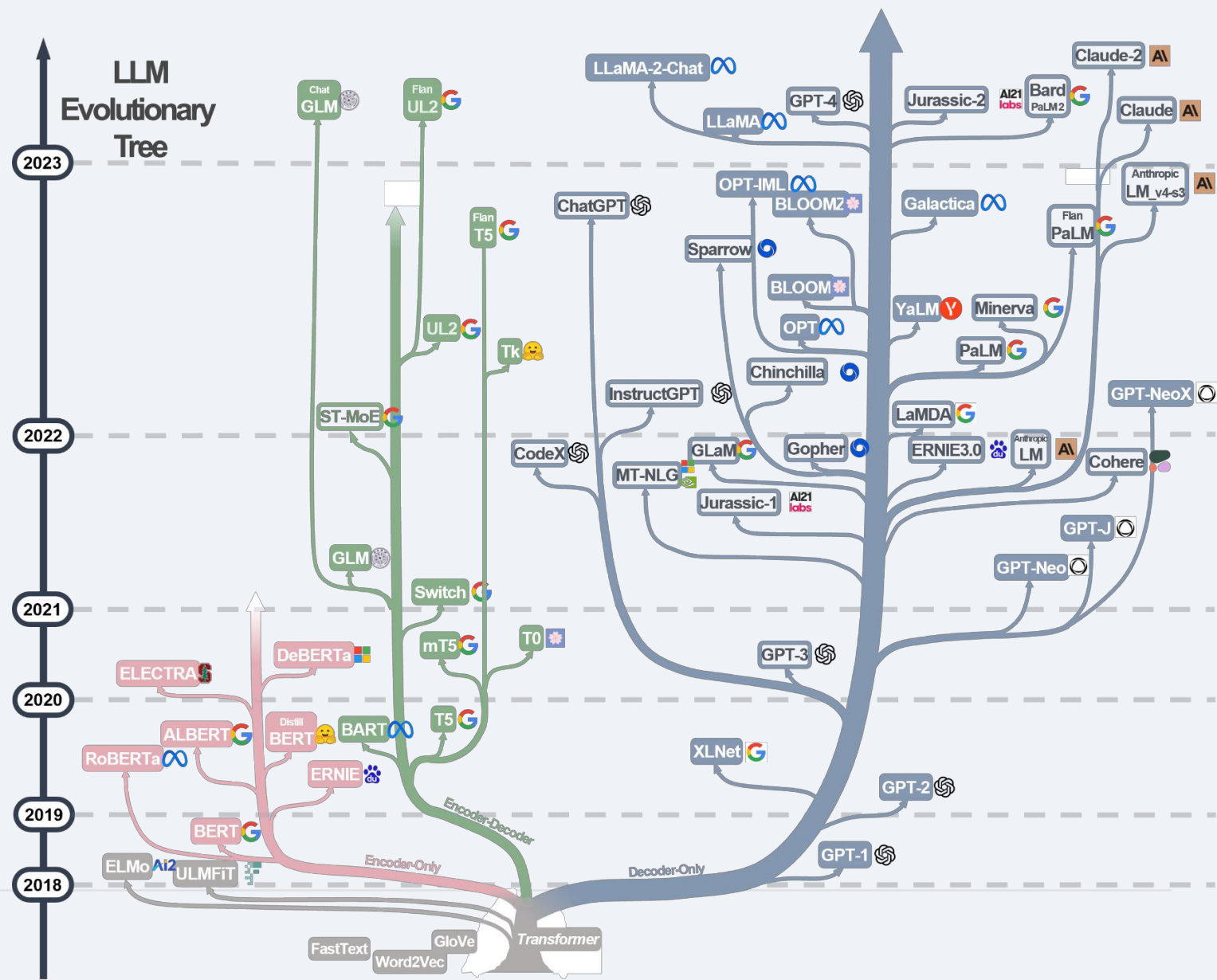
算给定单词序列在句子中出现的概

$$\square \sum_{s \in V^+} P_{LM}(s) = 1$$

□ 语言建模 (Language modeling) :
根据前文预测下一个单词

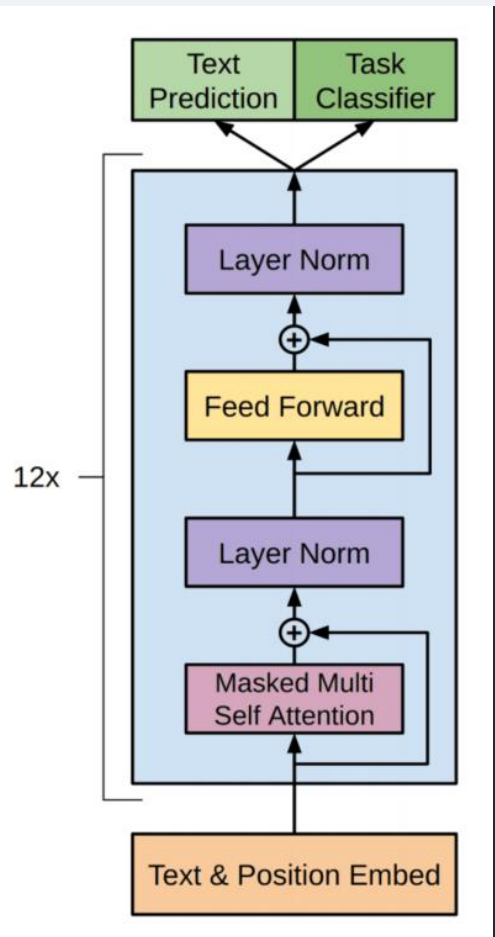
根据前文预测下一个单词

The animal didn't cross the

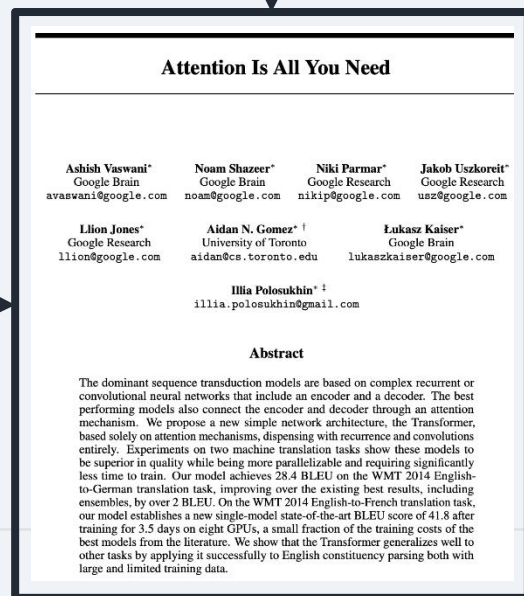
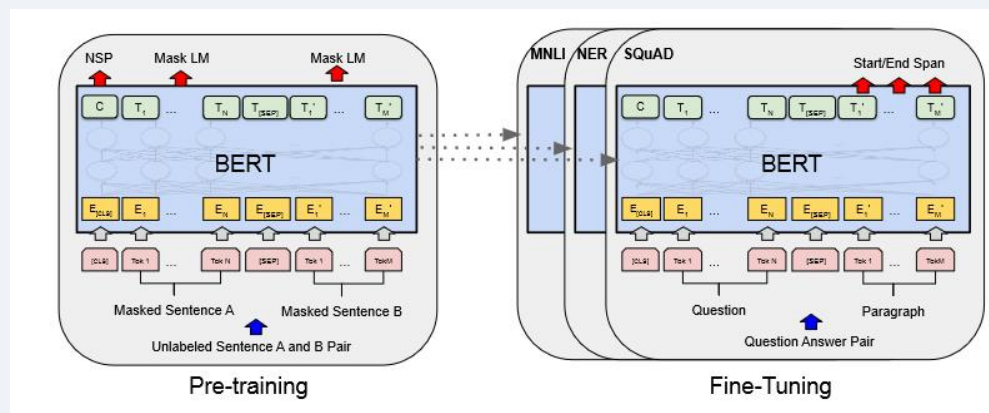


3.4.1 大语言模型的训练

GPT-1预训练



BERT模型框架



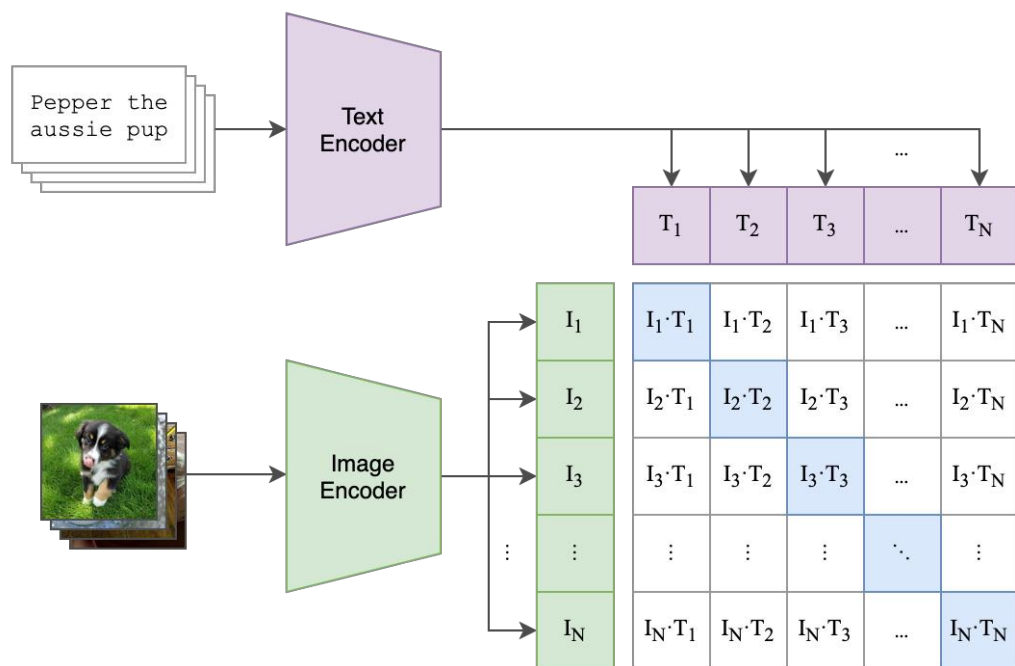
无论是 BERT 还是 GPT 都基于 Transformer

- 近年来，大语言模型正在快速发展。2018年，Google和OpenAI分别推出了BERT和GPT。
- 解码器模型主导了LLM的发展，尤其是自2021年GPT-3推出之后。编码器模型最初由BERT带来的爆炸性发展，逐渐消失了。
- LLM可大致分为GPT和BERT两种风格。

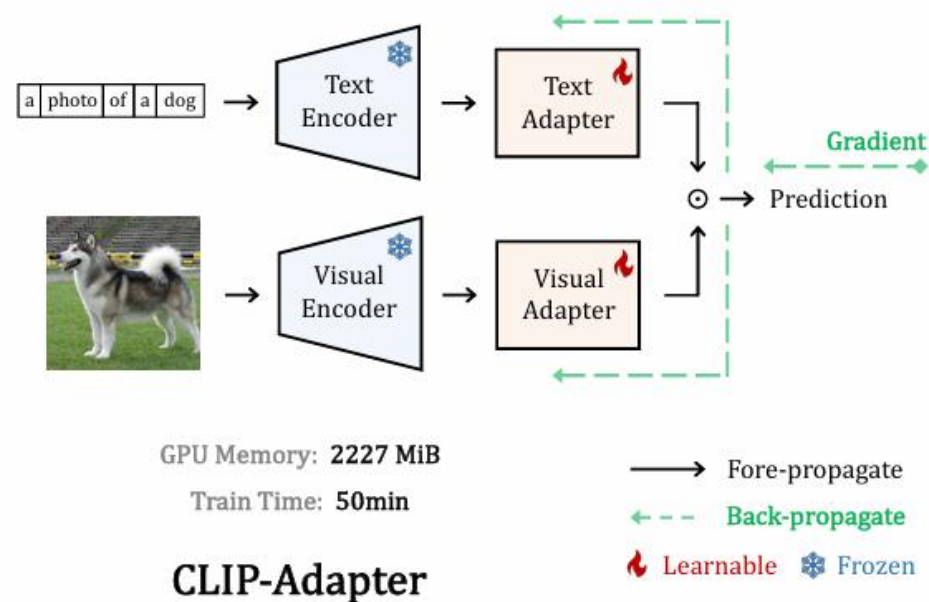
✧ 3.4.2 视觉大模型的训练

▣ 训练范式：过大规模图文对进行对比预训练，结合下游任务微调（可选）。

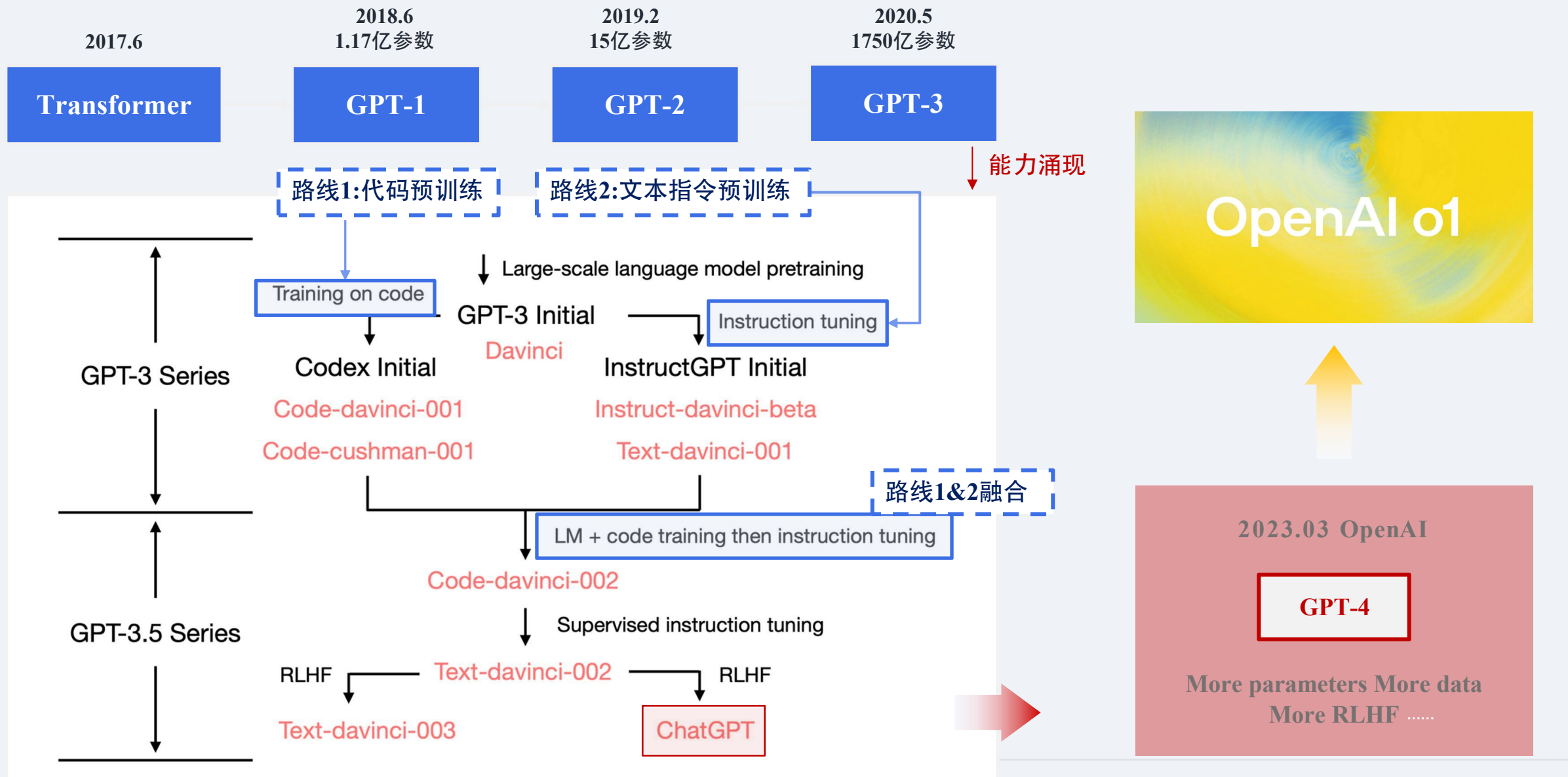
Contrastive pre-training



Task Adaptation



3.4.3 GPT 系列模型演化与核心技术



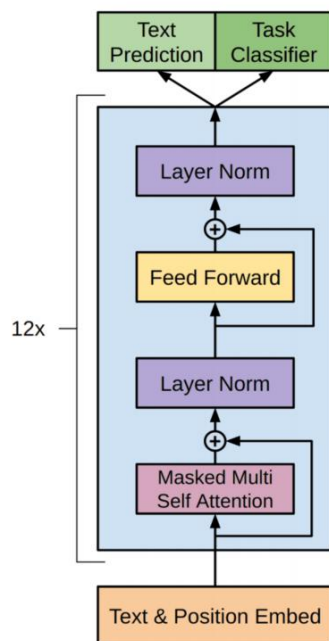
3.4.3 GPT 系列模型演化与核心技术

GPT-1

核心逻辑

在无标签的数据集上学习一个通用的语言模型（**预训练阶段**），再根据特定任务进行有监督微调（**微调阶段**）。

预训练阶段



Transformer模型
解码器+最大化似然函数

Transformer解码器与编码器不同，不包含输入的右侧信息，这与下一句预测是一致的

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$
$$P(u) = \text{softmax}(h_n W_e^T)$$

k : 上文窗口大小 u : embedded tokens n : Transformer层的数量

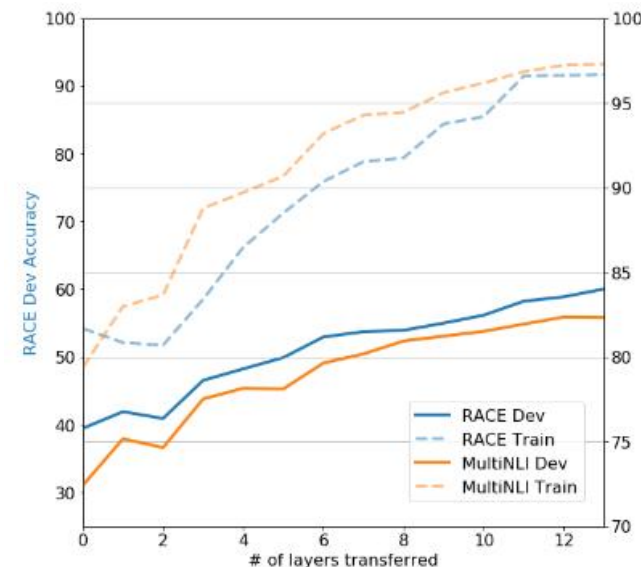
微调阶段

对于不同任务，在输入中添加特殊token，在模型的输出部分附加新的结构（参数）。

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y | x^1, \dots, x^m).$$

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C})$$

增加transformer的层数，可提高评测数据集下的精度。



3.4.3 GPT 系列模型演化与核心技术

GPT-2

核心逻辑

使用无监督的预训练模型完成有监督任务，即**任何任务均可视作生成任务**。

相比 GPT-1 模型提升与能力扩展

01

模型架构无创新，大幅度增加参数量

117M → 1.5B

02

引入任务信息作为预测条件

$p(\text{output}|\text{input}) \rightarrow p(\text{output}|\text{input}; \text{task})$

03

增添百万级网页作为数据集进行训练

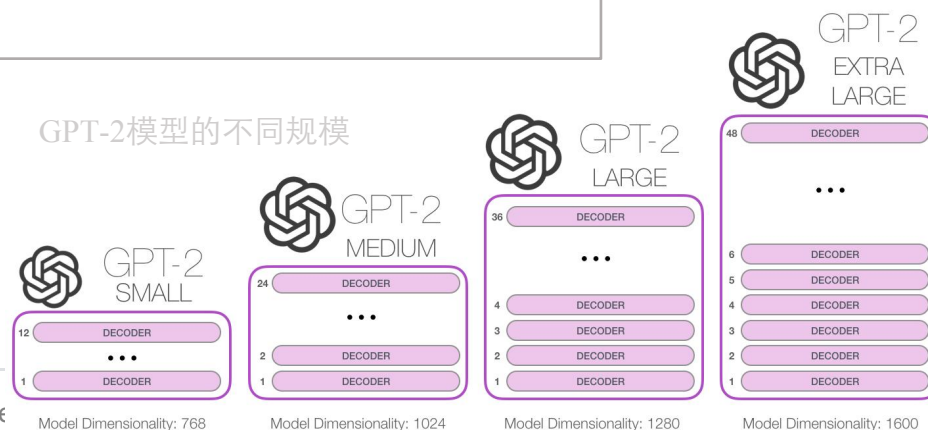
设想语言模型在不进行特定监督的条件下具有多任务处理能力。

04

侧重于zero-shot设定下的语言模型能力

模型根据给定指令理解并完成任务，不再依赖下游任务数据进行参数微调。

GPT-2模型的不同规模



3.4.3 GPT 系列模型演化与核心技术

GPT-3

核心逻辑

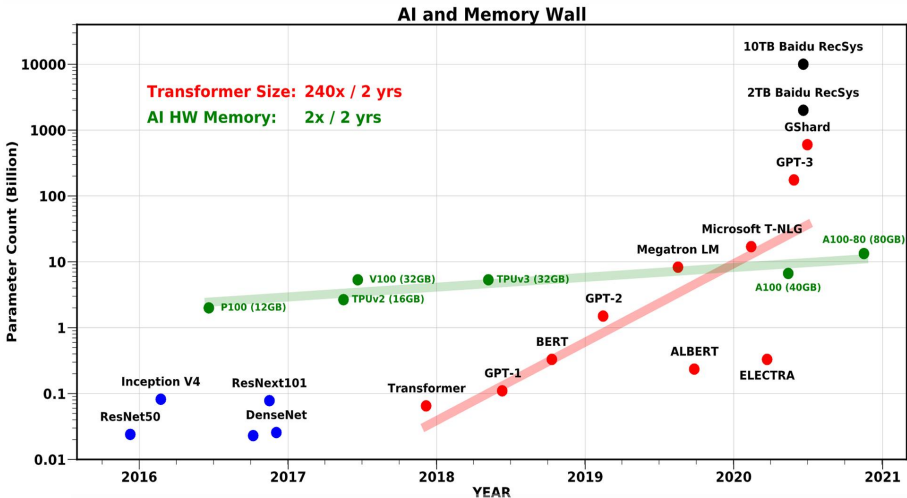
GPT-2 在其训练规模下已达到拟合能力上限，仍无法充分捕捉语言复杂性。Scaling Laws 表示继续扩大模型和数据规模能持续带来性能提升。

相比 GPT-2模型提升

- 01 训练数据规模大 - 45TB
- 02 超大模型规模 - 1750亿参数量
- 03 相同的模型架构，延续无监督模型应用到有监督任务的思想
- 04 能力涌现 - 上下文学习 (in-context learning)
具备极强的少样本甚至零样本学习能力

缺点

- 理解能力不足：从左到右生成式学习 (vs. BERT)
- 推理能力不足
- 训练代价巨大
- 不能够更好地遵循用户意图



大规模语言模型发展历程

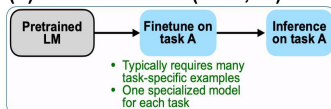
3.4.3 GPT 系列模型演化与核心技术

InstructGPT

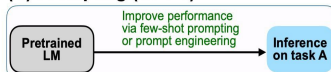
核心逻辑

通过人类反馈调整语言模型以使其在广泛任务中与用户意图适配

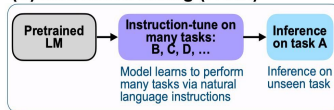
(A) Pretrain-finetune (BERT, T5)



(B) Prompting (GPT-3)



(C) Instruction tuning (FLAN)



- 指令精调 -- 通过编辑输入挖掘预训练模型蕴含的潜在知识，以更好的完成下游任务。

基础

编辑下游任务的输入，使其形式上模拟模型预训练阶段使用的数据和任务

以「情感分类任务」为例

□ 传统监督学习

输入：“今天被批评了”

输出：“消极”

□ 提示学习

输入：“今天被批评了，感觉很——。”

输出：“沮丧” → 映射为 “消极”

提示学习的加强版

通过构造“指令”并微调的方式学习人类交互模式的分布

模型面临的不再是单纯的补全任务，而是各种不同任务的“指令”

例：

- 请帮我找出以下代码中的bug；
- 请帮我制定一份云南旅行计划，预算1w之内

✧ 3.4.3 GPT 系列模型演化与核心技术：InstructGPT

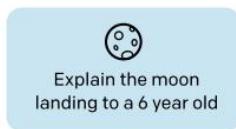
训练过程

❑ 构建指令数据集，有监督微调GPT-3

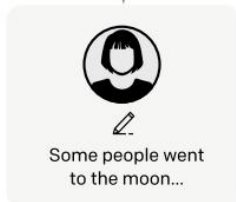
Step 1

Collect demonstration data, and train a supervised policy.

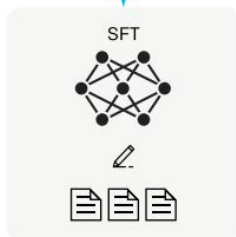
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.

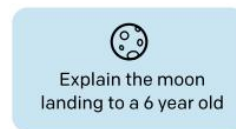


❑ 基于人类反馈的强化学习

Step 2

Collect comparison data, and train a reward model.

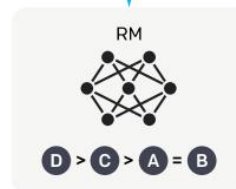
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



❑ 模型与用户需求对齐

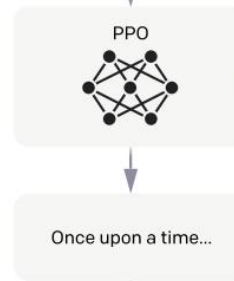
Step 3

Optimize a policy against the reward model using reinforcement learning.

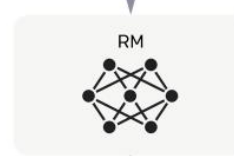
A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



3.4.3 GPT 系列模型演化与核心技术：GPT-4

■ 2023年3月14日，OpenAI发布GPT-4，标志着向通用人工智能的又一步。

从 ChatGPT 到 GPT-4



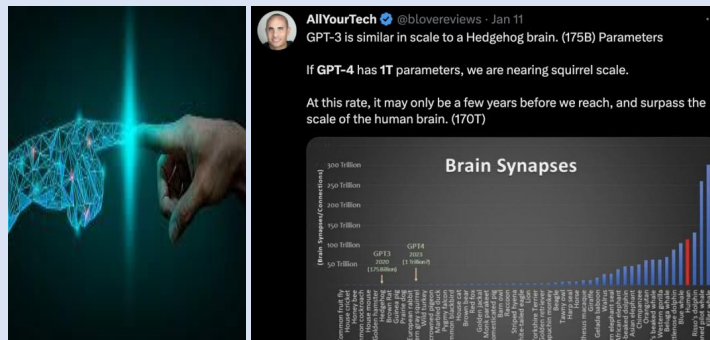
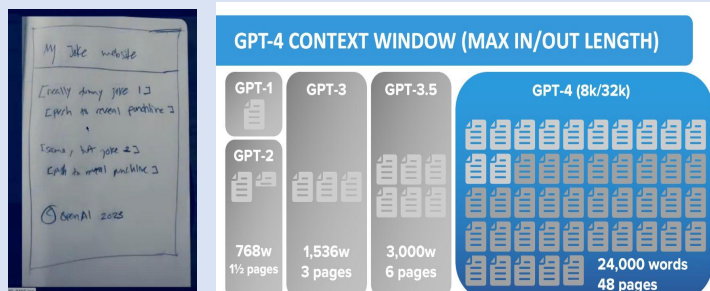
+ 人类意图对齐

+ 引入领域专家

可控性 + 安全性

+ 图片模态输入

+ 超长文本输入



+ 预训练数据量

+ 模型参数量

3.4.3 GPT 系列模型演化与核心技术: OpenAI o1

两大关键技术

强化学习

模型在回答问题之前,先产生一系列的内部思维链,通过**试错**来优化推理路径,从而显著提高了解答复杂问题的准确性。

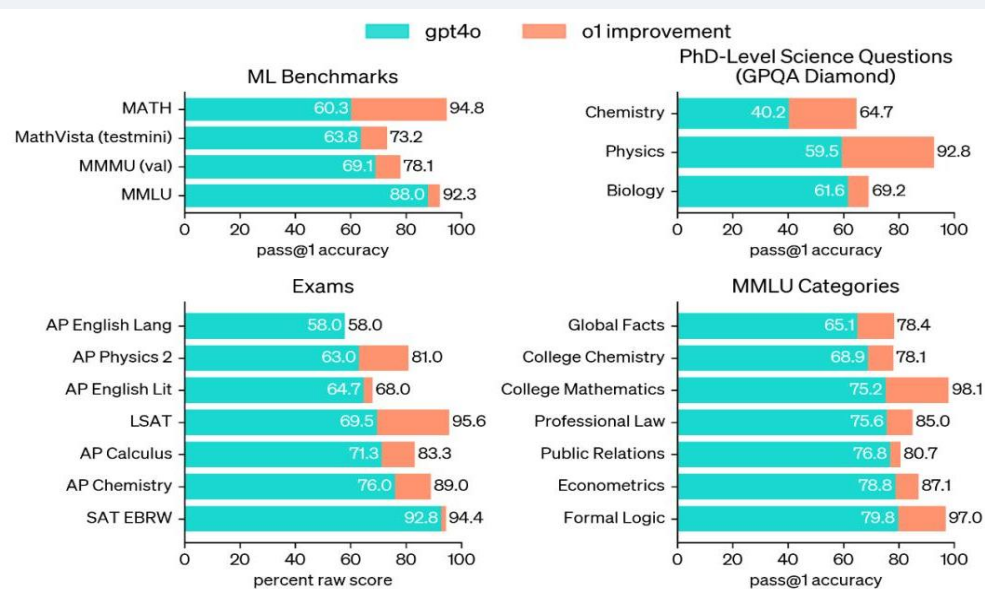
思维链 CoT

将复杂问题分解成一系列**简单的、逻辑连贯的步骤**来解决,类似于人类通过逐步推理来解决问题。理论上,引入CoT,固定深度、常数精度的Transformer模型就可以解决任何由大小为T的布尔电路所解决的问题。这表明CoT显著扩展了模型的表达能力,使其能够处理更复杂的问题。



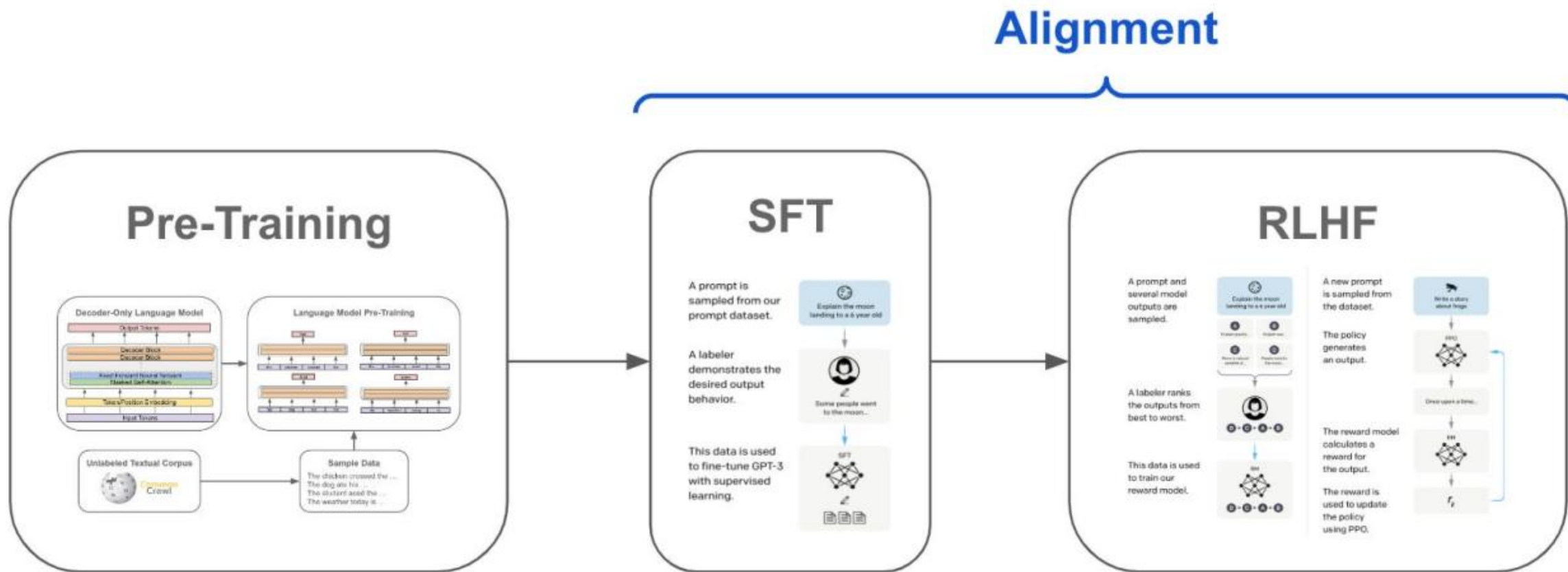
性能显著提升 - 在数学、代码、长程规划等问题上

- 竞争性编程问题 (Codeforces) 中排名第 89 个百分位
- 美国数学奥林匹克竞赛 (AIME) 资格赛中跻身美国前 500 名学生之列
- 物理、生物和化学问题的基准 (GPQA) 上超过了人类博士水平的准确性



3.4.4 大语言模型训练

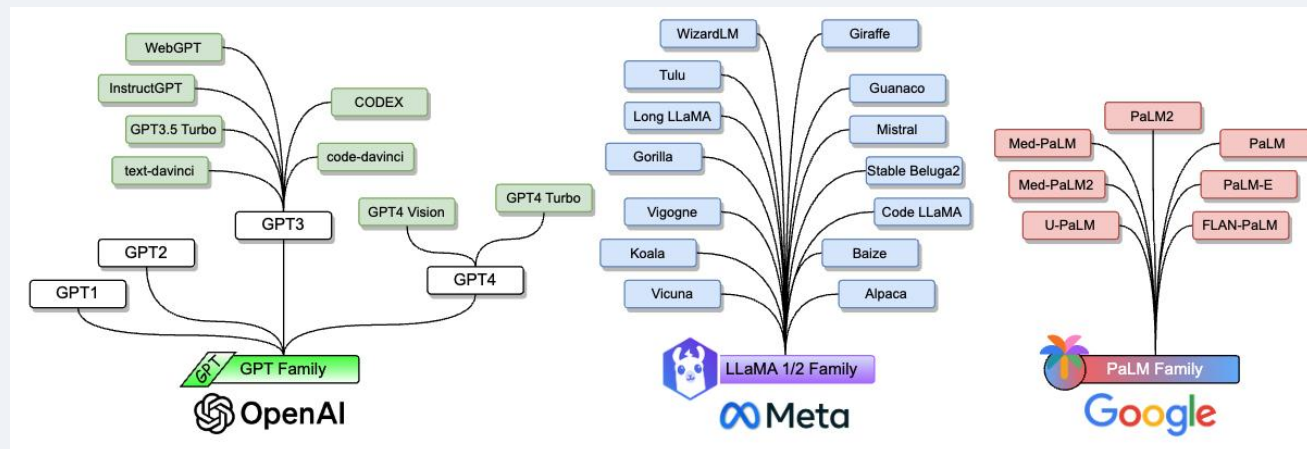
训练范式：预训练-微调（SFT）-基于人类反馈的强化学习（RLHF）



3.4.4 大模型族谱

主流大模型家族

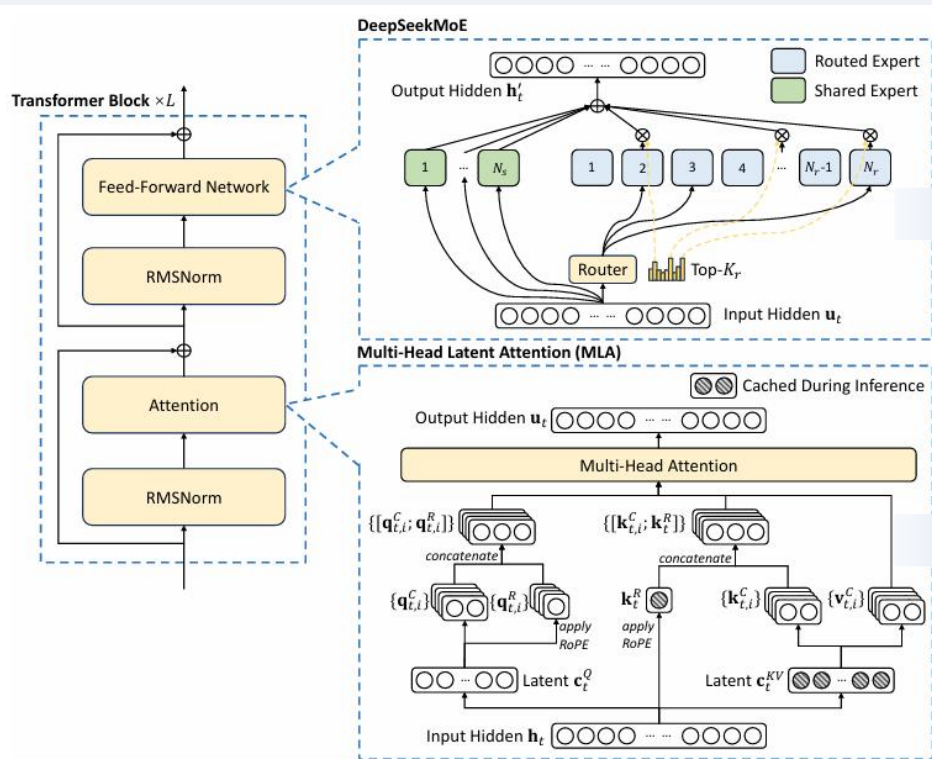
- ❑ OpenAI
- ❑ Meta
- ❑ Google
- ❑ Qwen
- ❑ DeepSeek



Qwen - Qwen { 1.8B, 7B, 14B, 72B } - Qwen1.5 { 0.5B, 1.8B, 4B, 14B, 27B, 72B, 110B, 128B, 224B, 32B, 72B, and 110B } - Qwen2 { 0.5B, 1.5B, 7B, 57A, 72B } - Qwen2.5 { 0.5B, 1.5B, 3B, 7B, 14B, 32B, 72B }	Qwen-Audio - Qwen-Audio { 7B } - Qwen2-Audio { 7B }	Qwen-Math - Qwen2-Math { 1.5B, 7B, 72B } - Qwen2.5-Math { 1.5B, 7B, 72B }	DeepSeek-R1 - DeepSeek-R1 { 671B } - DeepSeek-R1-Zero { 671B }	DeepSeek-MoE - DeepSeek-MoE { 16B } - DeepSeek-V2 { 230B, 15.7B } - DeepSeek-V2.5 { 230B } - DeepSeek-V3 { 671B }	DeepSeek-Coder - DeepSeek-Coder { 1.3B, 6.7B, 33B } - DeepSeek-Coder-V1.5 { 7B } - DeepSeekCoder-V2 { 230B, 16B }
Qwen-VL - Qwen-VL { 7B } - Qwen2-VL { 2B, 7B, 72B }	Qwen-Coder - CodeQwen1.5 { 7B } - CodeQwen1.5 { 0.5B, 1.5B, 3B, 7B, 14B, 32B }	QwQ QwQ-32B-Preview { 32B }	Janus - Janus { 1.3B } - JanusFlow { 1.3B } - Janus-Pro { 1B, 7B }	DeepSeek-Prover - DeepSeek-Prover { 7B } - DeepSeek-Prover-v1.5 { 7B }	DeepSeek-Math DeepSeek-Math { 7B }
		QvQ QvQ-72B-Preview { 72B }		DeepSeek-VL - DeepSeek-VL { 1.3B, 7B } - DeepSeek-VL-2 { 3B, 16B, 27B }	DeepSeek LLM DeepSeek-LLM { 7B, 47B }

3.4.5 国内大模型产品：DeepSeek V3

基本架构



DeepSeekMoE (Mixture of Experts)

通过将模型拆分为多个“专家”，仅激活相关专家，大幅降低参数量与算力开销。引入**共享专家 (Shared Expert)** 用于提取全局性知识，**路由专家 (Routed Expert)** 则处理输入中的特定专业特征，实现细粒度专家调度。

MLA (Multi-Head Latent Attention)

对注意力的键值对进行**低秩联合压缩**，将注意力输入压缩为 latent vector，仅在需要时映射回高维空间恢复 key 和 value，从而显著减少 KV 缓存，提升推理效率。

架构创新

MTP (Multi-Token Prediction)

在每个位置预测多个未来 token，使训练信号更密集，提升数据效率；同时帮助模型**预先规划表征结构**，更好地建模未来 token 分布。

Auxiliary-Loss-Free Load Balancing

为每个专家引入偏置项 b_i 并加权亲和度分数 s_i ，动态调整专家激活顺序，无需辅助损失即可实现**高效负载均衡**，提升专家利用率与模型稳定性。

✧ 3.4.5 国内大模型产品：DeepSeek R1

基本训练方式

监督微调 SFT (Supervised Fine-Tuning)

▣ 预训练模型

使用大规模通用数据集训练基础模型。

▣ 收集任务数据

收集与目标任务相关的标注数据（如情感标签文本等）。

▣ 监督微调

用任务数据优化预测与标签间的损失，使模型学习任务特定特征。

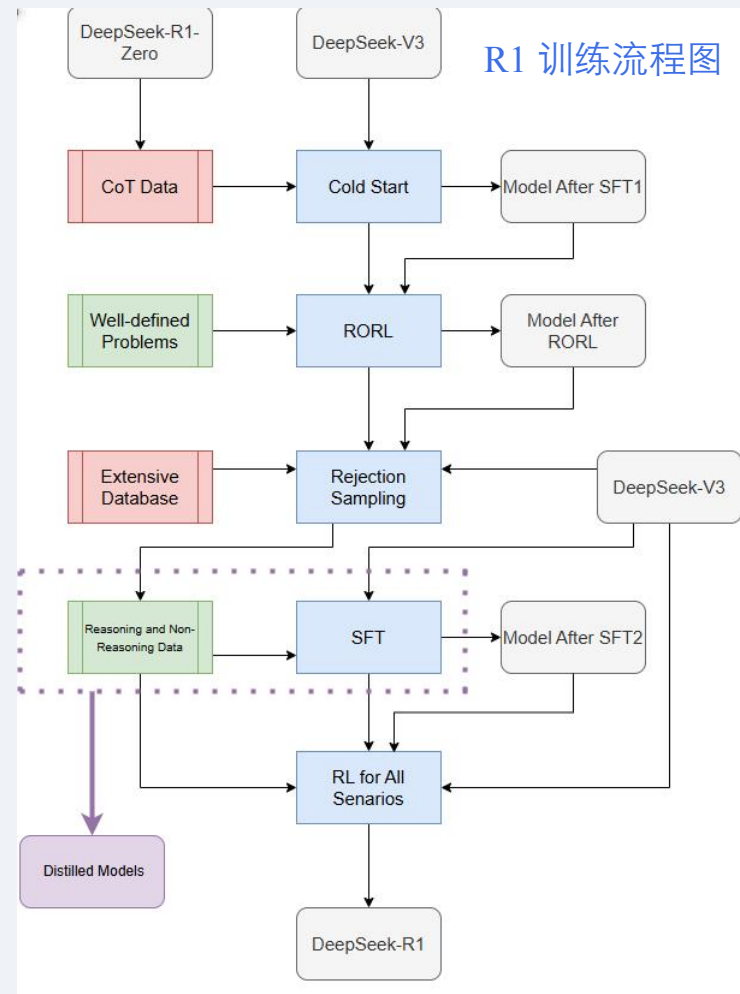
推理导向强化学习 RORL (Reasoning-Oriented Reinforcement Learning)

▣ 奖励模型 (ORM)

基于数学、编程等具有明确正误答案的任务，构建 outcome-based reward。

▣ 强化策略 (GRPO)

采用 Group Relative Policy Optimization，无需训练价值函数。通过对同一输入的多个输出进行**群组奖励归一化**估计优势值，降低计算和内存开销。



✧ 3.4.5 国内大模型产品：Qwen3

▣ 多模型类型

Qwen3系列涵盖了密集架构（Dense）和混合专家（MoE）架构的模型，参数规模从0.6亿到2350亿不等。

▣ 统一框架

整合“思考模式”（用于复杂多步推理）与“非思考模式”（用于快速上下文响应），实现动态模式切换，无需在聊天优化模型与专用推理模型间切换。

▣ 多语言支持

从原先的 29 种扩展到 119 种语言与方言，显著增强了跨语言理解与生成能力，提升全球适用性。

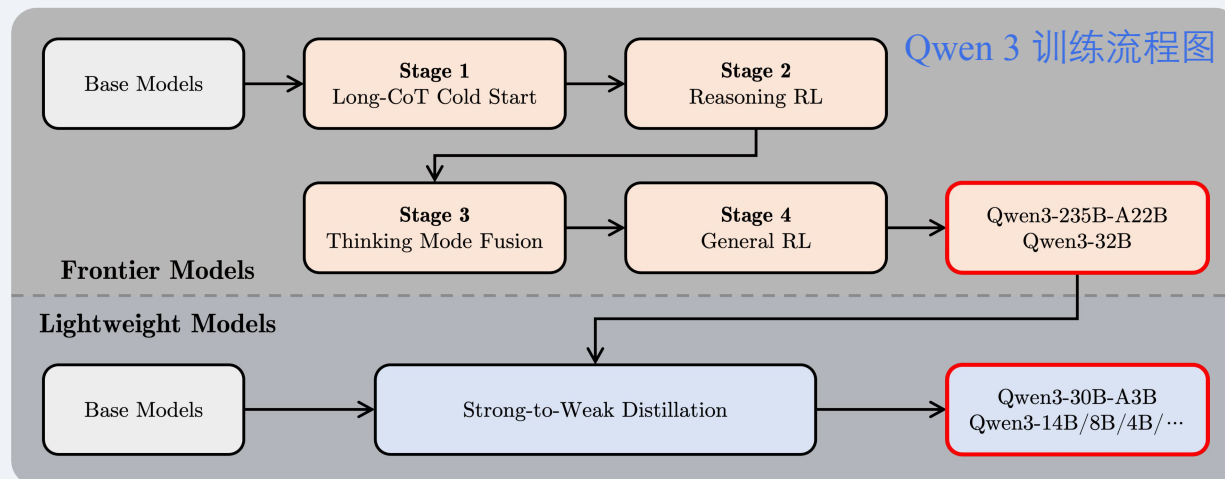


Table 9: Examples of SFT data for thinking and non-thinking modes during the thinking mode fusion stage. For the thinking mode, the /think flag can be omitted since it represents the default behavior. This feature has been implemented in the chat template¹ supported by the Hugging Face’s tokenizer, where the thinking mode can be disabled using an additional parameter `enable_thinking=False`.

Thinking Mode	Non-Thinking Mode
<code>< im_start >user</code>	<code>< im_start >user</code>
<code>{query} /think< im_end ></code>	<code>{query} /no_think< im_end ></code>
<code>< im_start >assistant</code>	<code>< im_start >assistant</code>
<code><think></code>	<code><think></code>
<code>{thinking_content}</code>	
<code></think></code>	<code></think></code>
<code>{response}< im_end ></code>	<code>{response}< im_end ></code>

3.4.5 国内大模型产品：Qwen2.5-VL

模型架构

由视觉编码器（Vision Encoder）、语言模型（LM）和多模态融合模块（Connector）三块构成。

预训练：三阶段

视觉预训练

+

多模态预训练

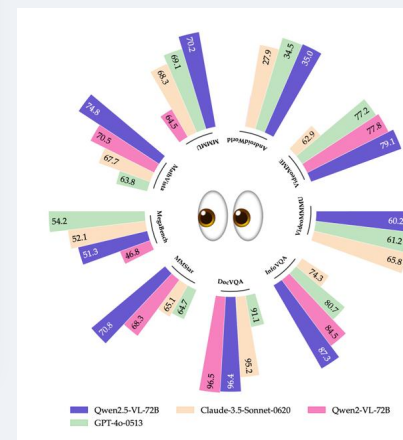
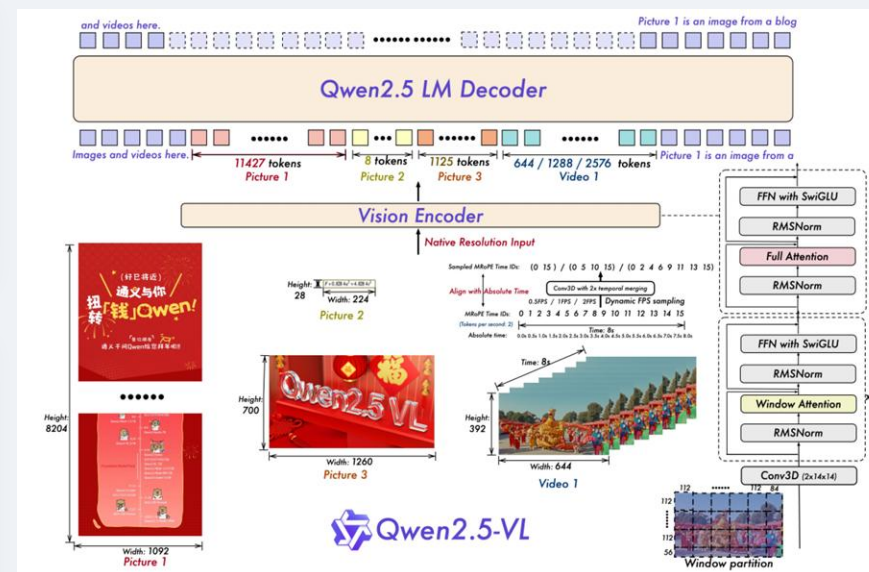
+

长上下文预训练

仅训练 ViT，使用
图像标题、视觉知
识和 OCR 数据。

解冻所有模型参数，
使用交错数据、
VQA、视频、智能
体等多种数据。

引入视频、智能体
数据，并增加序列
长度。



3.4.5 国内大模型产品：Doubao-1.5-pro

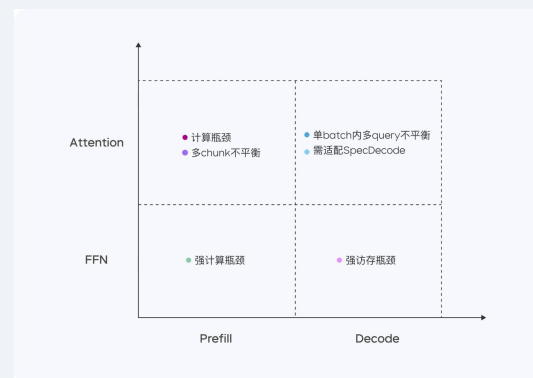
高效 MoE 模型结构

- 采用稀疏 Mixture of Experts (MoE) 架构，预训练阶段仅激活小部分参数即可达到优于大型稠密模型的性能。
- 通过研究稀疏度 Scaling Law，确定性能与计算成本之间的最优稀疏比例，实现效率与效果的平衡。

高性能推理系统

- 从计算结构上划分为四个象限：Prefill（输入处理）/ Decode（输出生成）× Attention / FFN，结合异构硬件与低精度策略优化各象限性能。
- 在降低总成本的同时，实现 TTFT（首 Token 输出时间）与 TPOT（每秒生成 Token 数）的双重最优。

	Doubao-1.5-pro	Llama3.1-405B	GPT4o-0806	Gemini-exp-1205	Claude-3.5-Sonnet-latest	Qwen2.5	DeepseekV3
Knowledge	MMLU	88.6	88.7	86.8	88.5	85.6	88.5
	MMLU_PRO	80.1	73.3	74.9	76.4	71.1	75.9
	GPQA	65.0	51.1	53.1	62.1	49.0	59.1
MATH	Math	88.6	73.8	75.9	89.7	78.3	83.1
	OlympiadBench	59.8	34.1	40.7	64.7	43.5	50.0
Code	MBPP+	78.0	72.8	78.3	78.6	76.5	79.3
	McEval	70.2	58.7	68.2	67.0	68.2	61.7
	FullStackBench	65.1	53.6	61.8	62.6	60.3	56.9
Reasoning	BBH	91.6	89.2	91.7	92.6	88.3	92.3
	DROP	93.0	91.2	79.8	89.7	88.3	87.4
Instruction Following	IFEVal	89.5	86.0	85.7	89.8	89.3	84.1
	SysBench	67.6	58.9	62.2	69.0	69.0	47.2
Chinese	CMMLU	90.9	75.4	77.3	84.3	81.2	84.3
	C-Eval	91.8	72.7	76.0	83.9	80.0	84.1



✧ 3.4.6 大语言模型的能力与应用：Beyond Chat

▣ LLM 的核心能力：自然语言推理能力

- 能在复杂语境中理解并推导隐含逻辑关系
- 可进行归纳、演绎、类比等多种推理形式
- 能利用广泛的世界知识辅助决策和规划
- 支持结构化输出

▣ LLM 的多种应用

🔧 LLM for Task Formulation

- 将自然语言问题自动转化为结构化的规划问题（如 PDDL），便于传统求解器处理。

🏠 LLM as Planner

- 基于语言推理能力在庞大状态动作空间中制定行动计划，无需显式建模。

📊 LLM as Evaluator

- 结合世界知识对状态价值进行评估，判断其与目标状态的距离，引导搜索方向。

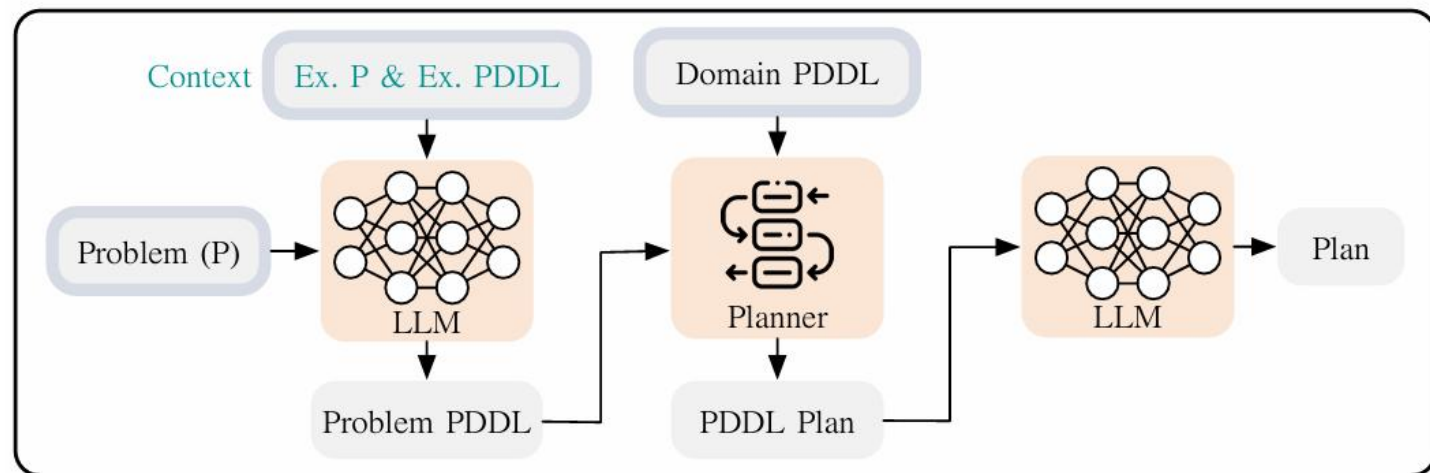
🎯 LLM as Reward Model

- 评估智能体生成动作的合理性或有效性，在缺乏明确奖励函数的情况下，提供奖励信号。

✧ 3.4.6 大语言模型应用：LLM for Task Formulation

LLM+P

- 首次将经典的规划器与LLM结合，利用LLM的自然语言处理能力与少样本学习能力，将规划问题从自然语言转换为规划领域定义语言（PDDL）格式，再借助经典规划器的高效搜索能力生成最优或正确规划，最后由 LLM 将规划结果转回自然语言。
- 与 LLM as Planner 范式相比 LLM+P 在处理长期的机器人规划问题上有着更高成功率。



LLM + P (In-context Learning)



(a) grasp bottle

(b) free gripper

(c) grasp soup can

(d) place soup can

(e) re-grasp bottle

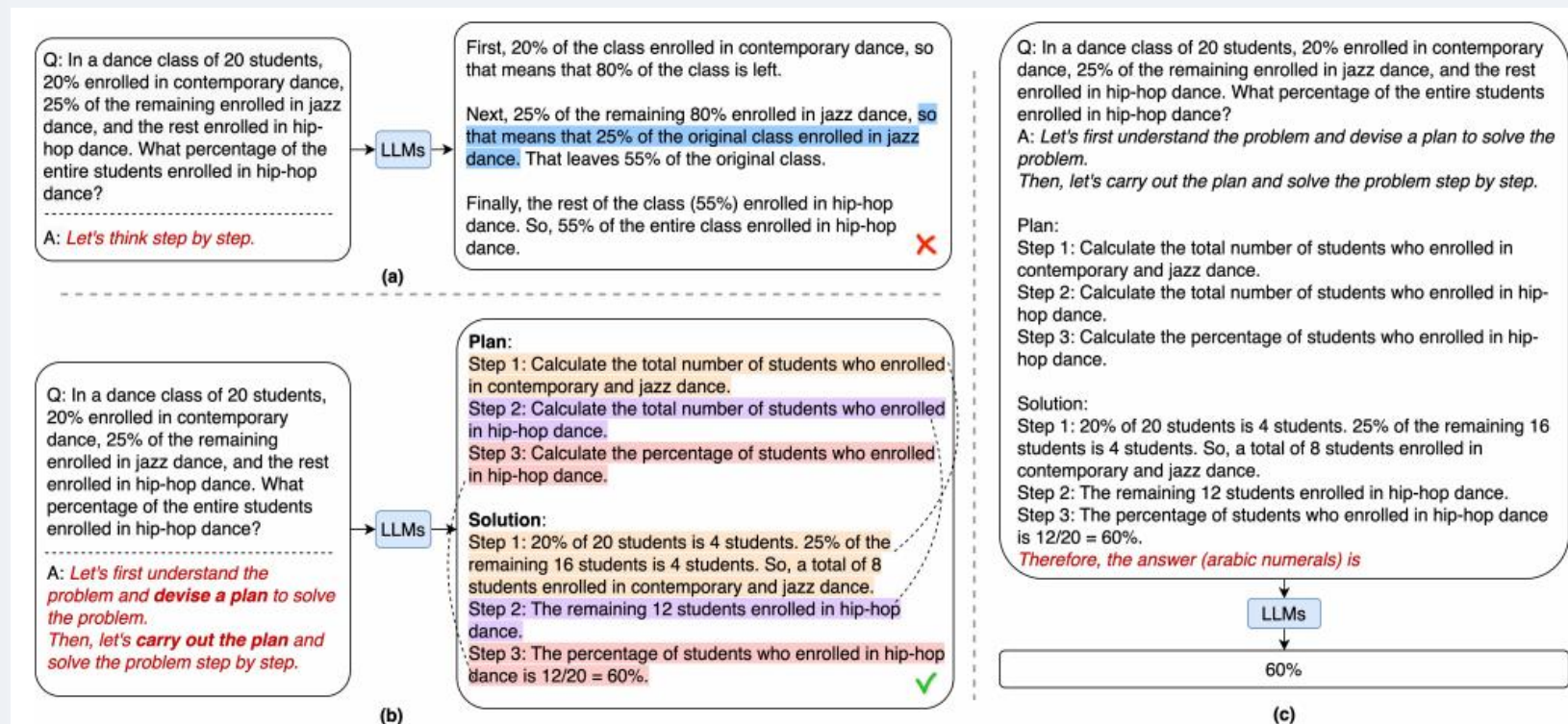
(f) place bottle

3.4.6 大语言模型应用: LLM as Planner

Plan-and-Solve

Plan-and-Solve是思维链的变体方法，它通过规划提示引导LLM在生成过程中自动完成任务分解与求解，并最终借助答案提取提示提取出最终答案。该方法属于隐式规划范式，在一次生成中完成规划与求解，无需与环境或外部模块交互，也不依赖搜索算法。

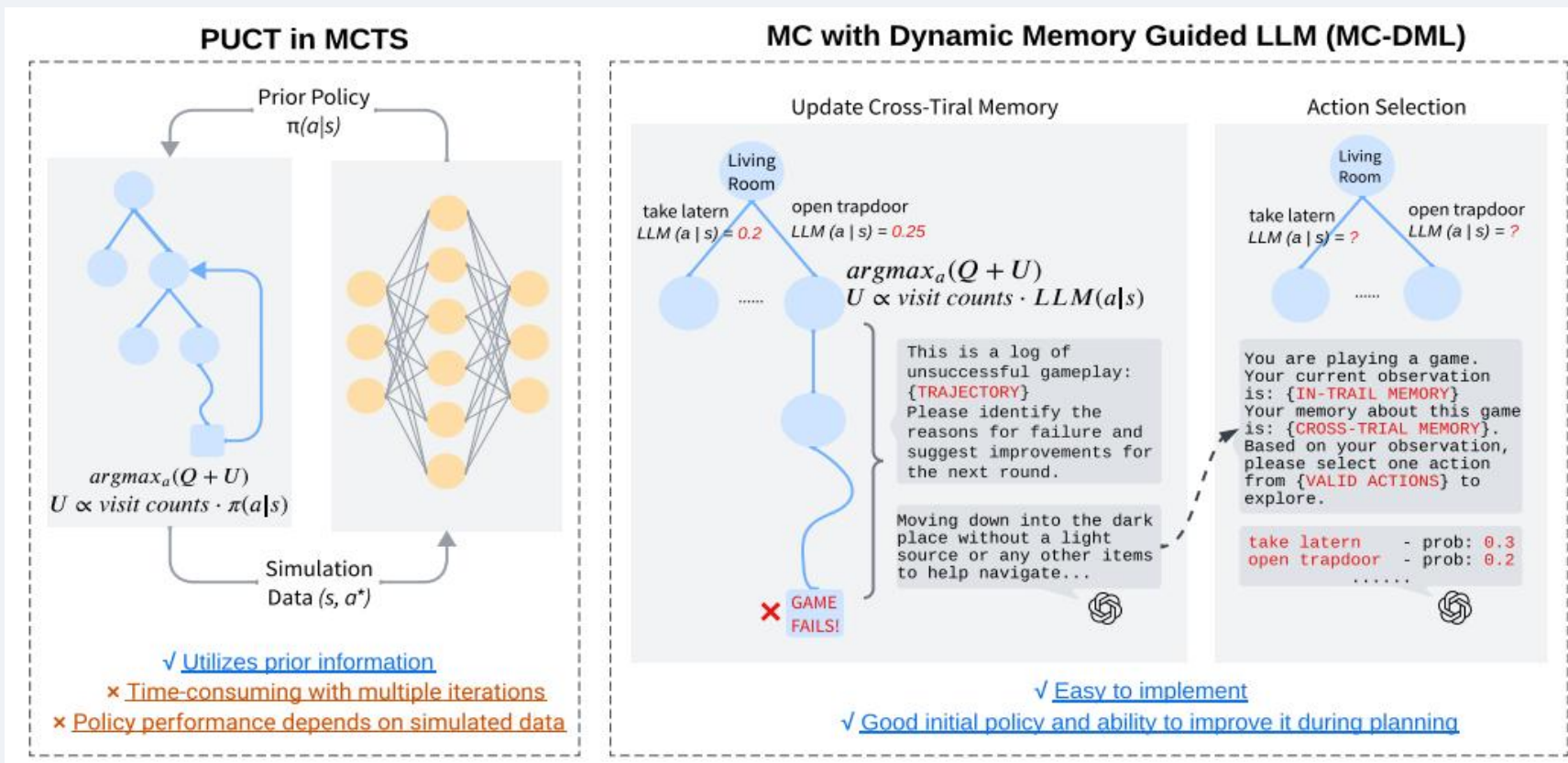
相较传统思维链，Plan-and-Solve能够有效避免推理遗漏与计算错误。在零样本设定下，该方法在算术、常识与符号推理任务中均展现出显著性能提升。



3.4.6 大语言模型应用：LLM as Evaluator

MC-DML

结合了MCTS算法与LLM，利用LLM的世界知识生成MCTS算法的先验策略，替代传统神经网络的策略估计。同时，MC-DML算法还引入了记忆机制，提示智能体根据当前交互历史以及过去的失效模式动态调整对动作的估值，从而大幅缓解了原有MCTS算法的短视问题。



✧ 3.4.6 大语言模型应用：LLM as Reward Model

结果奖励

仅在全局推理过程及给出最终答案后给予奖励反馈。

- ❑ 结果奖励模型（ORM）
- ❑ 基于规则的结果奖励：DeepSeek-R1(-Zero)

过程奖励

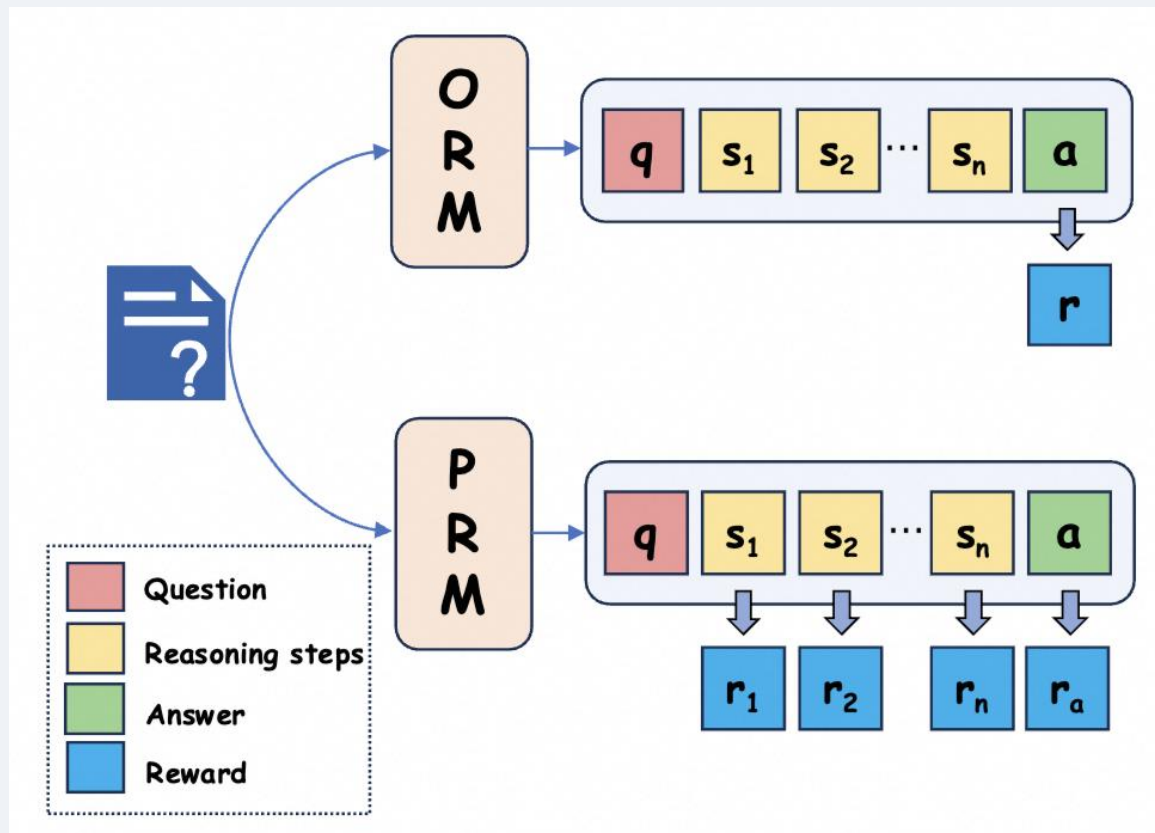
为推理过程的每一步和最终答案给予奖励反馈。

- ❑ 过程奖励模型（PRM）
- ❑ 过程奖励+结果奖励

生成式奖励

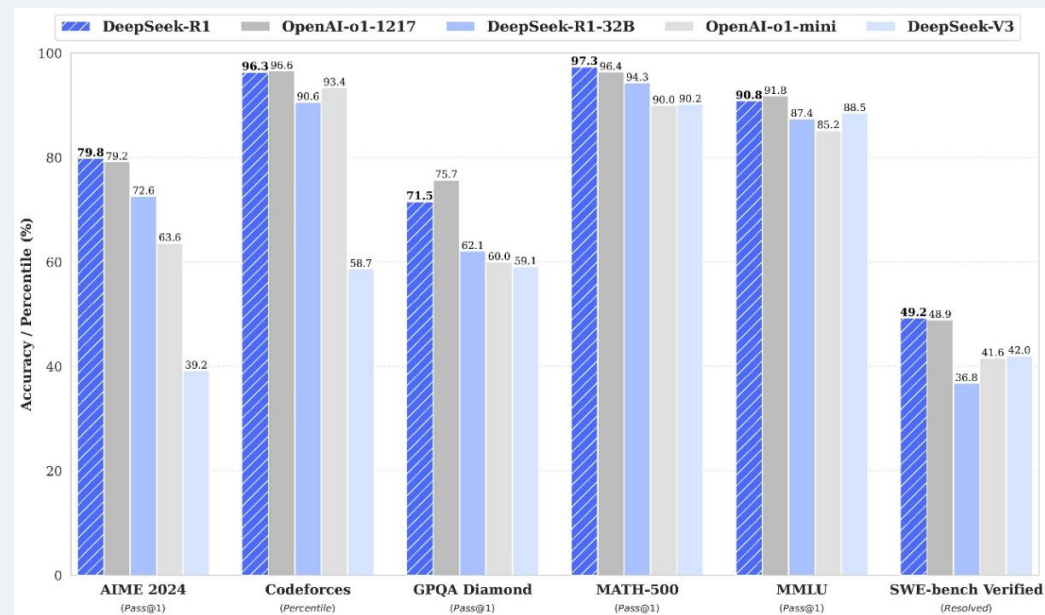
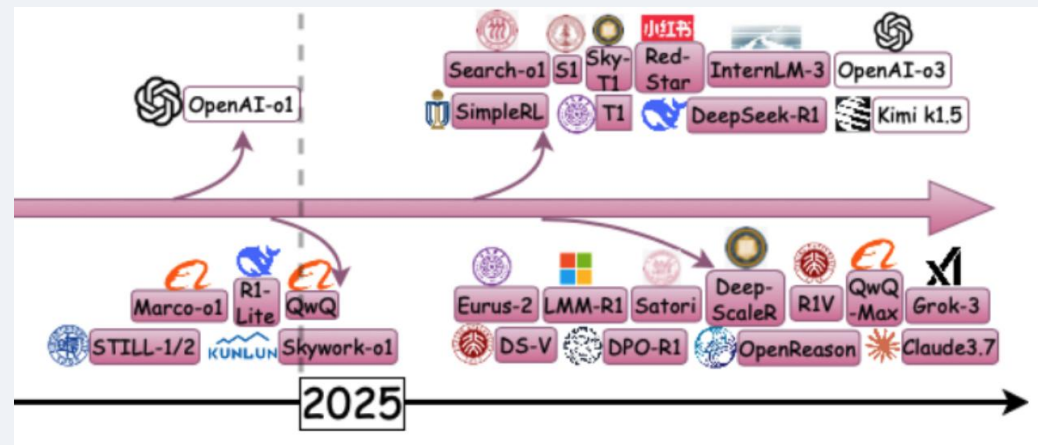
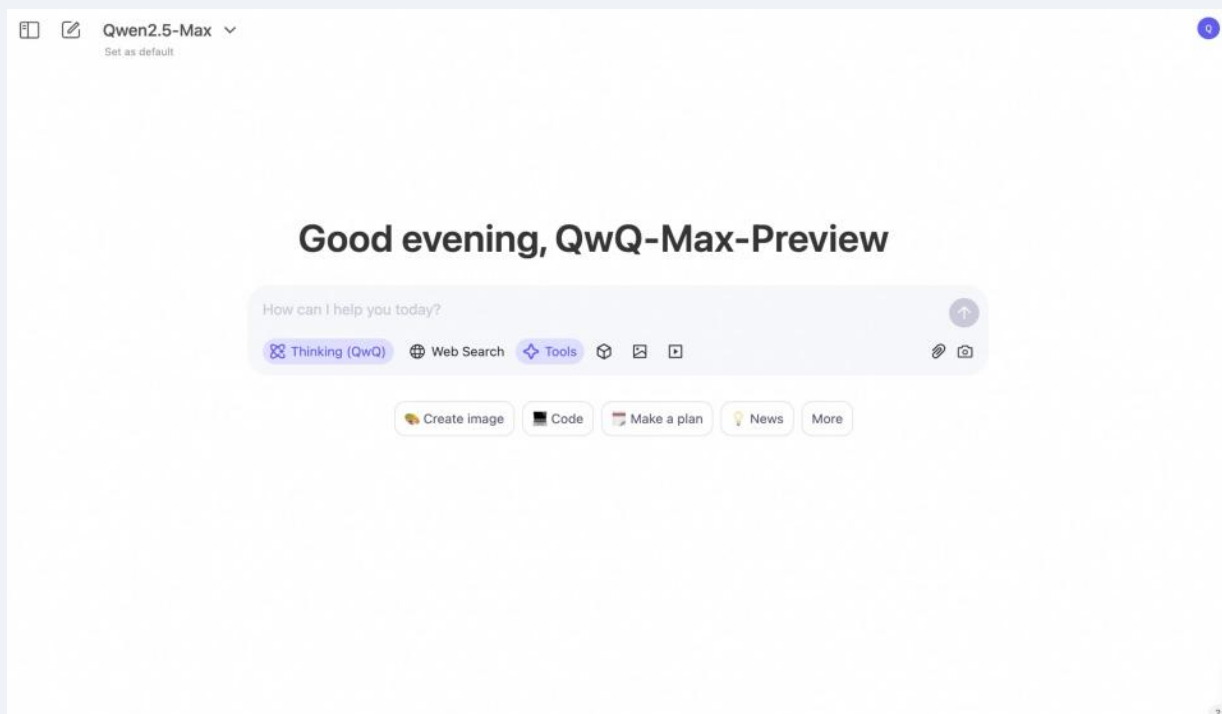
为单个或一组推理过程生成文本评价。

- ❑ 生成式奖励模型（GRM）



3.4.7 大语言模型推理

- 主流推理大模型
- OpenAI-o1&o3
- DeepSeek-R1(-Zero)
- QwQ(-max)



Guo, Daya, et al. "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning." arXiv preprint arXiv:2501.12948 (2025).

Li, Zhong-Zhi, et al. "From System 1 to System 2: A Survey of Reasoning Large Language Models." arXiv preprint arXiv:2502.17419 (2025).

✧ 3.4.7 大语言模型推理

输出行为

- ❑ 探索与规划：推理过程表现出强烈的探索行为，如制定新假设、寻求替代解决方案；
- ❑ 验证与检查：长期战略规划宏观行动+细致验证和迭代检查的微观行动，如“稍等”、“或者”等；
- ❑ 长度与时间：延长的输出长度（超过2000个token）和推理时间

训练动态

- ❑ 数据效率：仅专注于困难样本的CoT数据即可实现一定的泛化能力
- ❑ 稀疏训练：无需大规模数据集或密集奖励信号，如STILL2使用5000个蒸馏样本即可实现较好的性能；
- ❑ 参数特征：较大的语言模型（超过30B）由于其自身较强的复杂推理能力更适合推理大模型的训练。言推理能力
 - 能在复杂语境中理解并推导隐含逻辑关系
 - 可进行归纳、演绎、类比等多种推理形式
 - 能利用广泛的世界知识辅助决策和规划
 - 支持结构化输出

思维链提示

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

✧ 3.4.7 大语言模型推理

□ 大模型推理的优势

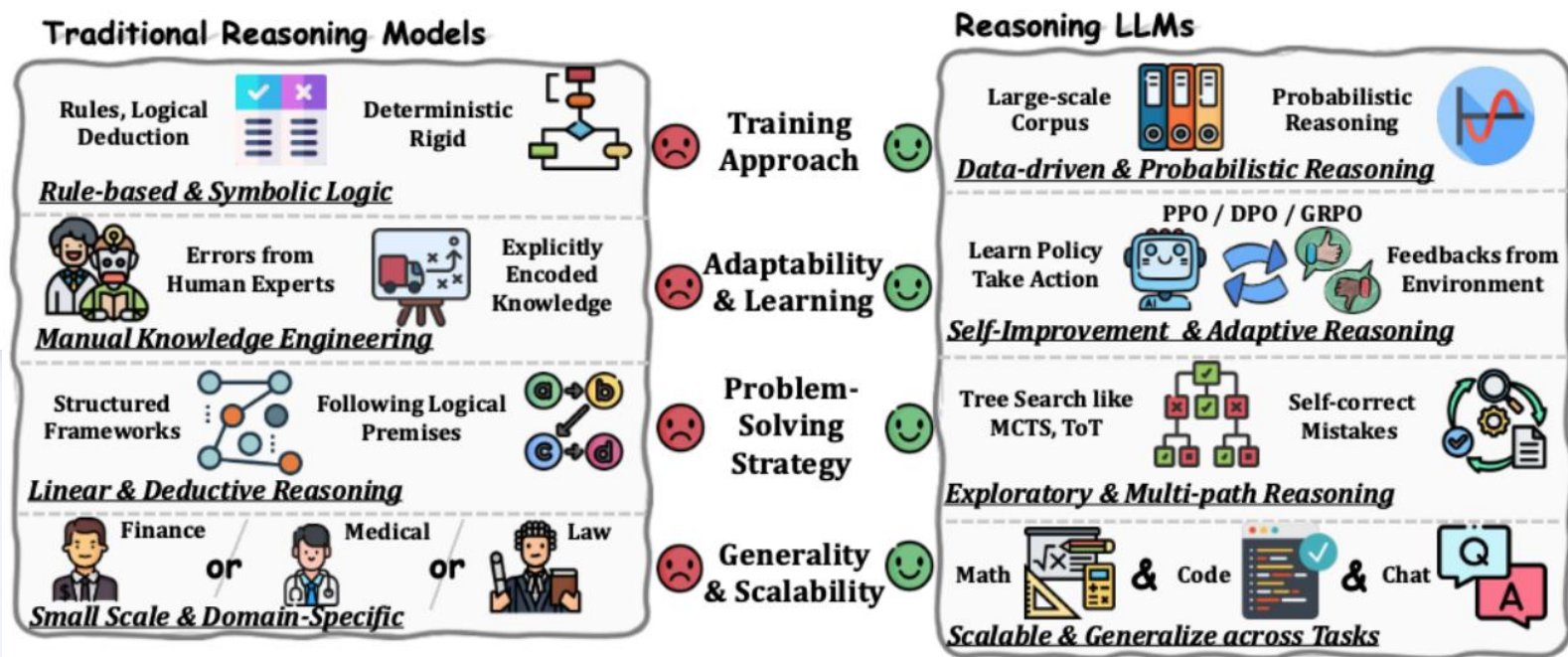
- 相比于传统AI推理方法，推理大模型通过数据驱动和概率推理的训练方式，能够实现灵活、可泛化、易扩展的推理模式。

□ 大模型推理的局限性

- 简单问题复杂化：探索不必要的推理路径，效率低下；
- 复杂问题欠考虑：处理复杂问题时逻辑严谨性不足，推理能力显著下降。

□ 如何增强大模型推理

- 微调
- 提示
- 测试时拓展
- 增强推理架构
- 推理效率提升

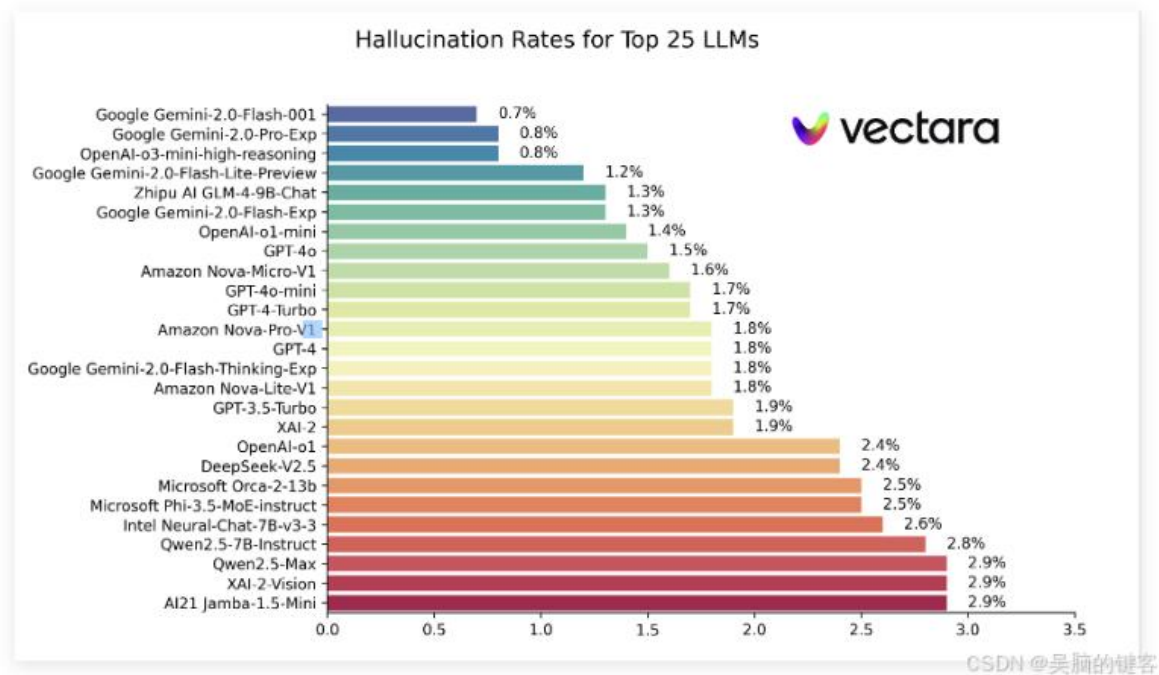


✧ 3.4.8 大语言模型的“幻觉”问题

- ❑ **大模型幻觉，也被称为 AI 幻觉**，是指大型语言模型在生成内容时，产生与事实不符、逻辑错误或无中生有等不合理信息的现象。
- ❑ 比如在回答历史事件时，可能会编造不存在的细节或人物；在进行科学知识讲解时，给出错误的理论或数据。
- ❑ 其产生原因主要包括：模型训练数据存在偏差、不完整或错误，导致在学习过程中引入了不准确的信息；模型基于概率分布生成内容，在某些情况下会选择一些看似合理但实际错误的路径。大模型幻觉会影响信息的准确性和可靠性，在信息传播、学术研究等领域可能带来不良影响。**因此，在使用大模型时，需要对其输出内容进行仔细验证和甄别。**

主流大语言模型非幻觉率排行

近日，Vectara 发布了一份名为“幻觉排行榜”的报告，比较了不同大型语言模型（LLM）在总结短文档时产生幻觉的表现。这份排行榜利用了 Vectara 的 Hughes 幻觉评估模型（HHEM-2.1），该模型定期更新，旨在评估这些模型在摘要中引入虚假信息的频率。根据最新数据，报告指出了一系列流行模型的幻觉率、事实一致性率、应答率以及平均摘要长度等关键指标。



* 非幻觉率：AI生成的回答中不包含幻觉的比例.即回答真实、准确并与正确答案一致的内容所占的比例

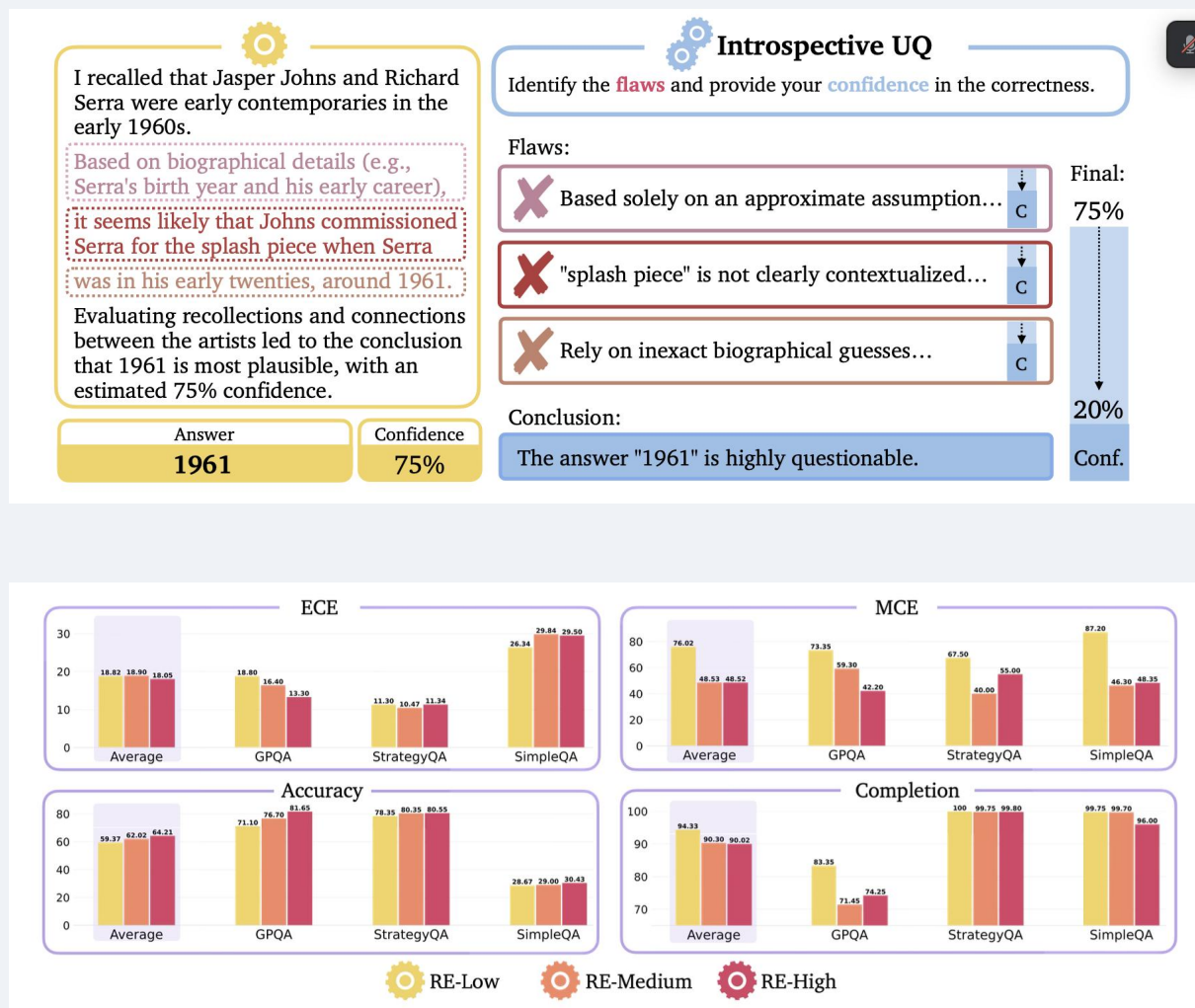
3.4.8 大语言模型的“幻觉”问题

- ❑ **问题1:** 推理模型是否校准良好？即，它们对自己的答案的自信度是否与它们的实际准确性相匹配？
- ❑ **问题2:** 更深入的推理是否能改善模型的校准？当模型被允许进行更多的思考步骤时，它们的置信度和准确性会发生什么变化？
- ❑ **问题3:** 推理模型能否通过自我反思来改善其校准？
- ❑ **实验结果:** 推理模型通常过于自信，即使在错误回答时，它们的置信度也常常超过85%。这意味着这些模型往往高估了自己答案的准确性。当模型被允许进行更深入的推理时，它们的置信度会进一步提高，但这种提高并不总是伴随着准确性的提升。



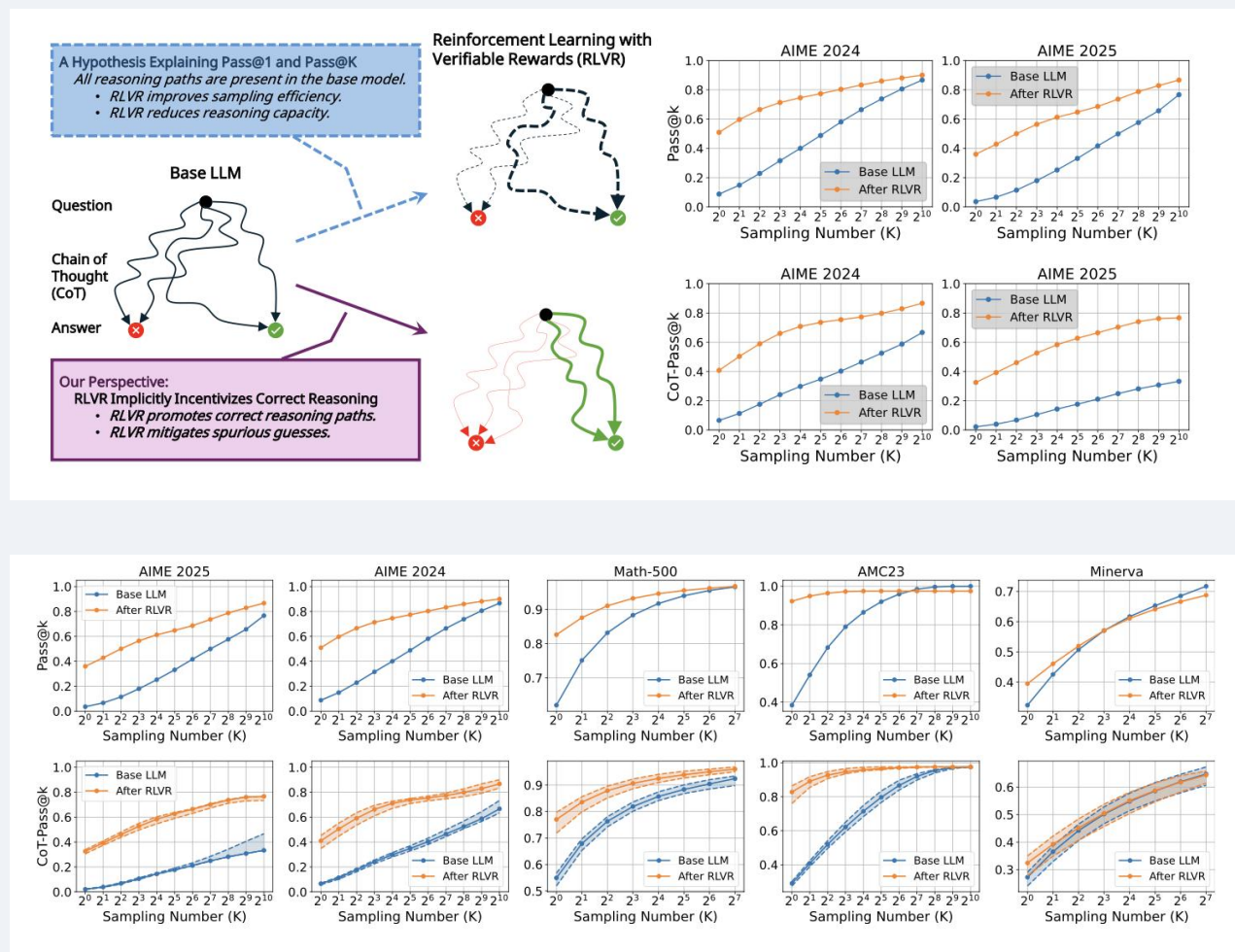
3.4.8 大语言模型的“幻觉”问题

- **内省不确定性量化 (Introspective UQ)**：该指标用于评估模型是否能够通过自我反思来改善其不确定性量化。具体来说，模型首先对问题进行推理并给出答案和置信度，然后另一个模型实例会分析第一个模型的推理过程，找出潜在的错误，并重新评估对原始答案的置信度。
- **反思效果**：内省不确定性量化的效果因模型而异。例如，o3-Mini和DeepSeek R1在经过内省后，其校准误差明显降低，这意味着它们能够更准确地表达自己答案的置信度。然而，也有一些模型，如Claude 3.7 Sonnet，在内省后校准情况反而有所恶化。



3.4.9 RL训练能否真正提升模型推理能力?

- pass@k指标的局限性：看模型在多次尝试中能否至少给出一个正确答案。只关注最终答案是否正确，而不考虑推理过程是否合理。
- cot@k指标的优势：不仅要求最终答案正确，还要求推理过程也正确。强大的语言模型作为自动验证器，来判断模型的推理过程是否正确。
- 实验结果：使用RLVR训练后的模型在CoT-Pass@K指标上表现得更好，这意味着模型的推理能力得到了真正的提升。模型从训练的最开始就在提升CoT的正确率 $P(CC|CA)$ ，而不是先学会投机取巧再学推理。
- 理论保证：统计意义上，GRPO算法偏好推理过程正确且答案正确的回答，因为答对的概率较高，而不是碰巧猜对的思维过程



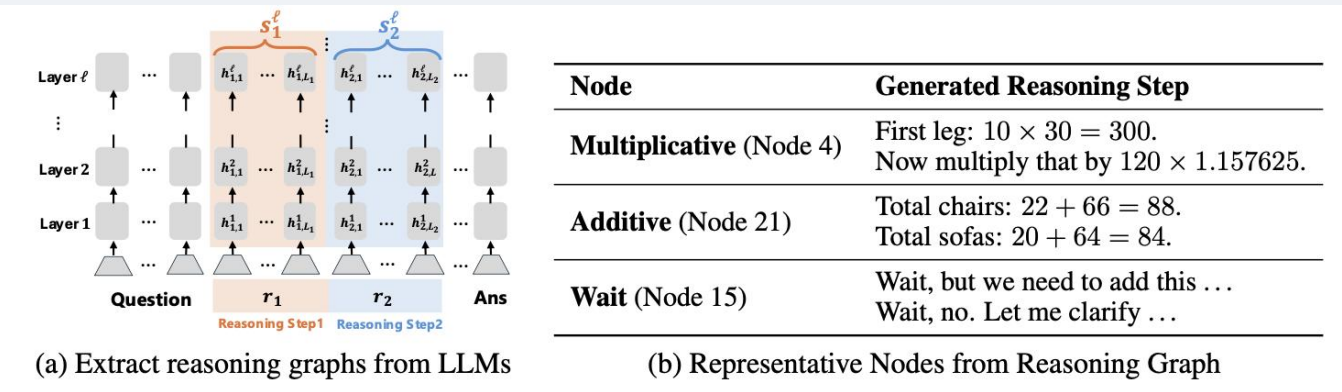
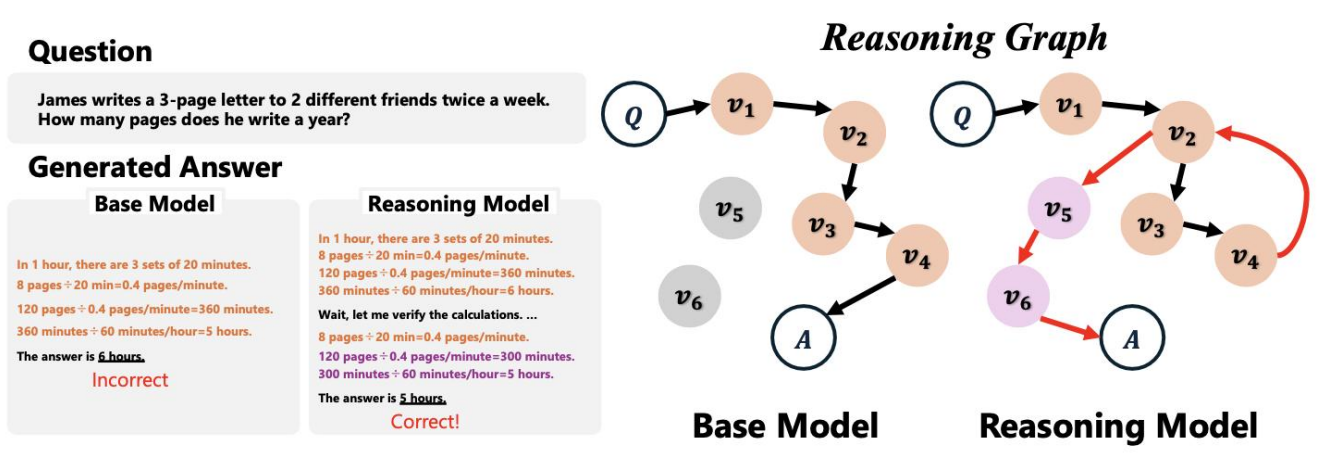
✧ 3.4.10 打开模型推理的黑盒：探究拓扑本质

❑ 从推理过程中提取推理图：

- 1. 准备数学问题数据集，提示模型生成推理步骤。
- 2. 提取每个推理步骤的隐藏状态，计算段表示。
- 3. 使用K-means聚类，聚类中心作为图的节点。
- 4. 根据模型访问节点的顺序构建边，形成推理图。

❑ 图的各项指标：

- 1. 循环检测：检测图中是否有重复访问的节点，计算循环检测比率和循环计数。
- 2. 图直径：计算图中任意两个节点之间的最大最短路径距离。
- 3. 小世界指数：评估图的聚类系数和平均路径长度，计算小世界指数。

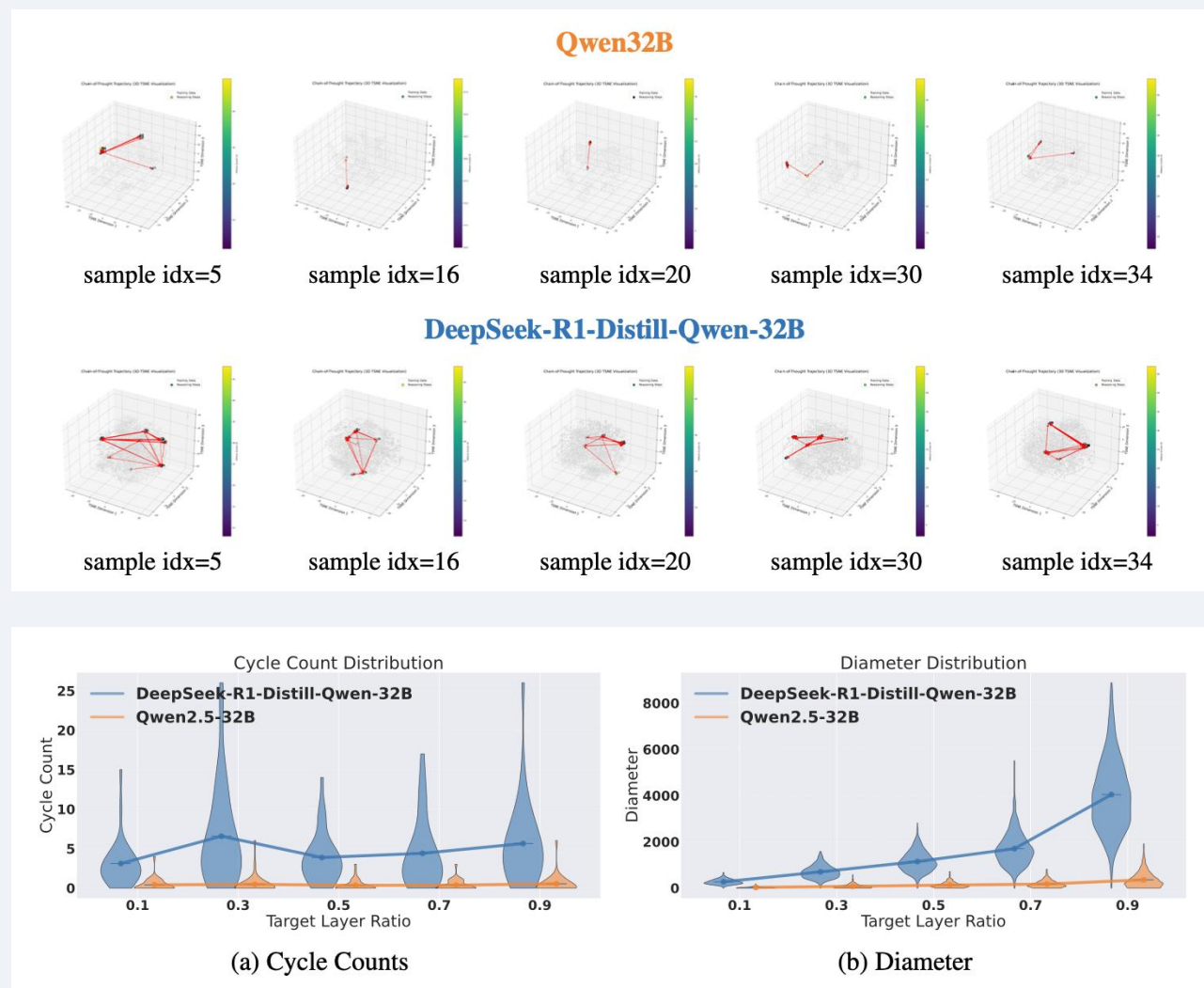


✧ 3.4.10 打开模型推理的黑盒：探究拓扑本质

□ 实验结果：

1. **循环**：大型推理模型的推理图中循环更多，这意味着它们会更频繁地回顾之前的思考步骤；
2. **直径**：大型推理模型的推理图直径更大，说明它们在思考时探索的范围更广
3. **小世界指数**：大型推理模型的小世界指数更高，这表明思考路径既紧密又高效，能快速找到关键点
4. **模型大小的影响**：通常随着模型规模的增大，这些优势会更加明显
5. **数据集的影响**：通过改进训练数据集，可以进一步扩大推理图的直径，提升模型的推理能力

□ **研究意义**：图属性，可以作为**评估和改进**模型的依据，帮助我们设计更好的训练数据，提高模型的推理能力



✧ 3.4.11 大语言模型的局限

▣ 被动响应、缺乏自主性

- 以响应提示（prompt-following）方式工作，不具备主动性
- 不具备“完成任务”的内在动机或结构
- 缺乏主动性与目标导向行为，无法自发推进任务进程

▣ 不具备执行能力

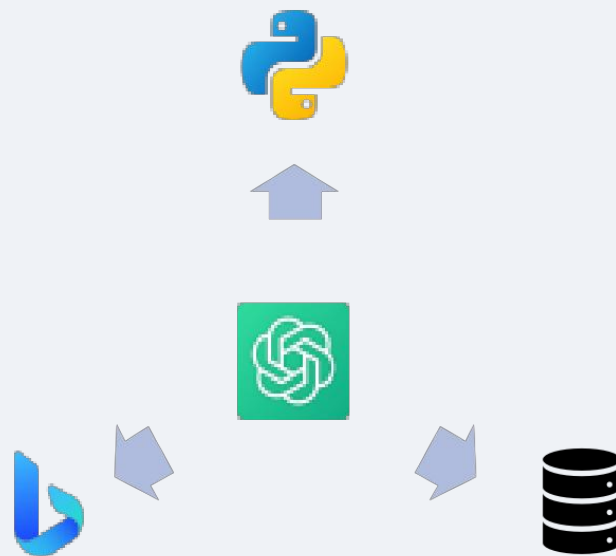
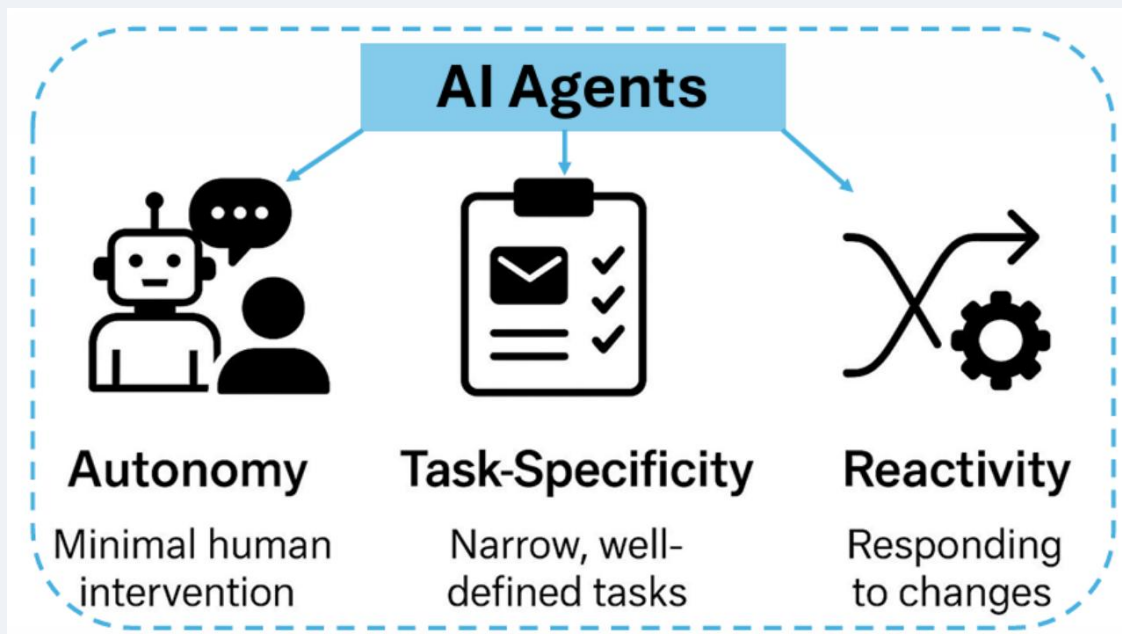
- LLM 的输出是文本，无法主动执行操作或控制系统环境
- 必须借助外部框架实现“行动”

▣ 缺乏真正的多步规划能力

- 虽然LLM 能够输出多步计划，但这些“多步规划”通常是一次性生成的静态文本
- 无法持续追踪任务状态，缺少记忆与中间反馈（除非在提示词中陈述）
- 无法感知任务是否完成，也不具备闭环调整与自我修正能力
- 幻觉问题
- LLM 可能生成不真实、虚构或错误的内容

✧ 3.5 从LLM到AI Agents

- ▣ 基础：以LLM作为核心
- ▣ 转变：从被动响应到主动完成任务，通过感知-决策-行动的闭环设计，实现任务导向型行为。
 - 工具调用
 - 记忆机制
 - 规划能力





3.5 LLM 在AI agents中的角色

核心逻辑

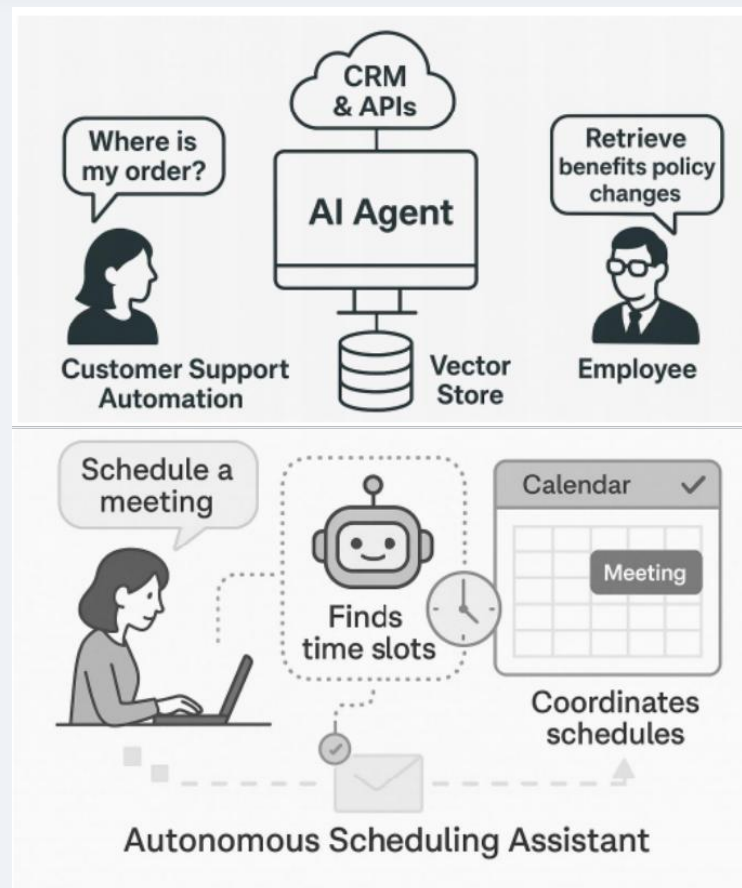
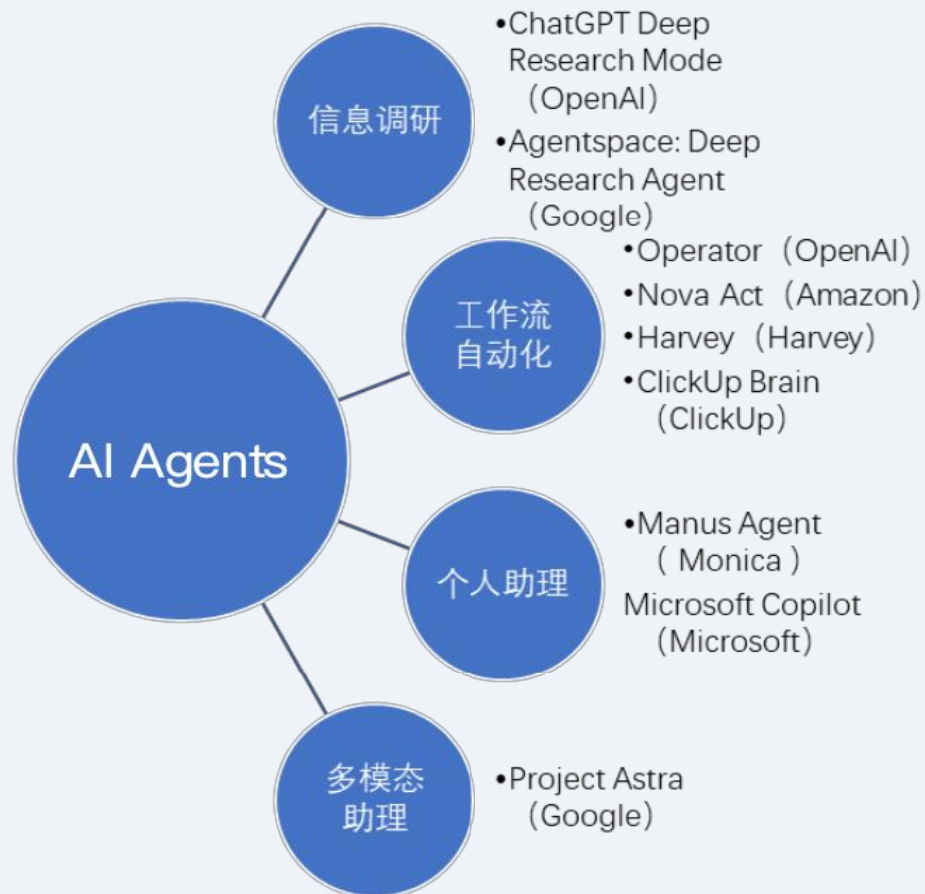
大语言模型（LLM）已成为现代AI智能体不可或缺的核心组件，它们在智能体架构中扮演着多重关键角色，从高层规划到具体执行和深层推理。

- LLM擅长将模糊目标转化为可执行的任务步骤，支持目标拆解与方案设定。它不仅可辅助传统规划模块，还能独立生成高层次的行动方案。其价值更侧重于“规划与方向”而非“执行与落地”，适合担任策略制定中枢。
- LLM本身不直接与环境交互，而是通过生成代码、API请求或操作指令，引导外部系统完成任务。它决定何时调用哪些工具，并整合反馈结果，是行动链条中的调度协调者，依赖稳定的外部接口支持。
- LLM具备强大的语言推理能力，可模拟思考路径、分析问题并修正偏差。它为智能体提供逻辑判断力、上下文理解与策略选择能力，是决策智能的基础所在，也是支撑其他模块有效协同的认知核心。

LLM角色	主要功能	机制/示例	对智能体的关键贡献
规划器 (Planner)	高层任务分解，战略决策制定	PDDL转换，计划生成，目标分解，子目标生成	目标导向性，适应性，复杂任务处理能力
执行器 (Executor)	与外部环境交互，工具调用，代码执行	API调用，函数执行，环境接口，生成文本形式动作	现实世界交互能力，任务完成能力
推理器 (Reasoner)	逻辑推理，问题分析，自我校正，知识应用	思维链 (Chain-of-Thought)，自我反思，上下文理解，异常处理	智能，连贯性，问题解决能力，决策能力

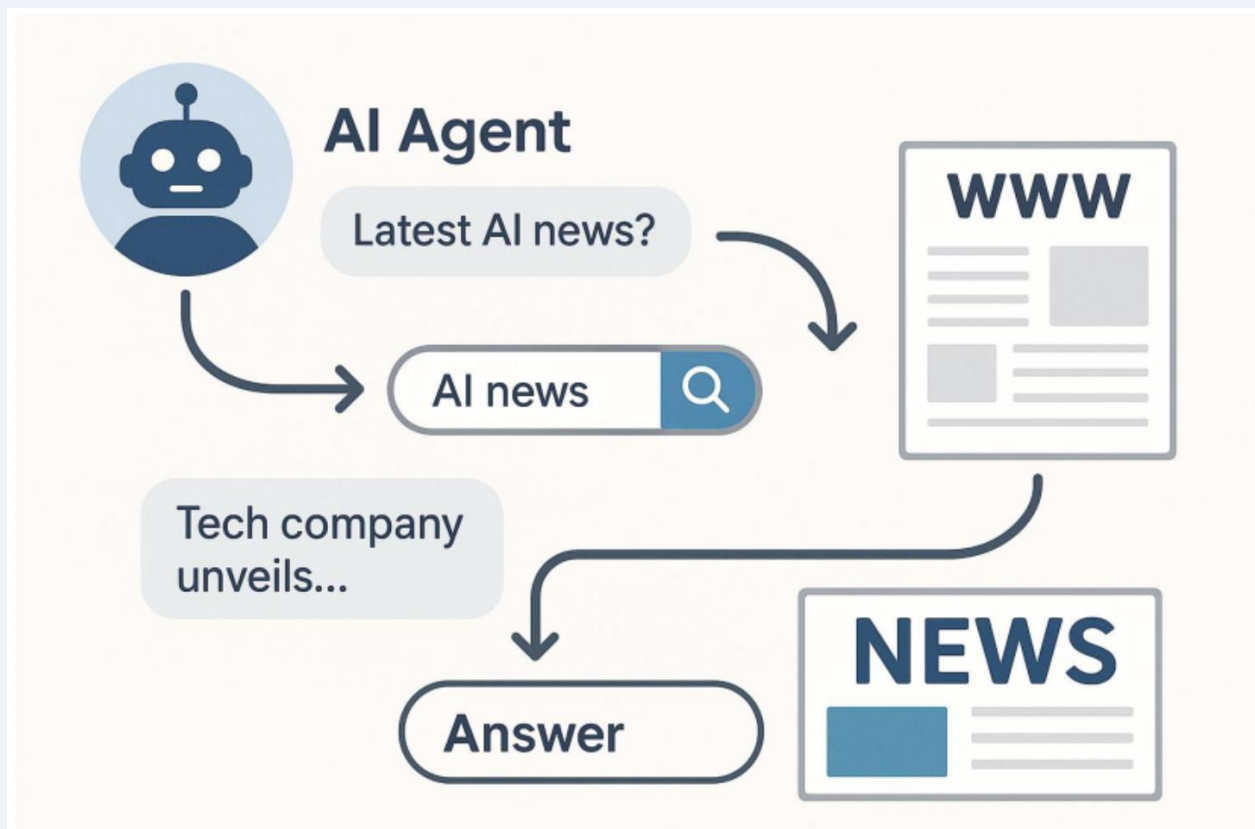
✧ 3.5.1 大模型智能体

▣ 将高层次目标转化为可执行的操作步骤，并自主完成任务。



✧ 3.5.1 大模型智能体

- ▣ 这些智能体的功能已超越传统的自然语言处理范畴，涵盖了决策制定、问题解决、与外部环境交互以及执行动作等多个方面。



核心属性

1. **自主性**
即在部署后能够以最少或无需人工干预的方式运行，通过感知输入、进行推理并实时执行动作来实现目标；
2. **任务特异性**
指它们是为狭窄、明确定义的任务而专门构建的，并针对固定领域内的重复操作进行优化；
3. **反应性与适应性**
即包含与动态输入交互的基本机制，能够响应实时刺激，并通过反馈循环或更新的上下文缓冲区随时间推移调整行为。

✧ 3.5.2 大模型智能体演进路线



✧ 3.5.3 大模型智能体核心模块

1. 规划

- 一个智能体要完成一项复杂任务，首先需要理解任务的整体目标，然后将其分解为一系列可执行的步骤，并为这些步骤制定详细的执行计划，最终确保计划能够被有效执行。
- 其中包括工具调用，这是其与外部环境交互的关键，使其能够利用外部资源扩展自身能力。

2. 记忆机制

- 有效的记忆系统使LLM智能体能够保留特定任务的长期信息以及来自实时交互的短期上下文，从而增强上下文理解和个性化体验。

3. 强化学习

- 强化学习为智能体提供了“试错—反馈—优化”的学习机制，使其能够在复杂、动态的环境中不断调整策略、提升表现。相比依赖静态数据训练，RL更适合处理长时程任务、实时决策和行为对齐问题，是实现智能体自主性与适应性的关键支撑模块。



✧ 3.5.4 大模型智能体「规划」模块

问题建模

- 人工分解：人工对具体任务或任务类型进行分解，明确定义规划问题的动作与状态。
- 外部工具分解：借助外部工具来形式化问题中的动作与状态。
- LLM自主分解：LLM参照prompt中的示例或依赖自身的世界知识自行对具体任务进行分解。

workflow

- 隐式规划：将规划过程融入到模型的生成过程中，一步生成规划结果。
- 逐步规划：根据当前状态以及来自环境或评估器的反馈，逐步生成下一步行动。
- 分层规划 (Agentic AI)：将任务划分为上层规划与下层执行两个层级乃至更多的层级：通常由规划智能体负责将整体任务拆解为多个步骤，执行智能体则逐个完成具体子任务。在执行过程中，系统可根据环境反馈动态调整上层规划。
-

搜索算法

- 蒙特卡洛树搜索
- 广度优先搜索
- 深度优先搜索
-

工具调用

- 网页搜索
- 代码执行
- 文档编辑器
-
- 知识库检索
- 多模态工具
- 外部软件接口



3.5.4

典型智能体架构

核心逻辑

这些系统代表了语言模型智能体从工具调用走向完整自治架构的不同路径，在推理、感知、记忆与执行之间形成闭环，核心能力均围绕“如何规划并完成复杂任务”展开设计。

- 当前智能体系统的发展日益聚焦于“如何自主理解目标并制定行动方案”。
- 以下四种架构在任务分解、推理控制与工具调用等方面形成差异化探索，代表了以“规划能力”为核心的智能体构建范式：

系统名称	系统定位	主要特点与机制	与Agent的关联性说明
Chameleon	多模态智能体框架	整合图像、文本、UI等多模态输入，基于LLM进行环境理解与行动选择	支持感知-决策-执行闭环，强化具身智能与环境交互能力
DeepSeek-Prover-V1.5	自动证明系统	基于LLM的程序验证工具链，结合指令微调与CoT完成复杂逻辑推理	属于目标导向型智能体，具备结构化推理与策略生成能力
ReAct	推理-行动集成框架	提出将思维链（Chain-of-Thought）与工具使用结合的Agent提示范式	启发了LLM自主行动机制设计，实现感知—思考—行动协同
RAG-Star	自反式知识增强智能体	将RAG（检索增强生成）与自我反思机制结合，强化长期任务处理能力	实现信息利用+自我调整，体现Agent的学习性与适应性提升

3.5.4 大模型智能体「规划」模块

Chameleon

- 一种即插即用的模块化推理框架，采用隐式规划的问题求解范式。
- 核心组件：一个工具清单和一个LLM规划器

规划器将原始任务分解为多个子任务，并根据工具清单生成工具调用流。随后，Chameleon按照规划好的流程依次调用相应的工具来解决这些子任务。

- Chameleon能够灵活组合LLMs、视觉模型、搜索引擎、Python函数等工具以应对复杂任务。

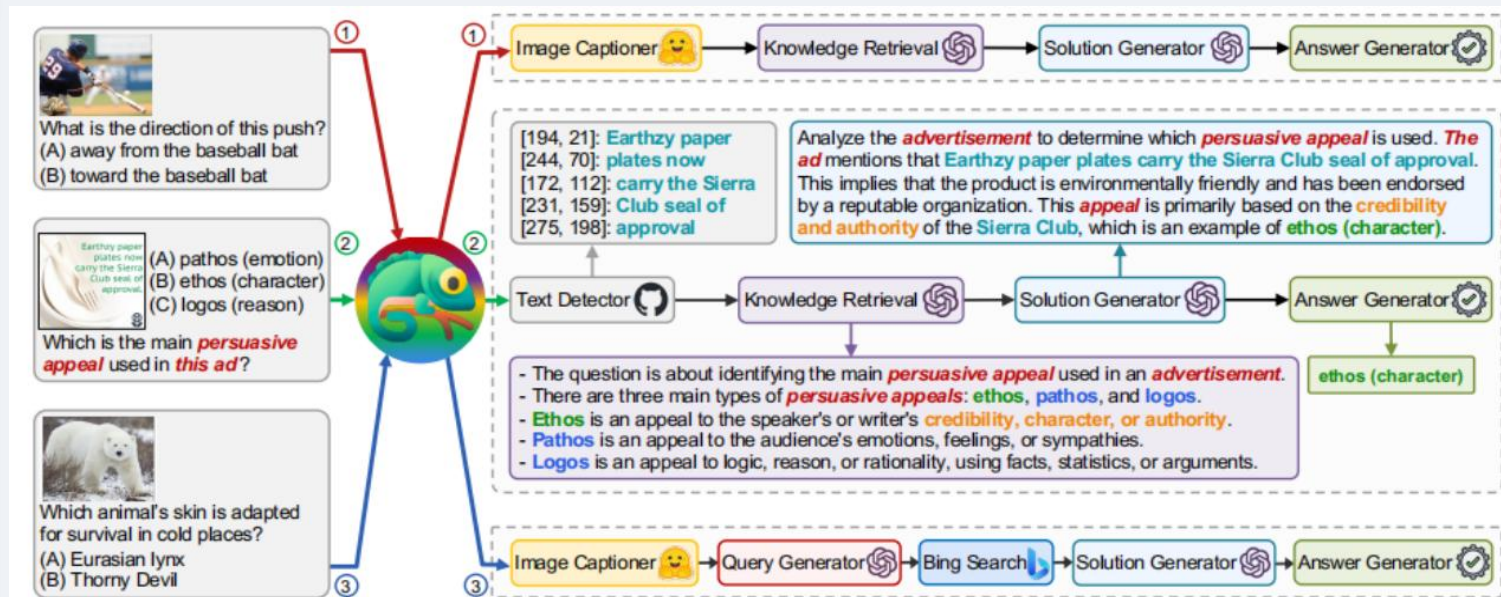


Figure 1: Examples from our Chameleon approach with GPT-4 on ScienceQA [32], a multi-modal question answering benchmark in scientific domains. Chameleon is adaptive to different queries by synthesizing programs to compose various tools and executing them sequentially to get final answers.

✧ 3.5.4 大模型智能体「规划」模块

DeepSeek-Prover-V1.5

- 结合了定理证明工具Lean与蒙特卡索树搜索算法（MCTS），采用逐步规划范式来解决形式化的定理证明问题

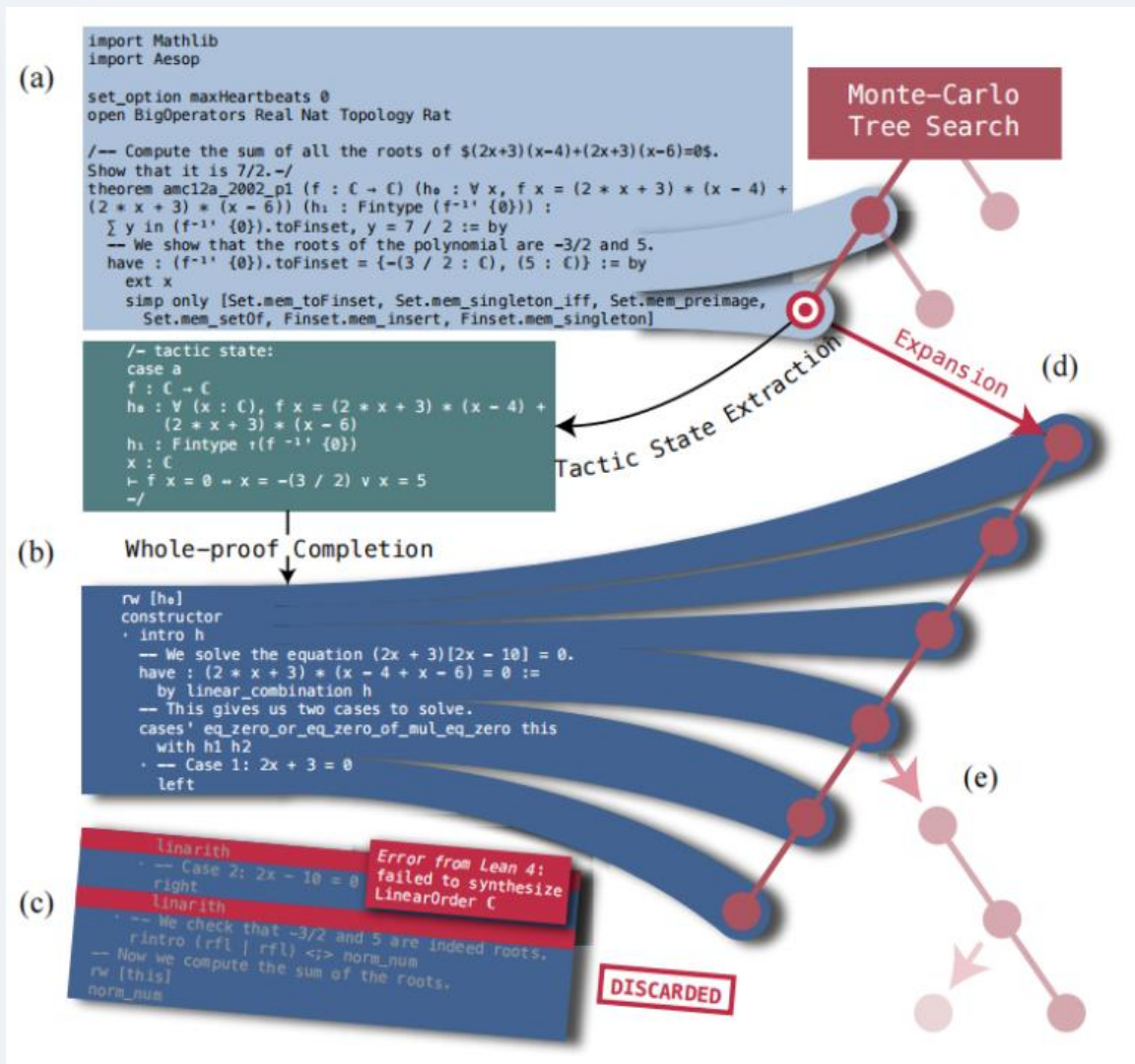
Lean被用于形式化表示定理证明过程与证明目标，MCTS则用于控制证明搜索树的扩展与回溯。

- 在后训练阶段

采用SFT与GRPO对预训练模型进行领域微调，使其更好地适应Lean环境下的定理证明任务。

- 在推理阶段

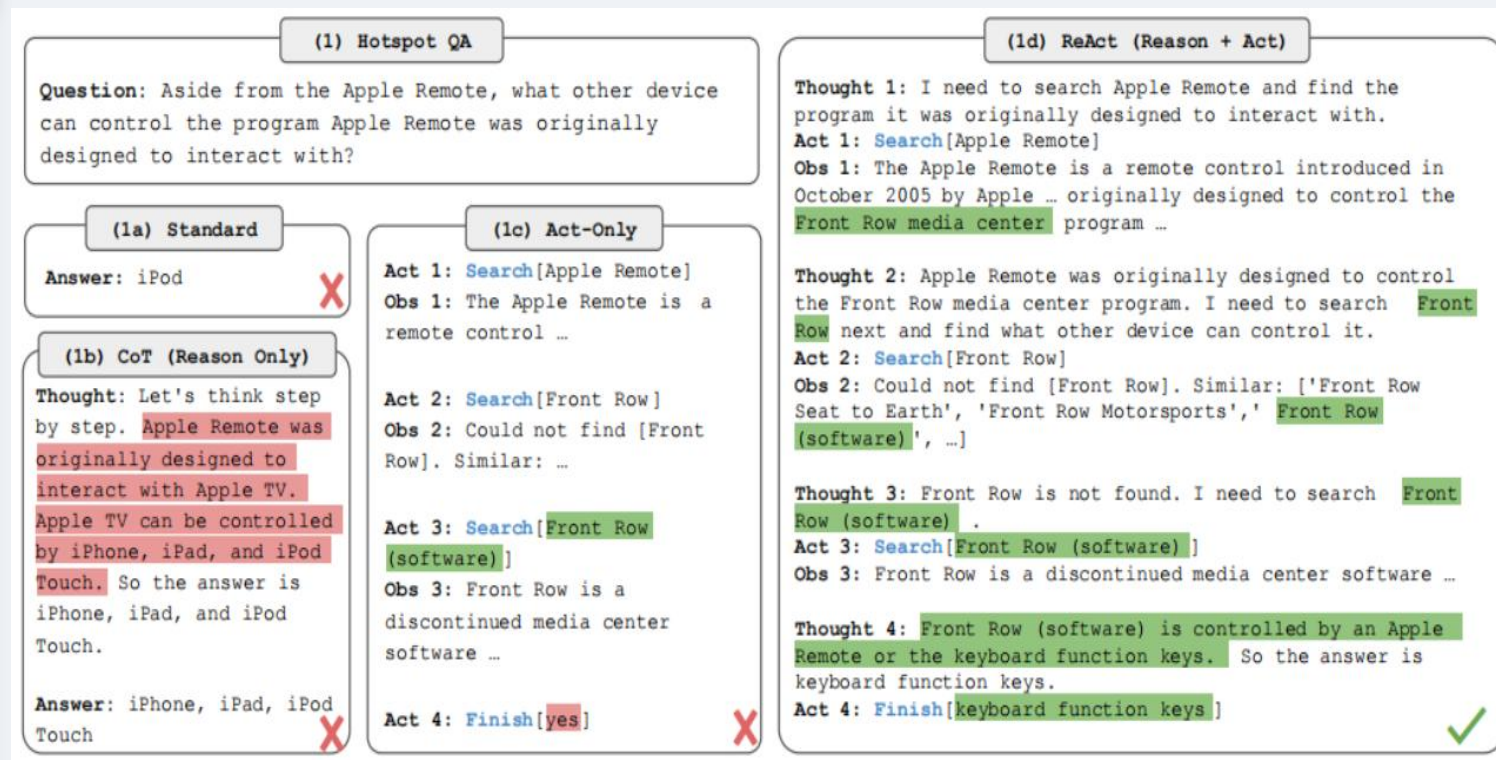
利用Lean抽象出定理证明过程中的状态，借助 MCTS 方法引导搜索过程，并采用“中断 - 重启”策略对搜索树进行有效扩展。此外，为增加探索性并缓解奖励稀疏问题，还引入了人工设计的内在奖励机制。



3.5.4 大模型智能体「规划」模块

REACT

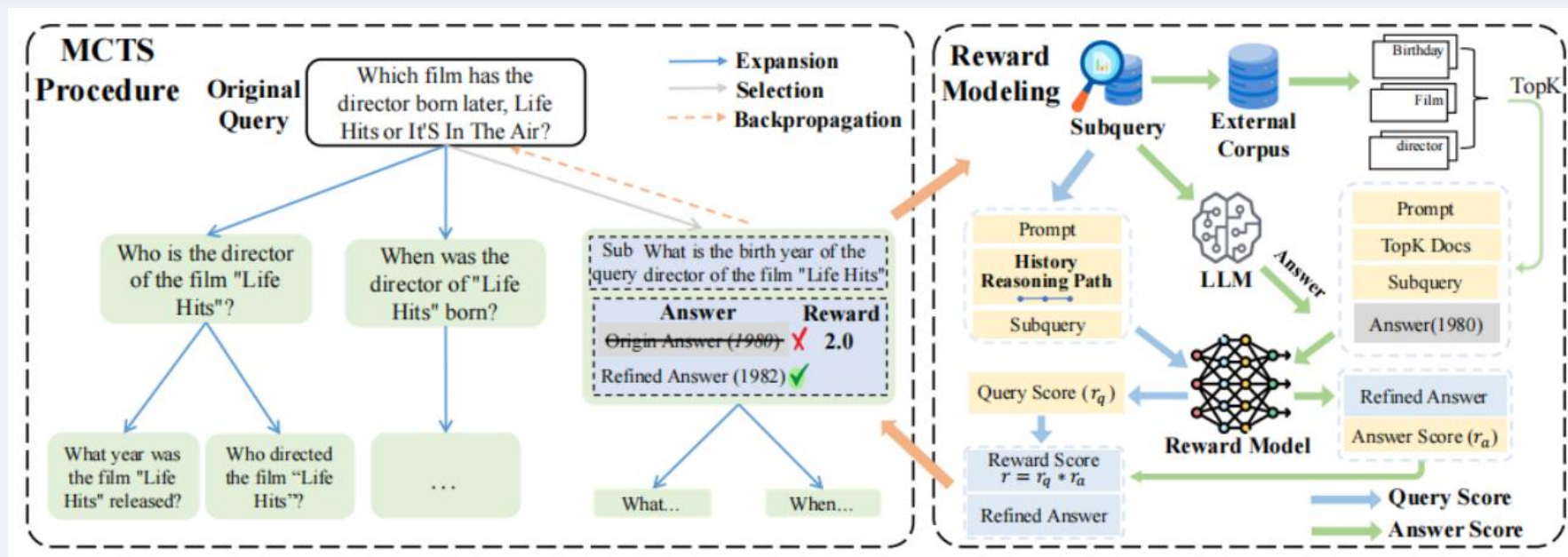
- 一种融合推理与行动的逐步规划方法，其核心在于推理与行动的交替进行。通过这种交替方式，LLM可以在解决问题的过程中不断地调整与优化自己的工作记忆和行动。
- 在需要与外部环境交互的长期规划任务中，REACT方法使得LLM可以根据环境反馈及时调整行动策略，从而更好地适应复杂多变的任务需求。不仅提升了模型的性能与泛化能力，还增强了模型在各类任务上的可解释性、可信度和易诊断性。



✧ 3.5.4 大模型智能体「规划」模块

RAG-Star

- 一种面向开放域多步问答任务的逐步规划方法，采用树状子查询生成与检索增强型验证机制相结合的方式，引导LLM自主将原问题拆解为多步子问答。每个子问答被视为一个搜索状态，并通过MCTS引导搜索过程，最终基于完整的问答历史综合生成最终回答。
- 同时将LLM奖励模型与RAG融合，用以评估子问答的价值、验证并优化其答案。该方法有效融合了外部知识与搜索过程，利用外部知识引导搜索过程并仅在必要时修正LLM的回答，从而避免内外部知识冲突。



✧ 3.5.4 大模型智能体「规划」模块

❑ 当前，大型语言模型（LLM）智能体的规划与任务分解能力是实现其在复杂场景中自主决策的核心。根据实现方式和控制程度，这些方法可被归纳为以下几种演进中的范式：

任务分解范式的转变

从早期依赖专家规则或外部工具链完成精确任务拆解（如基于PDDL的规划系统），逐步演进为由LLM直接完成自然语言目标的自动化分解。这一范式兼具灵活性与泛化性，虽在稳定性与一致性方面尚有挑战，但显著提升了任务适应能力与跨任务迁移的可能性。

规划机制的迭代优化

任务规划不再仅停留在单轮推理中的隐式链式思维（CoT），而是向具备交互反馈、自我修正与多轮策略更新能力的显式规划机制演化。结合强化学习（RL），智能体能够在试错中优化长时程策略，提升任务完成率，并实现更加稳健的决策调整与行动控制。

任务分解范式的转变

① 架构自动搜索

从人工设计固定工作流，转向在模块化空间中自动化搜索最优智能体架构。

② 去中心化协同

在多智能体系统中，摒弃中心控制器，通过去中心化演化协调实现自主专业化与任务分配。



3.5.5

大模型智能体「记忆机制」模块



- LLM Agent相较传统LLM的一个重要区别在于LLM Agent拥有记忆（Memory）机制。
- 相比于没有记忆机制的智能体，具备记忆机制的智能体拥有以下核心优势：

特性	无记忆机制的智能体 (Stateless Agent)	具备记忆机制的智能体 (Stateful Agent)
交互模式	无状态 (Stateless)：每次交互都是独立的，无法关联历史信息。	有状态 (Stateful)：能够理解并利用历史上下文，进行连贯的多轮交互。
任务能力	仅限于单轮、简单的指令，无法处理需要多步骤或长时程跟踪的任务。	能够执行复杂、长时程的任务，如项目规划、持续性研究和个性化助理。
用户体验	通用且重复：对所有用户提供相同的、无差别的响应，可能重复提问或犯错。	个性化与自适应：能够“记住”用户偏好和历史，提供量身定制的服务，体验持续优化。
学习与进化	静态不变：模型能力固定，无法从与用户的互动或任务经验中学习和成长。	动态进化：通过经验积累和反思，能够持续学习、优化策略并获得新技能。
运行效率	效率低下：可能进行大量重复性计算和工具调用，导致时间和资源浪费。	高效经济：通过缓存中间结果和成功经验，避免重复工作，降低成本。

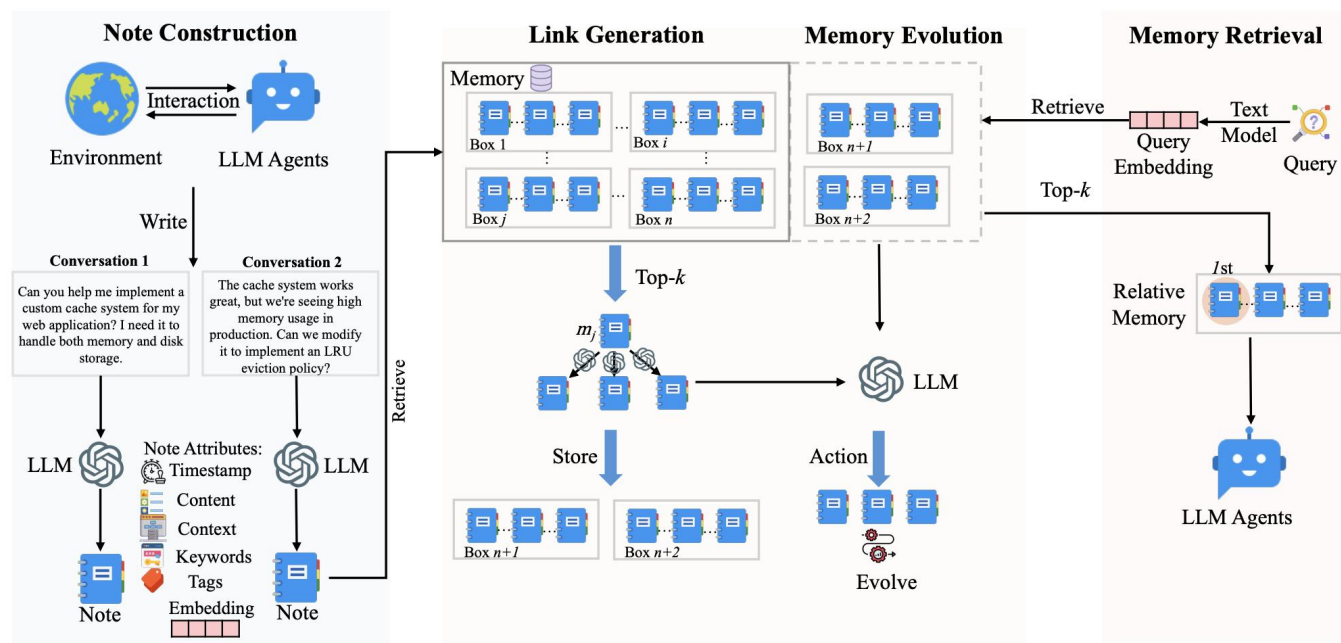
✧ 3.5.5 大模型智能体「记忆机制」模块

A-Mem(Agentic Memory)

- ❑ 核心思想源于Zettelkasten（卡片盒笔记法）。A-Mem将此方法与LLM的智能决策能力相结合，创建了一个能够自主组织和动态演化的记忆网络。
- ❑ A-Mem通过记录新知识 → 关联旧知识 → 在关联中深化理解 → 在需要时提取最相关的知识来解决问题的流程实现了代理性、动态演化、网络化组织和高效率。

系统架构的革新A-Mem的工作流

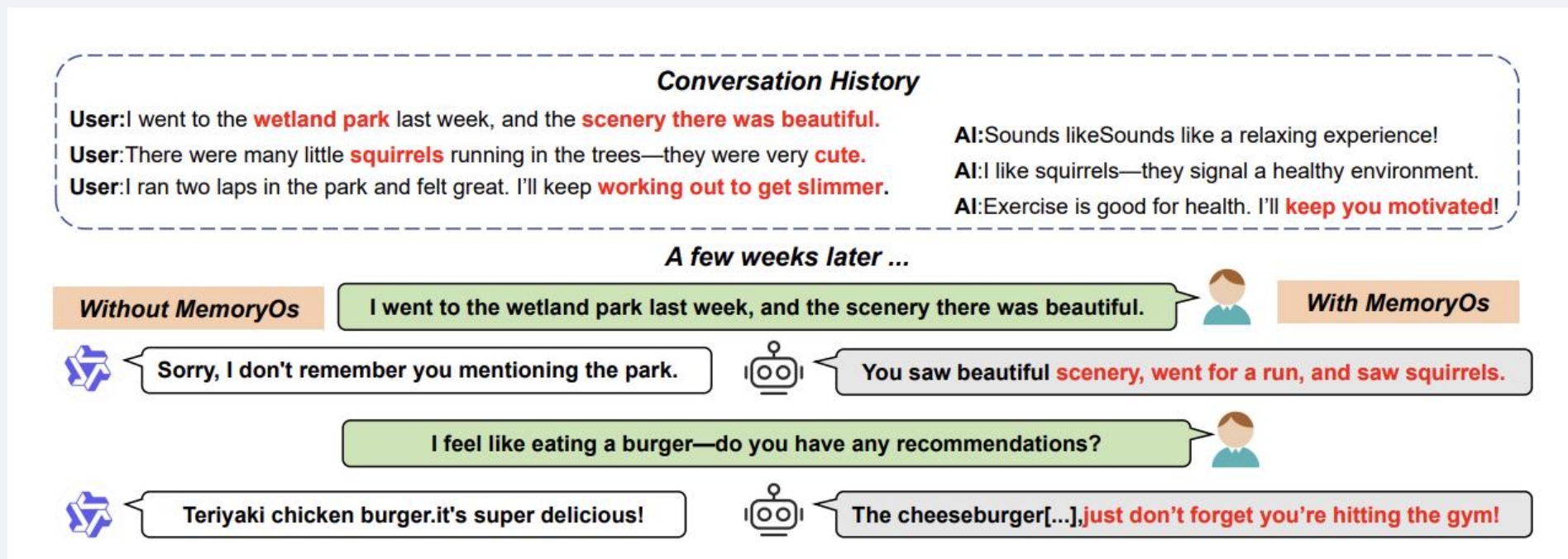
- 笔记构建 (Note Construction): 从经历中提取重点, 构建笔记
- 链接生成 (Link Generation): 将相关的笔记关联在一起
- 记忆演化 (Memory Evolution): 新笔记加入时, 用新引入的知识更新旧笔记
- 记忆检索 (Memory Retrieval): 当遇到新问题, 在记忆中检索



✧ 3.5.5 大模型智能体「记忆机制」模块

MemoryOS

- ❑ MemoryOS构建了一个模仿计算机操作系统内存管理原理的“内存操作系统”。这个系统通过一个协同工作的框架来全面管理AI智能体的记忆。
- ❑ 通过其分层架构（短时、中程、长程）存储对话记忆，根据对话热度和预设规则动态更新并迁移这些记忆，然后从各层级检索相关信息，最终整合成上下文以生成连贯且个性化的回复。



3.5.5 大模型智能体「记忆机制」模块

MemoryOS

该框架包含了四个模块：

❑ 内存存储 (Memory Storage)

负责以层级化结构组织和存储信息

❑ 内存更新 (Memory Updating)

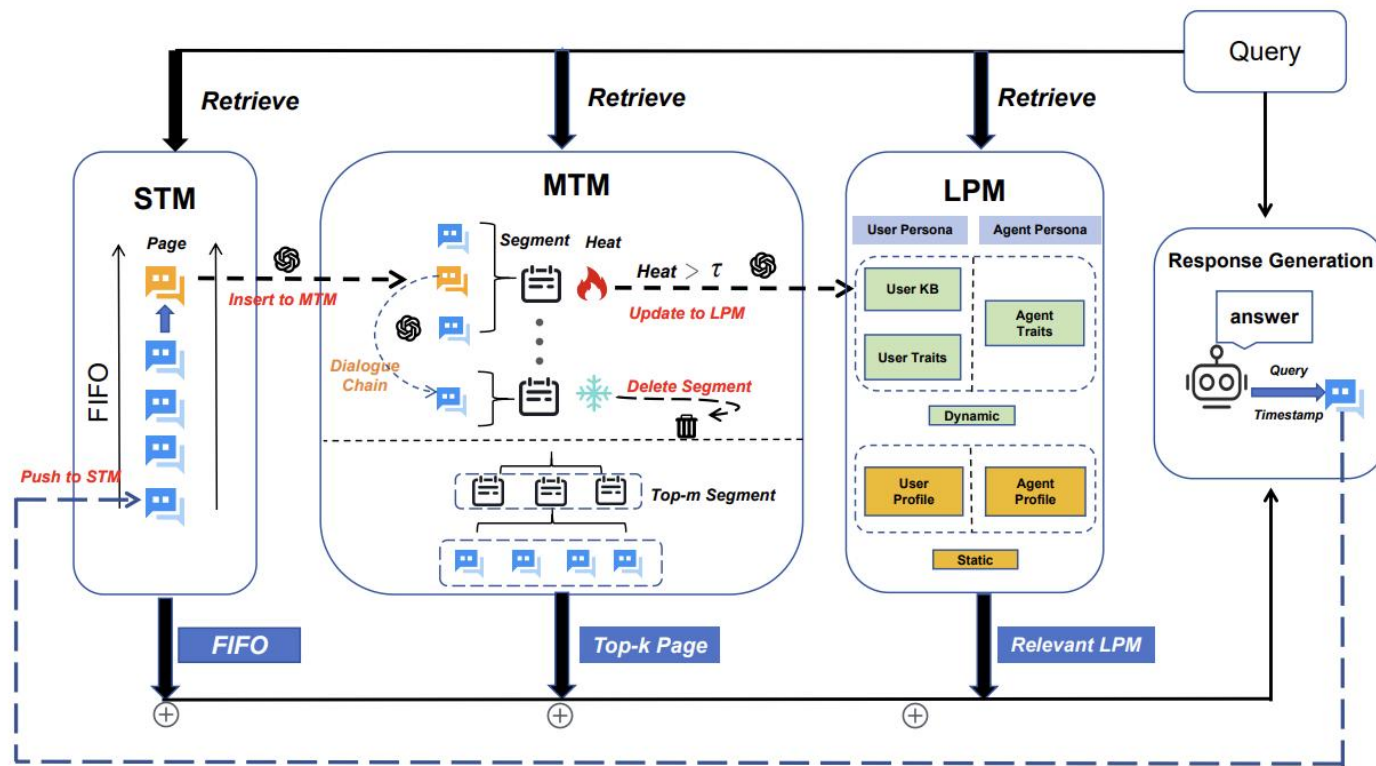
负责在不同层级的内存之间动态刷新和迁移信息

❑ 内存检索 (Memory Retrieval)

负责根据用户查询，从各层级内存中提取最相关的信息

❑ 响应生成 (Response Generation)

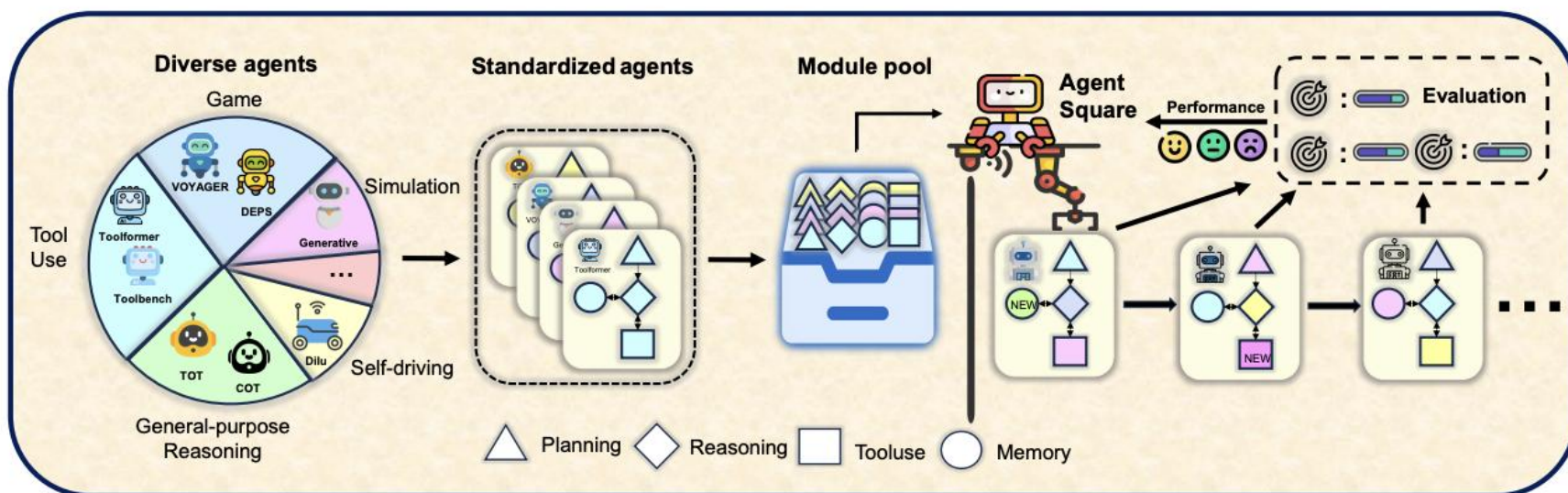
负责整合所有检索到的内存信息，生成连贯且个性化的回复



✧ 3.5.5 大模型智能体「记忆机制」模块

AGENT-SQUARE

- ❑ 四大核心模块：规划 (Planning)、推理 (Reasoning)、工具使用 (Tool Use) 和 记忆 (Memory)。
- ❑ 标准化接口：为这四类模块定义了统一的输入输出 (IO) 接口。这使得不同来源、不同设计的模块可以像乐高积木一样被轻松地组合和替换，极大地促进了模块的复用和创新。



3.5.5 大模型智能体「记忆机制」模块

AGENT-SQUARE

■ 基于上述模块化空间，AgentSquare框架被提出，它能够自动搜索并发现最优的智能体模块组合。该框架主要包含两大核心机制和一个加速器：

- 模块演进 (Module Evolution):

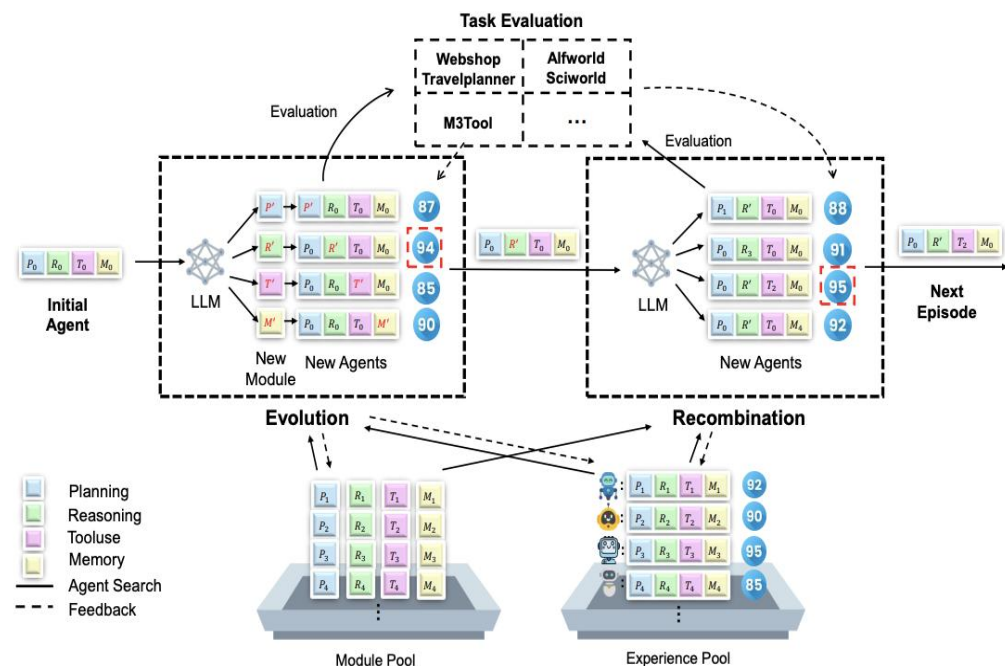
创造全新的、性能更优的模块，通过程序级别的优化来生成新的模块代码。

- 模块重组 (Module Recombination):

根据任务需求和已有经验，形成新的智能体配置。

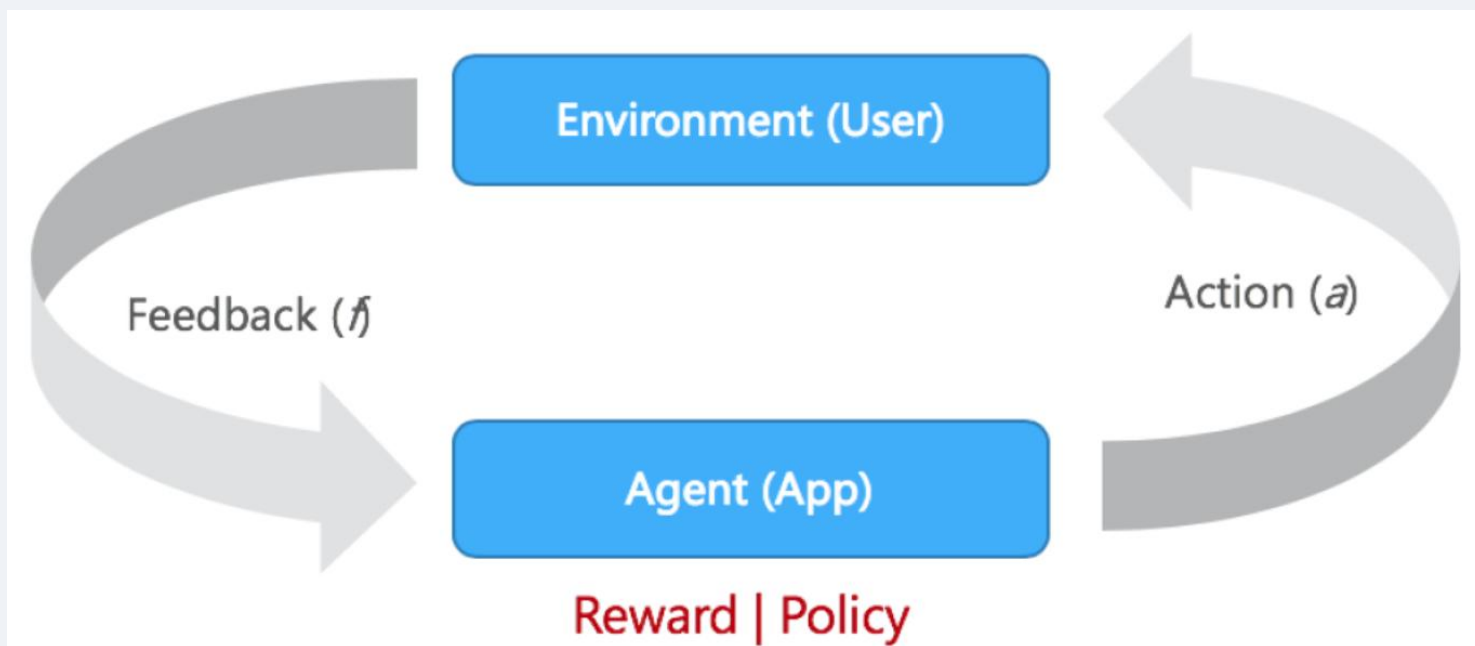
- 性能预测器 (Performance Predictor):

分析新智能体的模块组合以及历史上已评估智能体的性能数据，快速预测新组合的性能



✧ 3.5.6 大模型智能体「强化学习」模块

- ❑ LLM Agent的决策、规划、记忆等能力非常适合使用强化学习进行训练
- ❑ 类似于使用RL来训练传统LLM，RL提供了一个通过试错（trial and error）和结果反馈来学习最优策略的框架，这正是智能体所需要的



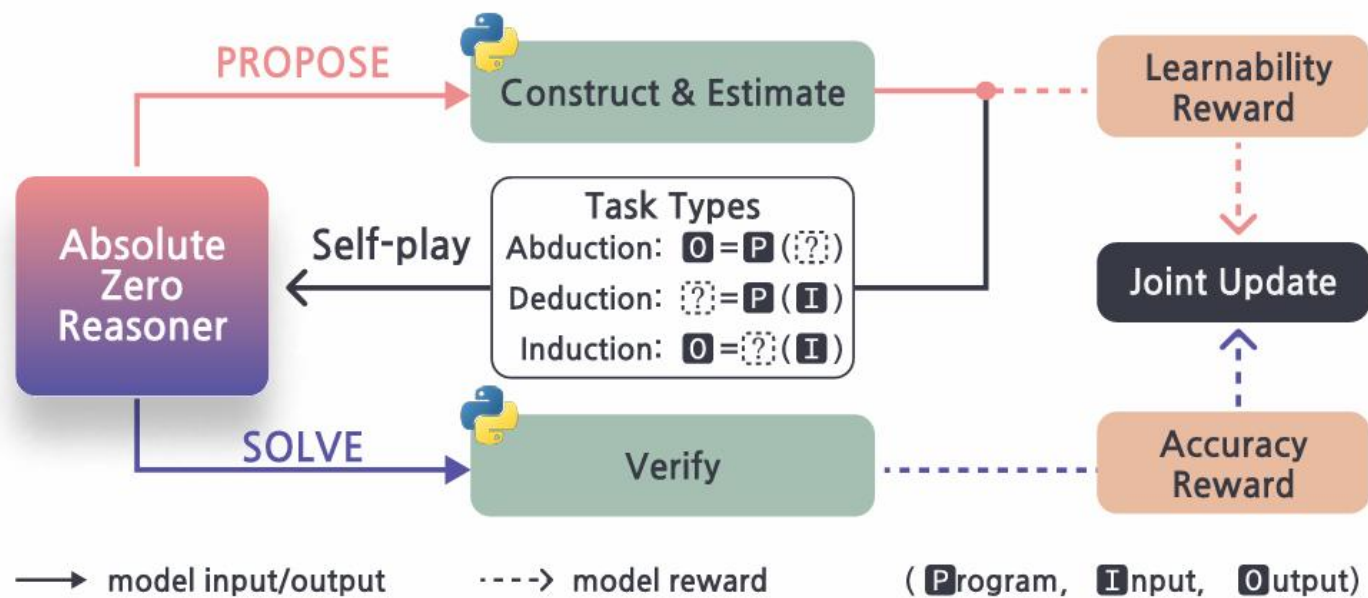
核心好处

- 实现动态决策与策略学习
- 提升在复杂环境中的适应与鲁棒性
- 解决长时程（Long-Horizon）任务
- 精细化行为对齐与控制
- 减少对大规模示范数据的依赖

✧ 3.5.6 大模型智能体「强化学习」模块

Absolute Zero Reasoner

- 使用强化学习自博弈强化LLM的推理能力
- 完全从“零”开始的RLVF（Reinforcement Learning with Verifiable Rewards）：
 - 人类专家收集的训练任务
 - 人类专家训练的奖励函数



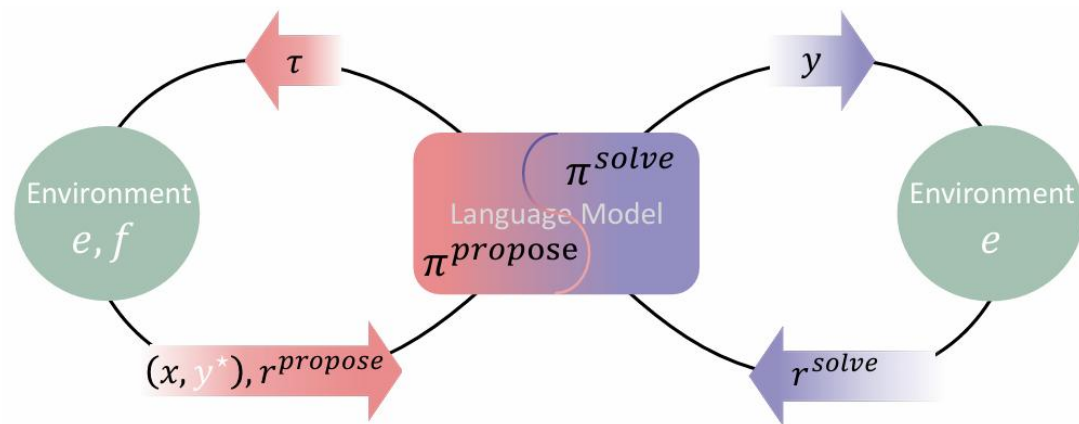
✧ 3.5.6 大模型智能体「强化学习」模块

Absolute Zero Reasoner

- ❑ 人类只需定义基于规则的奖励函数，以及设定任务类型
- ❑ 具体训练样本由自博弈系统自动生成

具体流程：

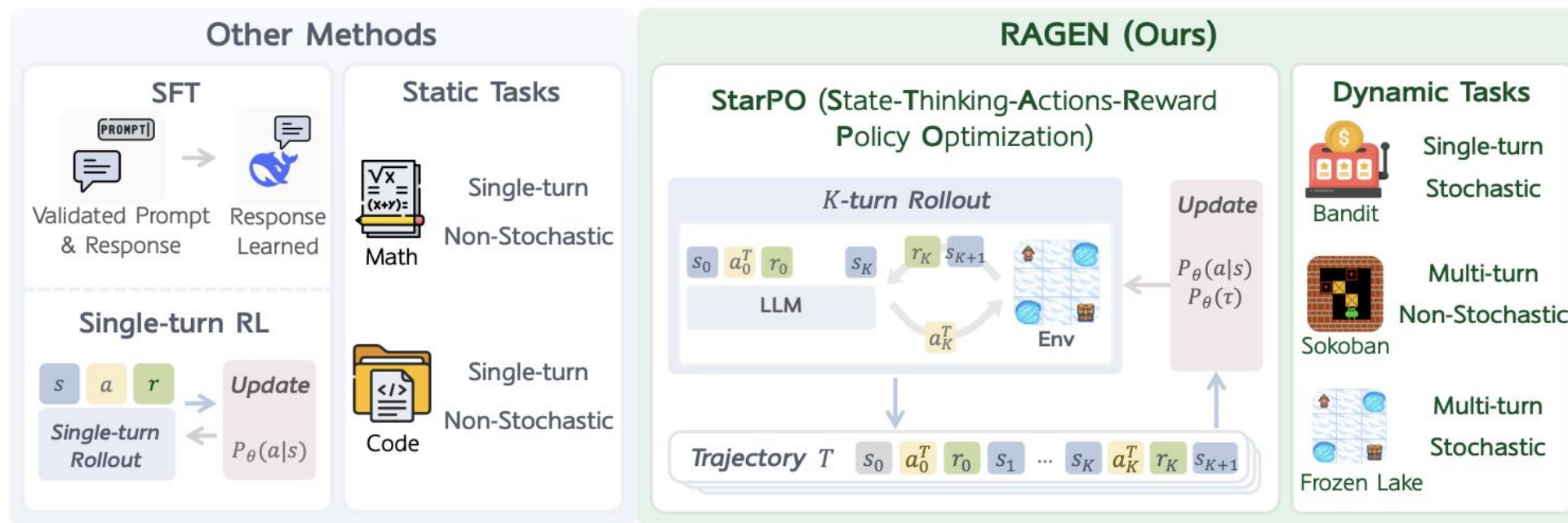
1. Proposer提供函数代码
2. Env验证代码的有效性并根据代码生成任务
3. Solver求解任务
4. Env验证答案的有效性
5. 采用Task-Relative REINFORCE++训练LLM



✧ 3.5.6 大模型智能体「强化学习」模块

RAGEN

- ❑ 直接将现有RL方法用于多回合智能体训练时，会面临训练不稳定、奖励信号设计复杂、泛化能力有限等挑战
- ❑ “哪些设计因素能够让自我演进的LLM智能体有效且稳定地学习？”
- ❑ 为解决上述问题，一个名为RAGEN的模块化系统被提出，并在该系统上实现了一个通用的智能体RL训练框架，名为StarPO。



✧ 3.5.6 大模型智能体「强化学习」模块

RAGEN

□ 通过在上述框架和环境中的大量实验，得出了以下三个核心发现：

- 智能体训练时易陷于“回声陷阱”
- 高质量的“试错”经验至关重要
- 真正的“思考”能力需要被精确奖励

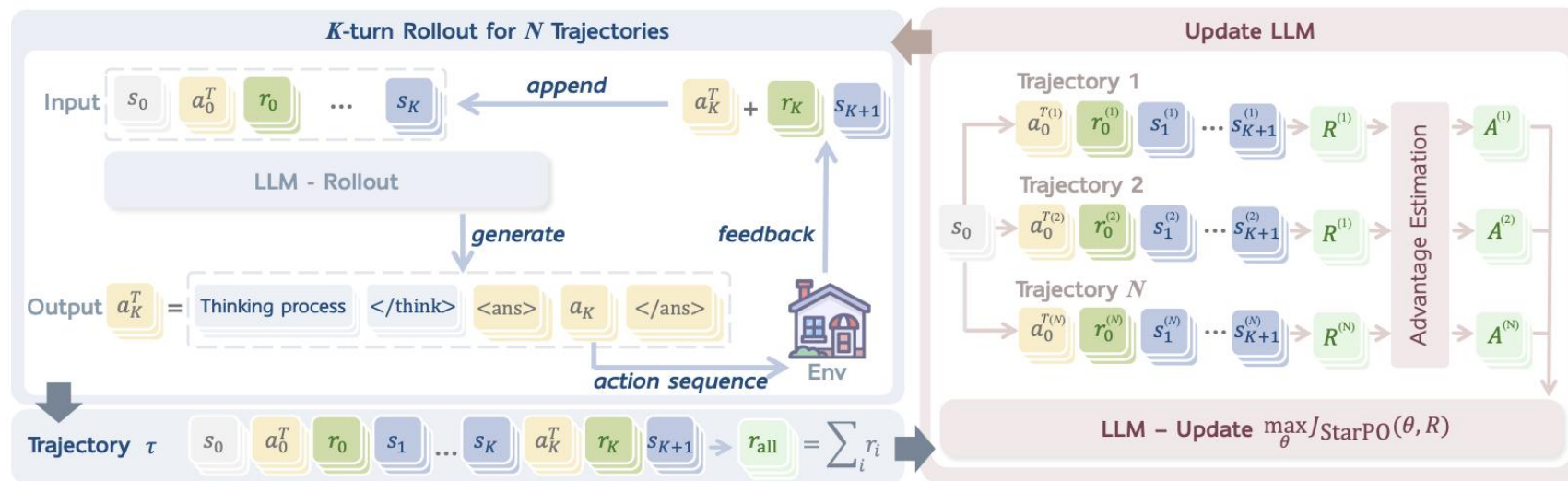
Trajectory-Level Objective in StarPO vs. Previous Methods

Previous methods (e.g., PPO (Schulman et al., 2017), GRPO (Shao et al., 2024a)):

$$J_{\text{step}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} [R(x, y)] \quad (\text{optimize single-turn output } y \text{ given input } x)$$

StarPO (ours):

$$J_{\text{StarPO}}(\theta) = \mathbb{E}_{\mathcal{M}, \tau \sim \pi_{\theta}} [R(\tau)] \quad (\text{optimize total reward over trajectory } \tau = \{s_0, a_0, r_0, \dots, s_K\})$$



✧ 3.5.7 AI Agents的局限

❑ 缺乏因果推断能力

混淆相关性与因果关系从而输出错误的结论。

❑ LLM本身的局限性

对提示词敏感、偏见、幻觉.....

❑ 有限自主性

只能完成特定的任务，依赖于人工预设的提示词、规划算法、对话形式.....

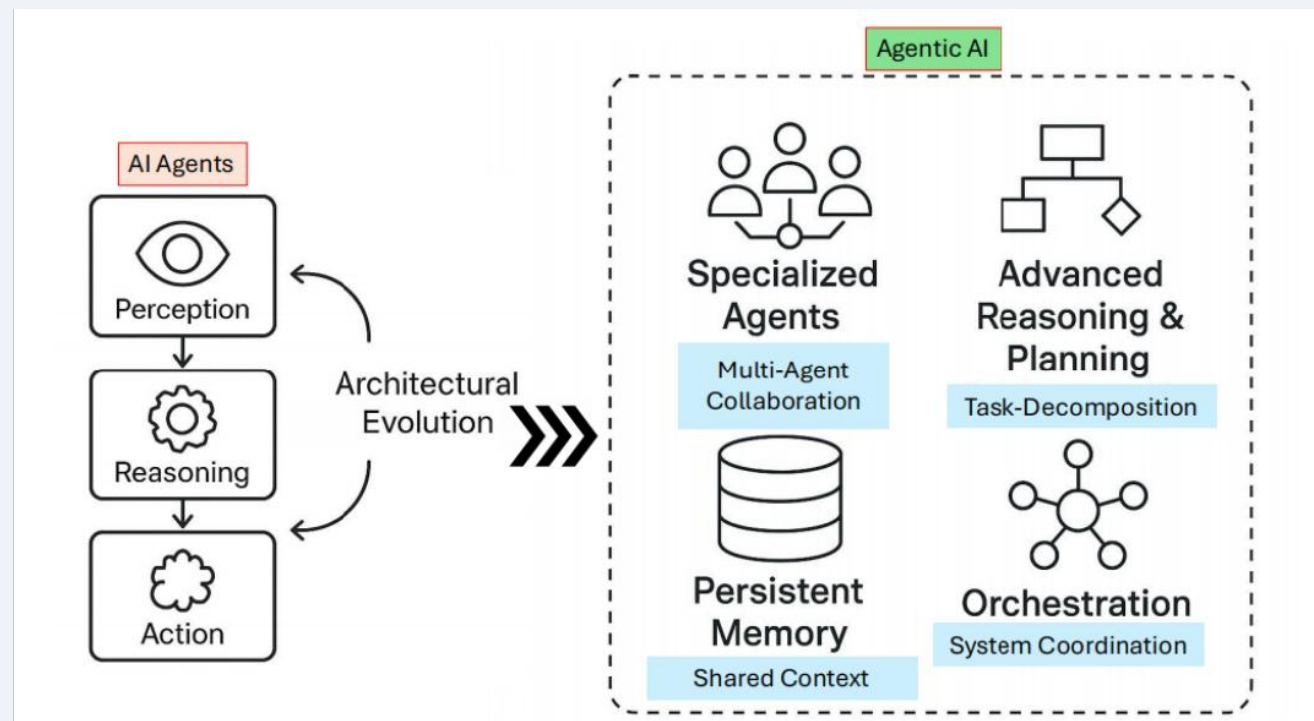
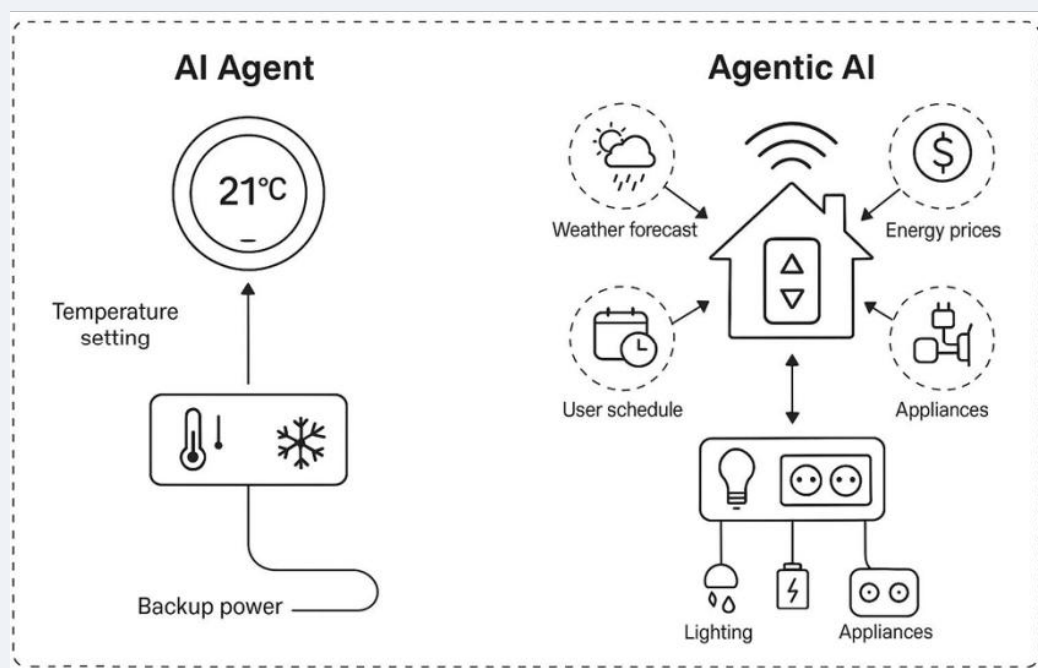
❑ 有限长期规划能力

在需要长期规划、应急响应的任务中表现不佳，智能体可能会陷入重复行为，如无休止地查询工具，或在子任务返回失败或模糊结果时无法适应。



✧ 3.6 从AI Agents到Agentic AI Systems

- ❑ Agentic AI 并不是 AI Agents 的“加强版”，而是范式的转变——从个体执行转向系统协同，一组具备分工与协作能力的智能体共同构成的集体智能。





3.6 代表性 Agentic AI 架构探索

核心逻辑

Agentic AI 架构强调语言模型从“任务响应者”转变为“自主行为体”，通过角色分工、任务规划、工具调用等机制构建具备目标导向性与行动能力的智能体系统。

- ❑ CAMEL、VOYAGER、MetaGPT 等系统体现了从对话式智能体 → 协作式智能体 → 具身型智能体的演化路径，推动了 LLM 在实际场景中的 Agent 化落地。
- ❑ AgentNet 与 Small Language Models 展示了模块组合 + 轻量化部署的新范式，为智能体在多任务、多环境中的高效适配提供了新思路。
- ❑ 从静态语言输出走向动态行为生成，这些系统共同标志着智能体从“模型层”向“系统层”演进的关键转折。

系统名称	架构核心思想	多Agent协作	适用场景 / 特点
CAMEL	基于角色对话的协作智能体，通过用户、助手、任务专家之间的多轮自然语言交互完成任务	明确角色协作	多角色对话任务、任务规划协作演示
VOYAGER	具身游戏世界中的强化学习智能体，具备自我演化知识库和技能成长机制	单智能体	Minecraft 等虚拟环境中的具身任务
MetaGPT	将软件开发过程结构化为多个角色模块（PM、程序员、测试等），使用LLM完成团队协作式任务	多角色分工明确	软件开发、任务管线结构化执行
AgentNet	基于Prompt编排的智能体系统，支持记忆、工具、控制流程模块自由组合	支持协同场景	通用Agent框架，适配多种任务
SLaM（Small Language Models）	探索轻量模型在智能体结构中的潜力，通过规划器+执行器+环境接口组合实现基本Agent能力	单智能体	面向边缘设备部署、高效推理任务

✧ 3.6.1 Agentic AI: CAMEL

▣ 角色扮演：

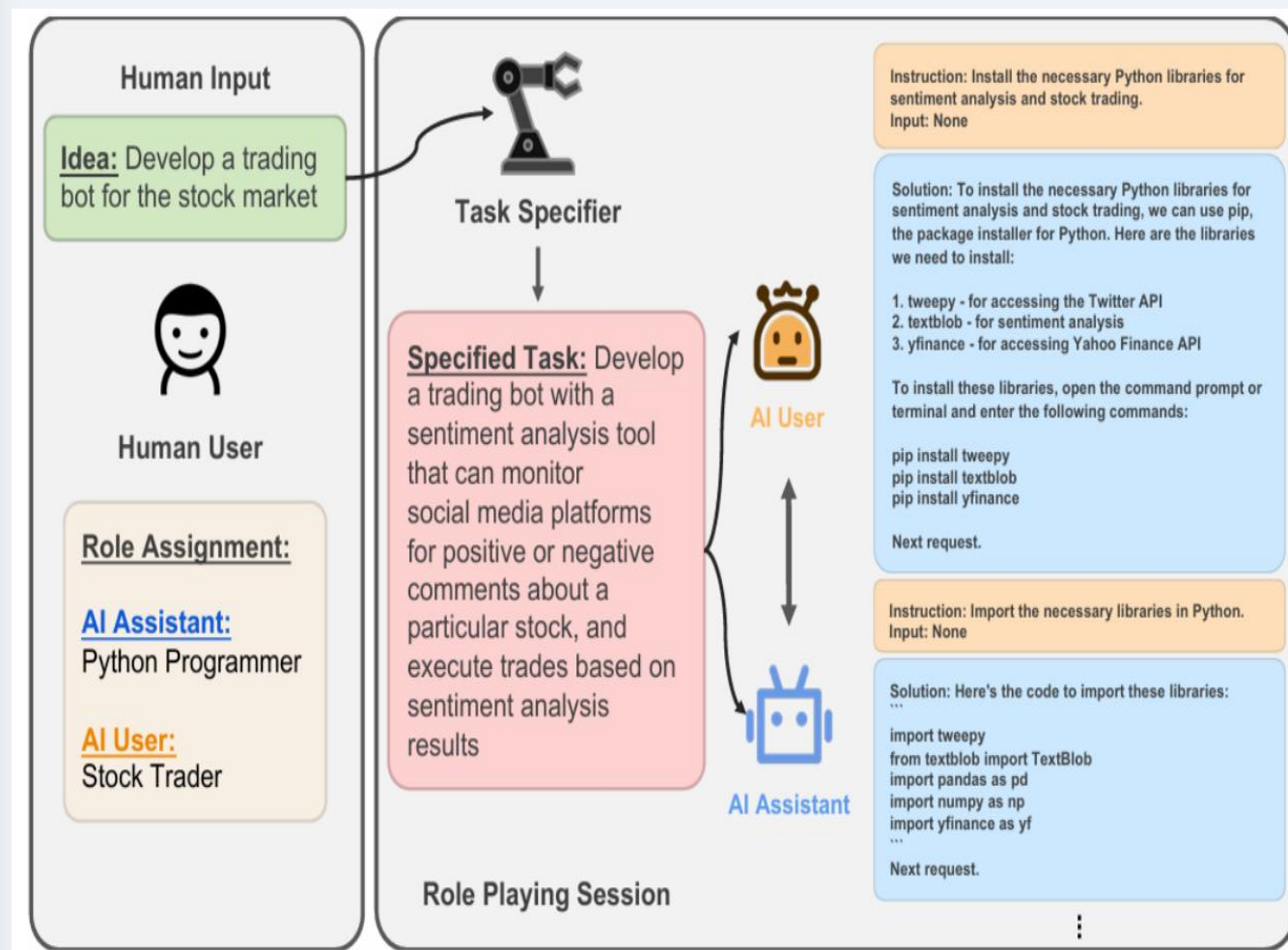
- AI User提出任务/指示
- AI Assistant提供解决方案/执行指令

▣ 起始提示：

- 指定智能体的身份（如程序员），或令智能体自行选择身份。
- 仅需人类提供一个初步想法，智能体便可自主推进对话。

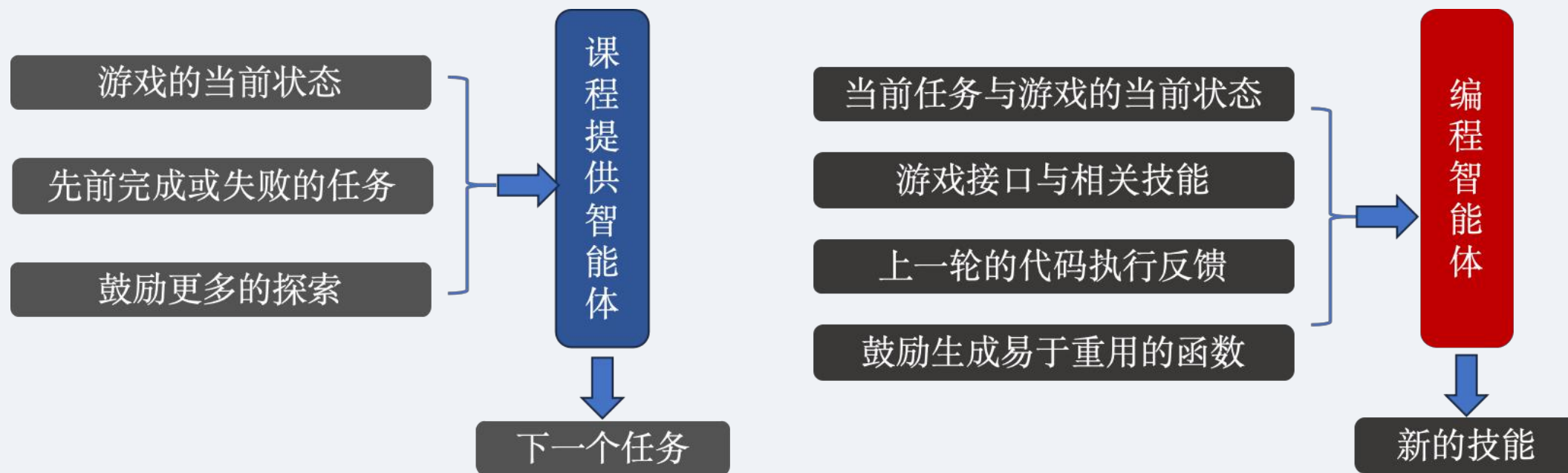
▣ 可能存在的问题：

- 角色反转
- 重复/循环
- Flake replies



✧ 3.6.1 Agentic AI: VOYAGER

- ❑ **课程提供智能体**：以在Minecraft中发现尽可能多的不同事物为目标，根据智能体当前的探索进度生成难度不断增长的课程任务。
- ❑ **编程智能体**：向课程提供智能体索要新的任务，生成函数代码形式的技能以解决该任务。
- ❑ **提示迭代编程机制**：根据上一轮代码的执行反馈构造新一轮提示，“执行 → 观察反馈 → 修正”的循环，直至技能验证智能体认可该技能的有效性。



✧ 3.6.1 Agentic AI: MetaGPT

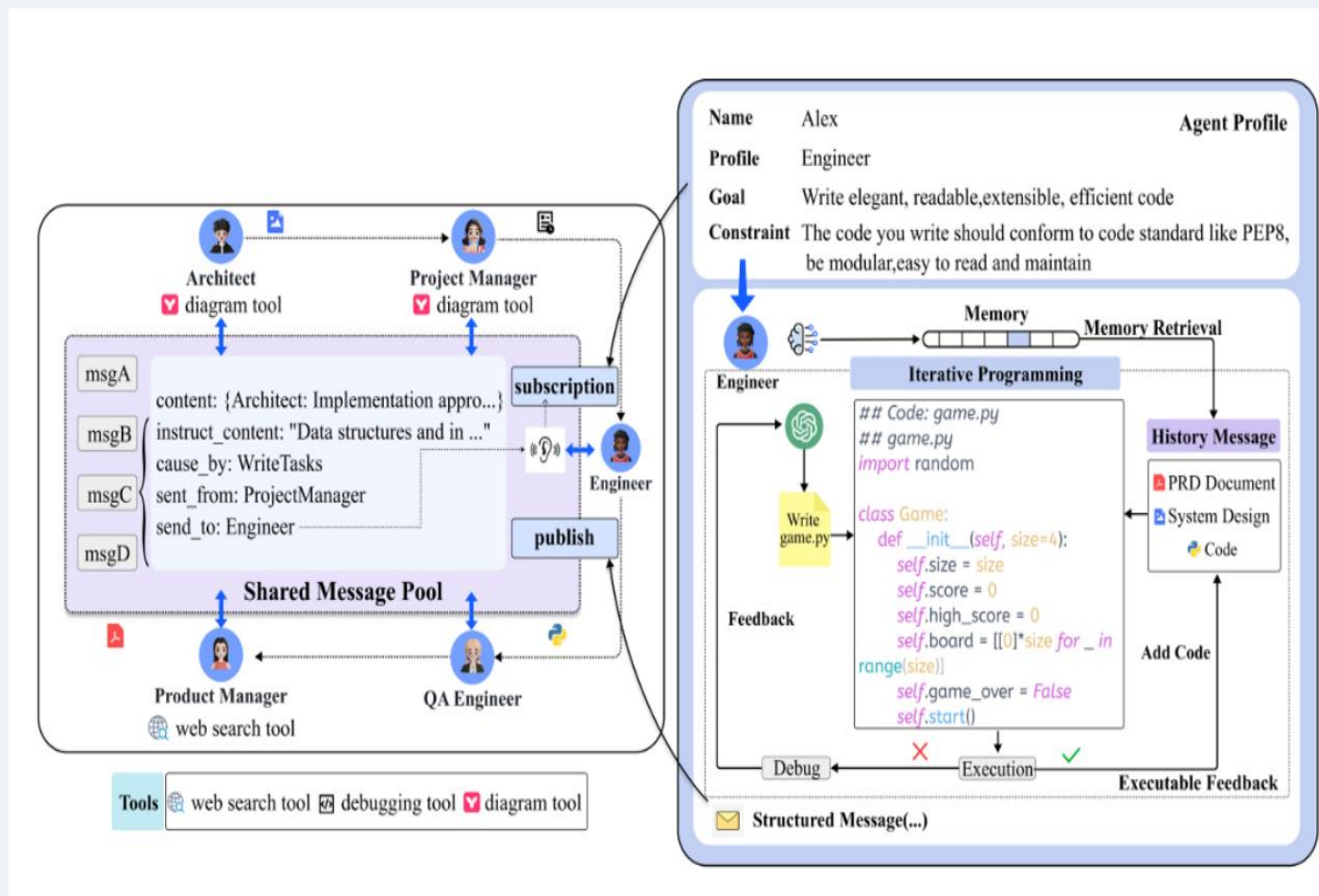
❑ 受人类协作模式 SOP (Standardized Operating Procedures) 启发的编程系统。

❑ workflow 设计

- 智能体的角色、分工与 workflow 模拟人类的软件开发团队。

❑ 通信协议

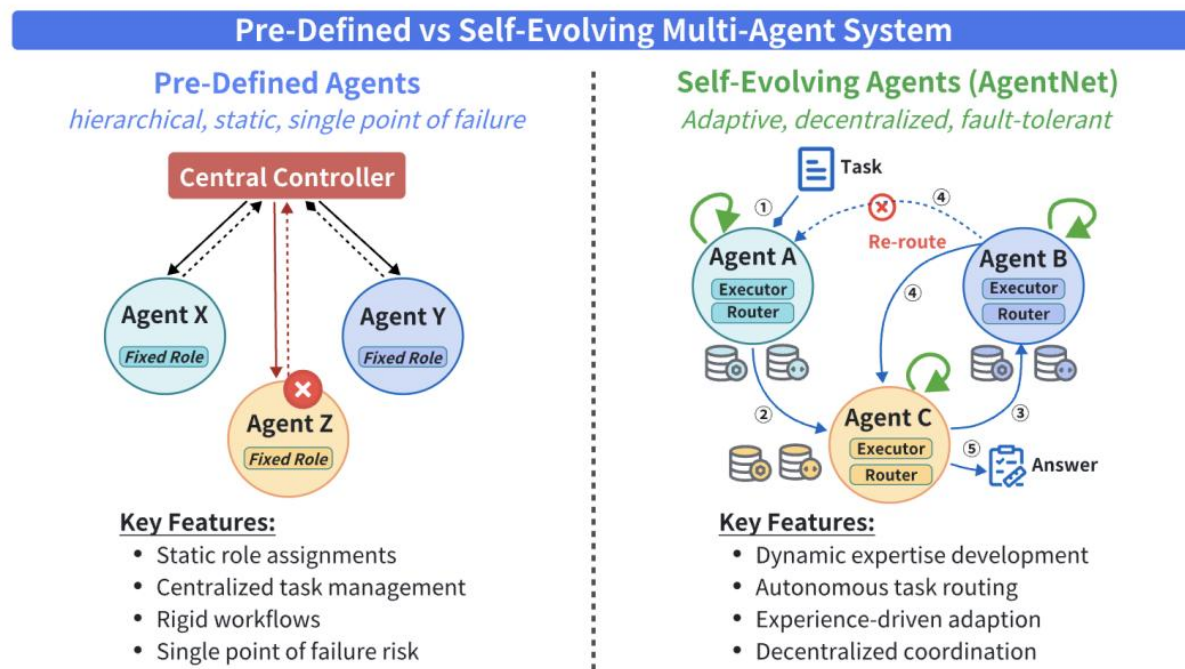
- 要求智能体按照特定格式输出中间结果。
- 采用发布-订阅机制进行信息传递，智能体可以直接从共享池中检索所需信息，而无需询问其他智能体并等待他们的响应。
- 每个智能体订阅与自身任务相关的信息类型。



✧ 3.6.1 Agentic AI: AgentNet

- ❑ 本工作提出了一种名为 AgentNet 的新型多智能体系统框架，旨在解决现有系统依赖中心化协调而带来的各种问题。
- ❑ AgentNet提出了一个完全去中心化、基于演进和协作的框架：

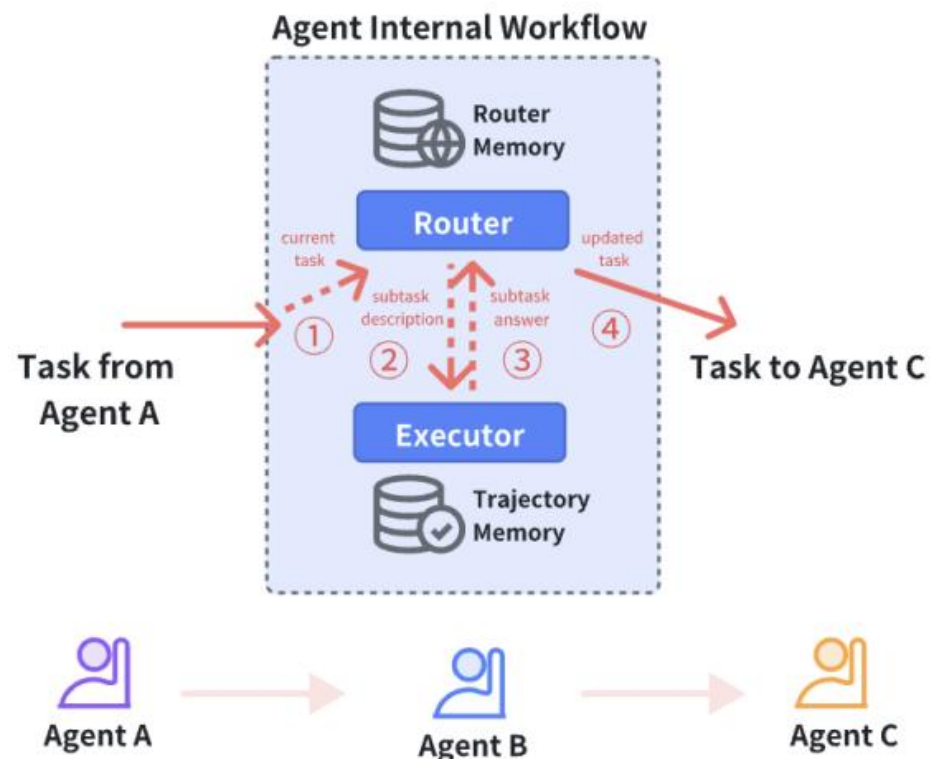
- 完全去中心化的范式 (Fully Decentralized Paradigm)
- 动态演进的图拓扑 (Dynamically Evolving Graph Topology)
- 基于RAG的自适应学习与专业化 (Adaptive Learning and Specialization)



✧ 3.6.1 Agentic AI: AgentNet

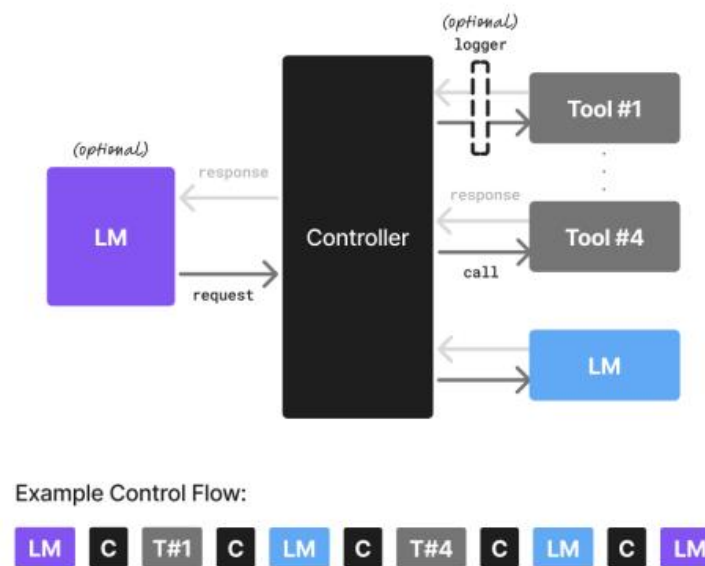
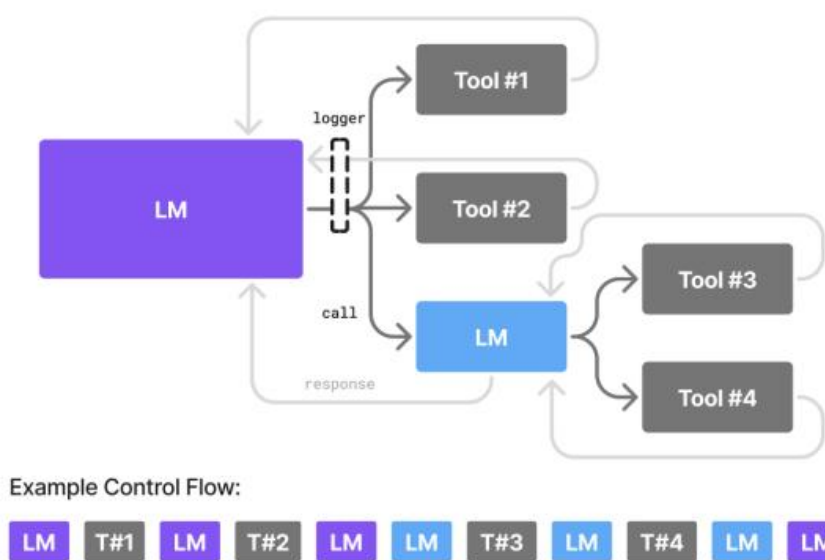
□ AgentNet内部 workflow

- 双重角色设计：每个智能体内部都包含两个关键组件：一个路由器（Router）和一个执行器（Executor）。
- 动态任务分配：当一个智能体收到任务后，它的路由器会根据任务需求和自身及其他智能体的能力，做出三种决策之一：
 - 执行 (Execute):
如果自己有能力和技能，就直接完成任务。
 - 转发 (Forward):
如果认为网络中有其他智能体更适合，就将任务完整地转发过去。
 - 拆分 (Split):
将复杂任务分解成多个子任务，自己执行擅长的部分，然后将其余子任务分派给其他智能体。



✧ 3.6.1 Agentic AI: Small Language Models

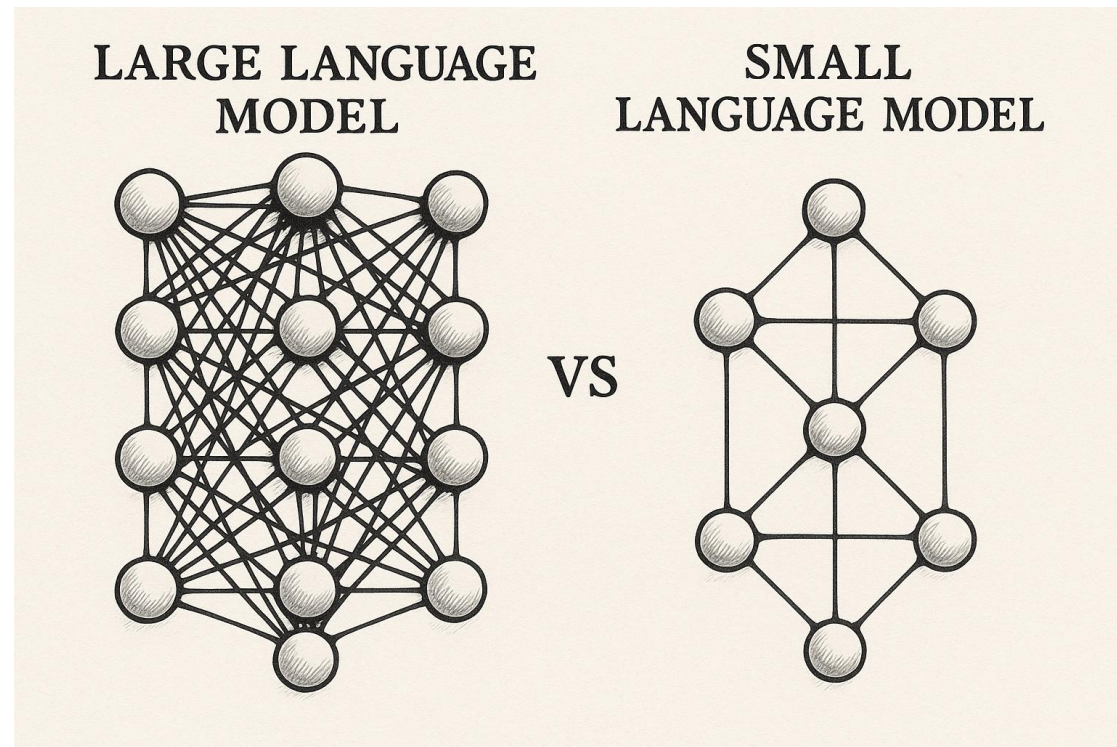
- 尽管目前LLM在AI智能体中占主导地位，但这是一种资源错配。对于大多数智能体中重复性高、任务范围窄的调用场景，SLM不仅能力足够，而且在操作上更适合，经济上也更划算，因此SLM才是智能体AI的未来。



✧ 3.6.1 Agentic AI: Small Language Models

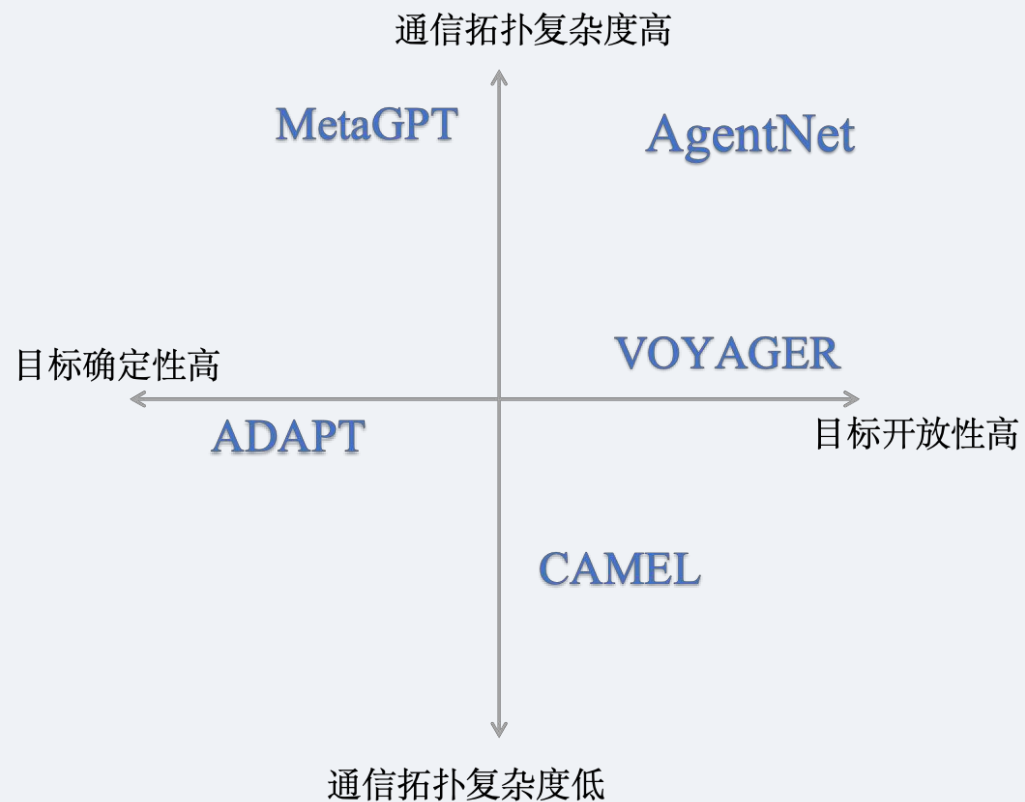
□ 论文从三个核心角度来支撑其观点：

- V1: SLM已足够强大 (Sufficiently Powerful)
- V2: SLM在操作上更适合 (Operationally Suitable)
 - 灵活性高
 - 行为对齐更易
 - 系统天然支持异构
 - 利于数据收集和持续改进
- V3: SLM在经济上更优越 (More Economical)
 - 推理效率高
 - 微调成本低
 - 支持边缘部署



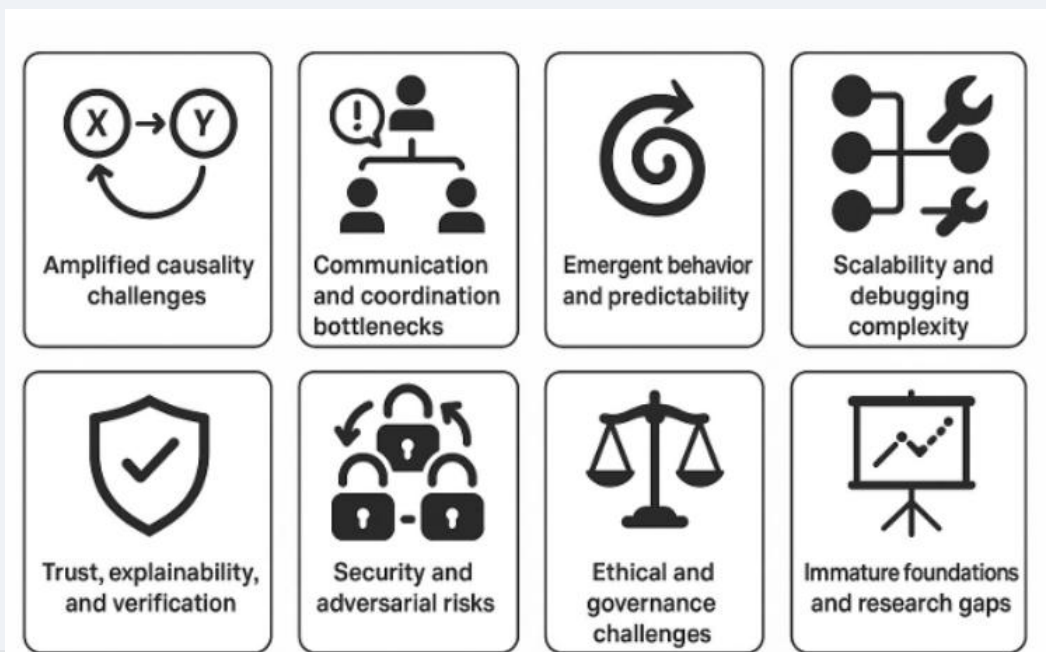
✧ 3.6.2 Agentic AI二维谱系图

- 任务目标：确定性 / 开放性
- 通信拓扑复杂度：简单 / 复杂



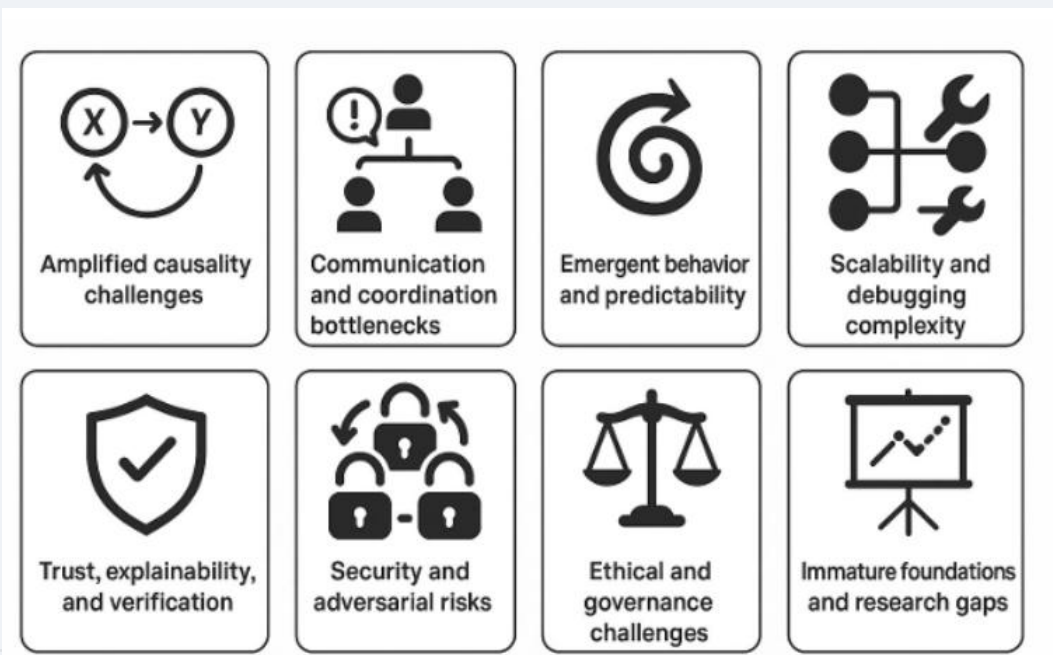
✚ 3.6.3 Agentic AI的局限

- ❑ 局限性的放大：AI Agents的部分局限性在Agentic AI系统中被放大，一个智能体的故障或错误输出可能会使下游的所有模块做出错误决策。
- ❑ 涌现性与不可预测性：多智能体协作既带来了创新性的方案也引入系统的不可预测性与安全风险。
- ❑ 系统复杂性：不断增加的系统复杂性、智能体行为的不可预测性使得进一步拓展系统规模与程序调式变得困难。例如由于系统的非组合性，添加组件可能会增加智能体的认知负担，从而使系统性能下降。

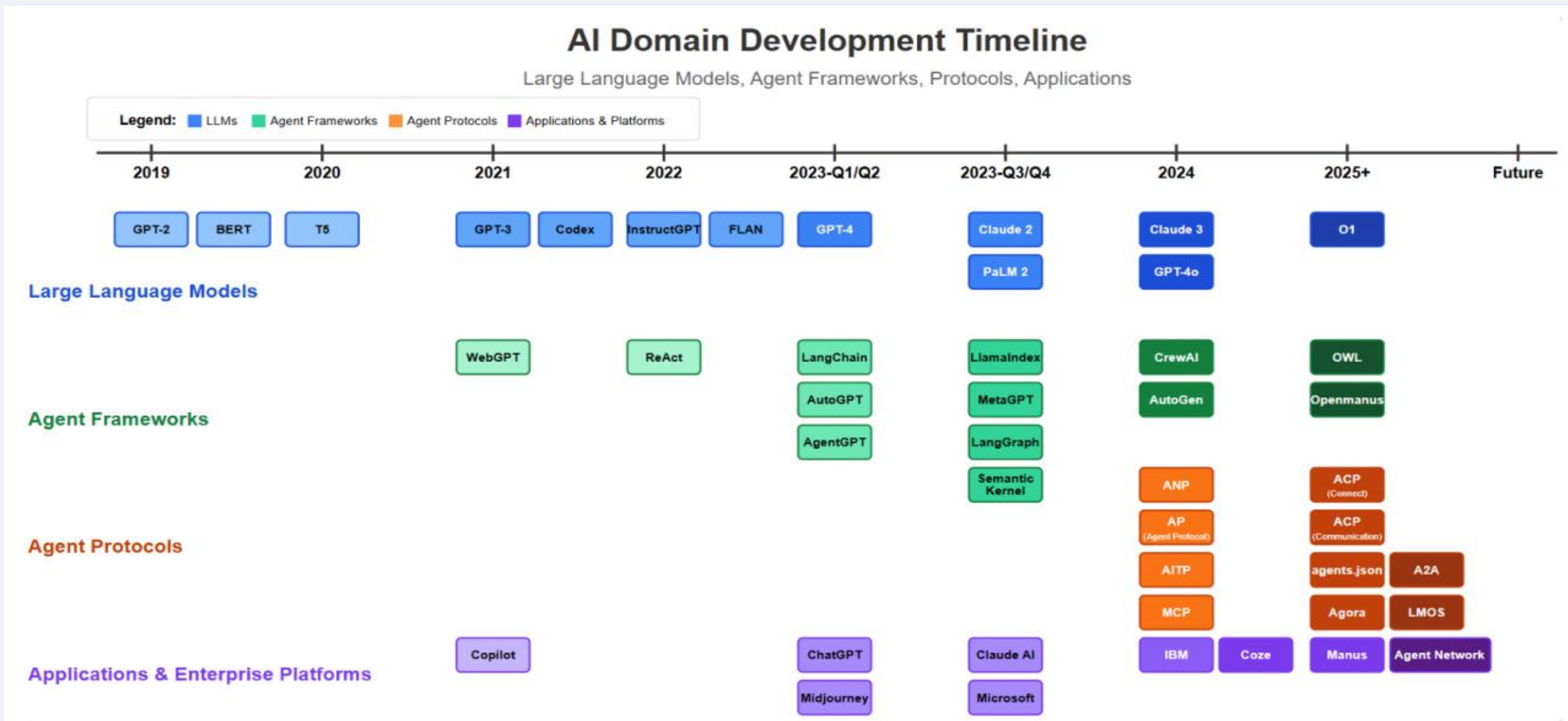


✧ 3.6.3 Agentic AI的局限

- ❑ 智能体之间的沟通和协作瓶颈：任务协作要求智能体之间具有共同的语境，但由于智能体信息处理能力的限制，系统中的智能体又不可能共享全局信息，此外智能体之间的有效沟通依赖可靠的通讯协议。
- ❑ 安全性与对抗攻击：与AI Agents相比Agentic AI系统的暴露的攻击面更大，即使只有一个智能体被提示词注入攻击、投毒或恶意操控，也可能导致整个系统的瘫痪。
- ❑ 伦理与治理挑战：当多个智能体协作产出错误或意外结果导致损失时，责任归属问题变得尤为突出。



3.7 大模型智能体框架与协议



✧ 3.7.1 大模型智能体框架

▣ 智能体框架加速了大模型智能体或智能体系统的构建、测试与部署。

▣ 快速构建复杂智能体系统

- 不必重复造轮子，实现模块化开发与复用
- 降低开发门槛，从写代码转向配置组件与提示词，聚焦任务逻辑而非底层实现

▣ 提升标准化与可扩展性

- 标准化工具接入接口，支持灵活组合与调用
- 提升系统间兼容性，易于集成与迁移

▣ 借力平台生态资源

- 部分框架构建了完善的生态系统（如工具库、模型仓、数据库服务等）
- 开发者可直接复用平台提供的组件与服务，提升效率与系统能力边界



3.7.1 大模型智能体框架

核心逻辑

通过结构化组织语言模型的感知、推理、规划与行动能力，构建具备目标导向、自主决策与持续交互能力的通用智能系统。

当前大模型智能体领域涌现出多种框架，它们在设计理念和功能侧重上各有特色，共同推动着智能体技术的发展。这些框架普遍强调模块化和协作能力，以应对复杂任务。

框架名称	主要目的	关键特性	典型应用
ReAct	通用范式，协同 LLM 的推理与行动	思考-行动交错循环，动态推理，外部环境交互	问答、决策制定、文本游戏、网络导航
AutoGen	开源框架，通过多智能体对话构建 LLM 应用	可定制、可对话智能体，对话编程，LLM/人类输入/工具组合	数学、编程、问答、运筹学、在线决策、娱乐
CAMEL	通信智能体框架，促进通信智能体自主协作	角色扮演（AI 助手/用户），起始提示，任务指定，对话式任务解决	AI 社会模拟、代码生成、指令遵循协作
CrewAI	Python 框架，创建自主 AI 智能体团队	团队（Crews）管理，流程（Flows）控制，基于角色的智能体，灵活工具，智能协作，任务管理	研究自动化、内容创作、软件原型
LangGraph	基于图框架，简化智能体与工作流程管理	基于图的框架，动态状态管理，自动化复杂工作流，智能体链接与协作	机器翻译、多语言翻译系统

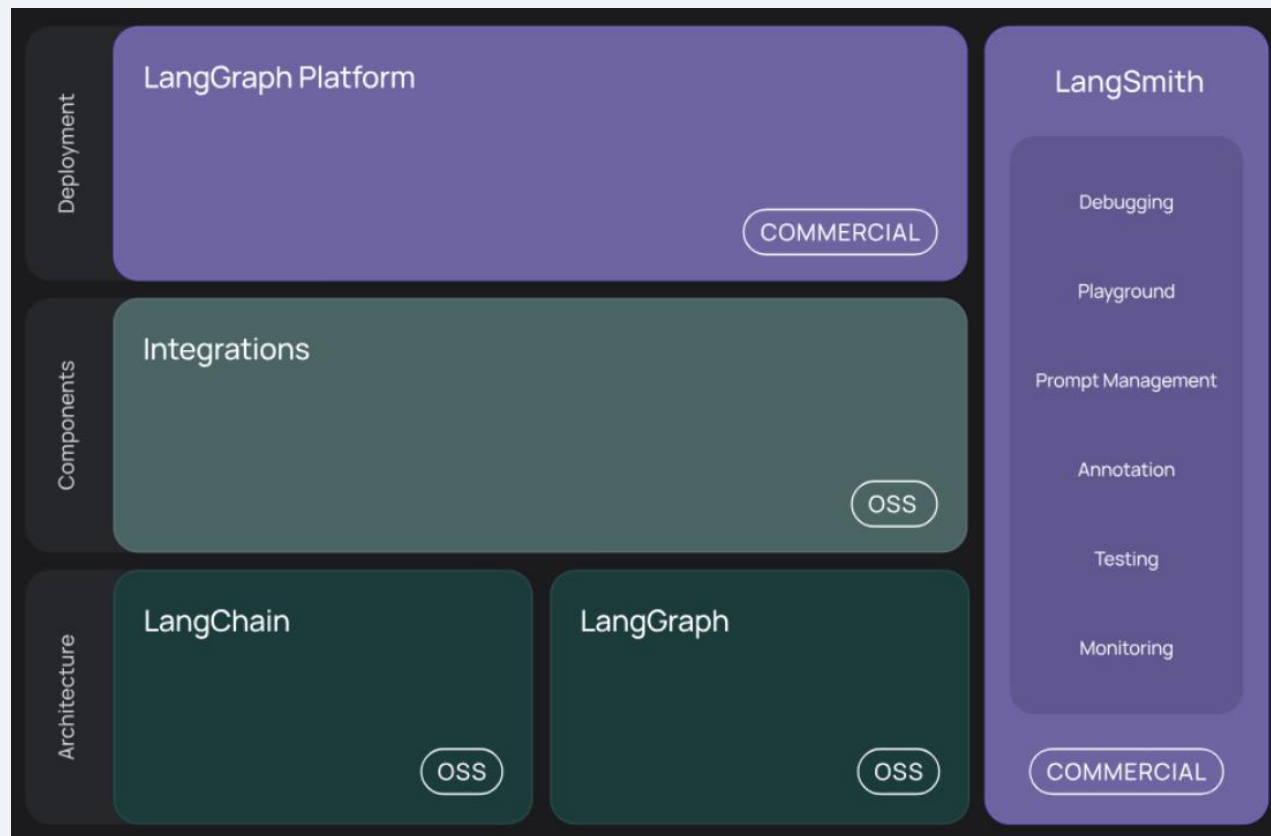
✧ 3.7.1 大模型智能体框架

▣ **LangChain**: 提供模块化组件，用于快速构建具备推理、记忆、工具使用等能力的LLM应用。

- Chains: 链将一系列模块化组件以特定方式组合起来

▣ **LangGraph**: 基于图的低层次智能体编排框架

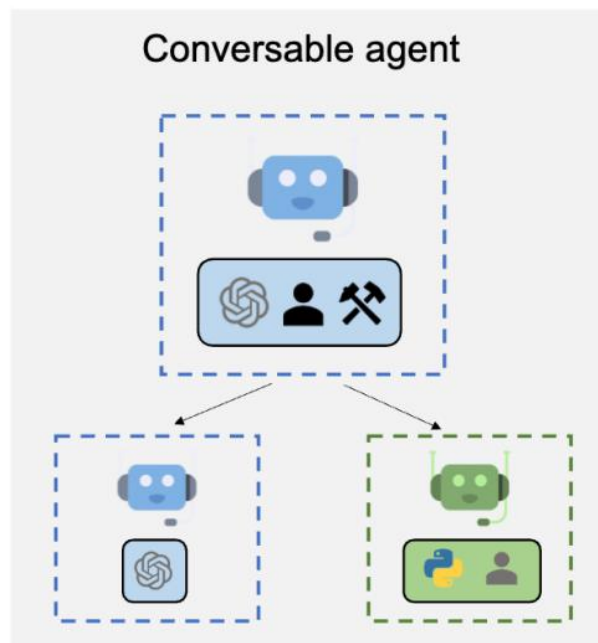
- Node: 处理任务逻辑
- Edge: 决定节点转移
- State: 当前系统状态



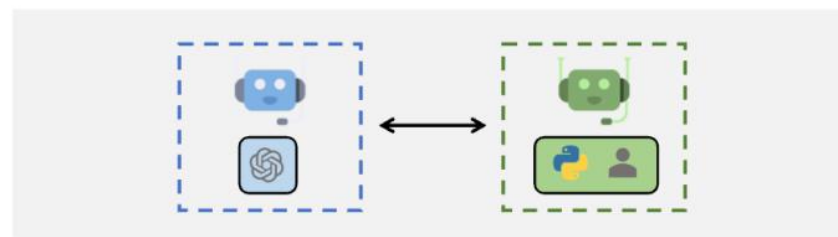
✧ 3.7.1 大模型智能体框架

AutoGen

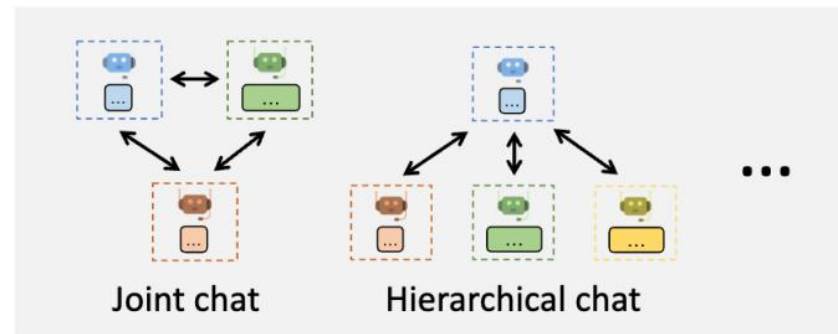
- ❑ 多智能体协作的开源框架。
- ❑ 基于多智能体对话的形式快速构建Agentic AI系统。
- ❑ 提供可对话智能体抽象，支持多智能体之间的交互、任务协作、工具调用以及人类参与。



Agent Customization



Multi-Agent Conversations



Joint chat

Hierarchical chat

Flexible Conversation Patterns

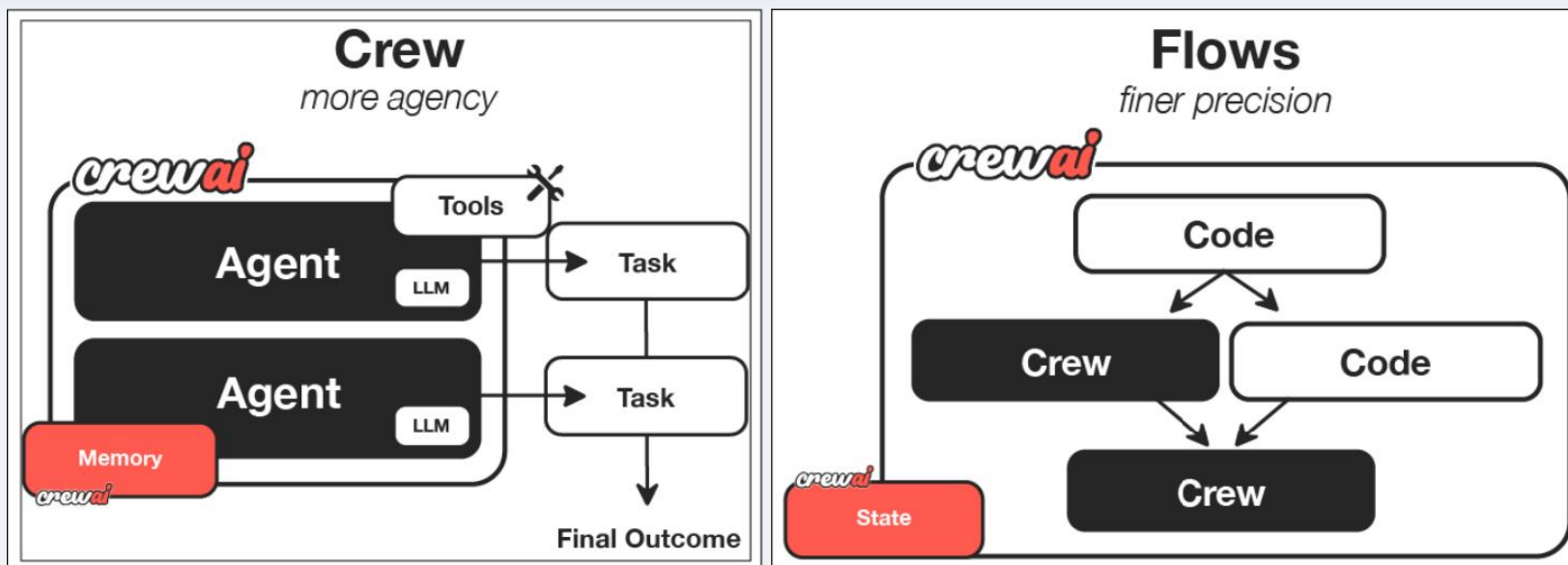
✧ 3.7.1 大模型智能体框架

CrewAI

▣ 支持团队协作与流程控制的多智能体系统框架

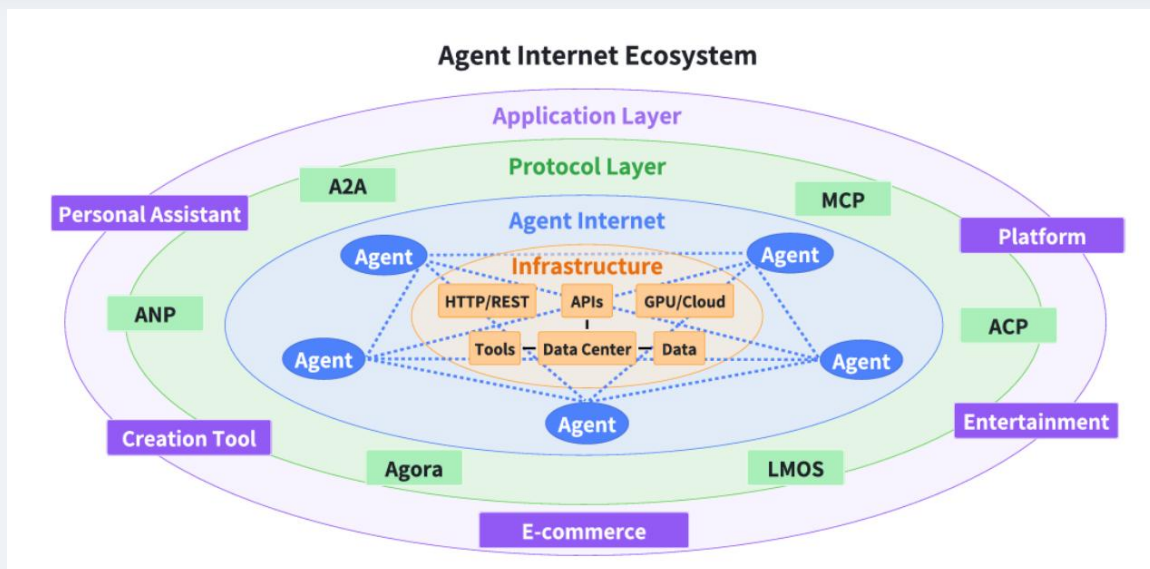
▣ 两大核心机制

1. **Flows**: 提供可编排的工作流机制，精确控制任务执行过程，支持条件判断、循环、状态管理等，适用于需要确定性结果、精确控制的业务。
2. **Crews**: 组织多个具有特定职责的智能体，构成“AI 团队”协同完成任务，适用于开发式调研、文本生成等开放任务。



✧ 3.7.2 大模型智能体协议

- ❑ 大模型智能体的应用场景日益丰富，涉及不同厂商、不同结构的机器人和设备。
- ❑ 然而智能体与外部实体之间的缺乏统一标准，调用与接口设计零散，阻碍工程效率与系统协同。
- ❑ 类似早期的互联网因不兼容的系统与有限的连接而支离破碎，严重影响互联互通与发展速度。
- ❑ 需要标准化的协议来支持智能体与外部实体之间的交互，形成稳定的智能体网络生态，提升系统兼容性、可扩展性与协作效率。



✧ 3.7.2 大模型智能体协议

▣ 智能体协议与API、GUI、XML的区别

- API：由服务方自定义，调用高效，但非标准化。
- GUI：对人类用户友好，但交互效率低，难以实现自动化。
- XML：支持跨平台的结构化数据传输，但缺乏对智能体行为和交互语义的抽象。
- Protocol：具备支持复杂、动态、可扩展交互的能力，是构建智能体网络生态必不可少的通信机制。

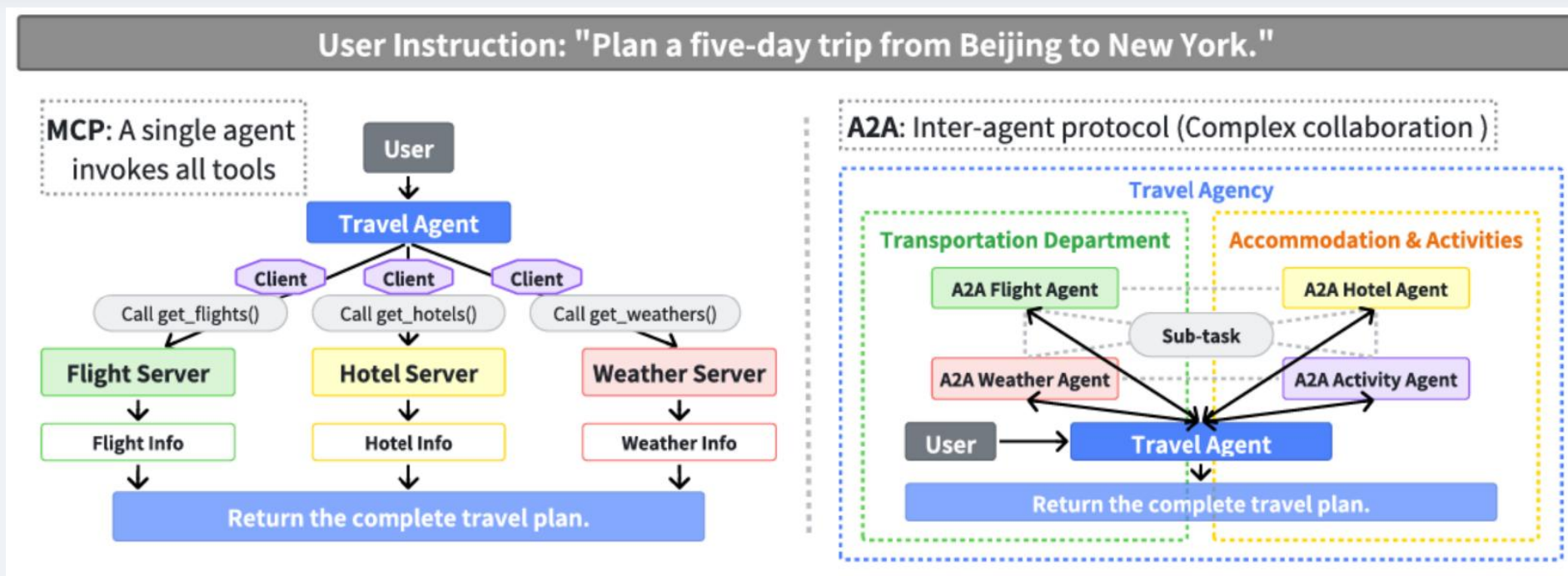
▣ 协议不是API的替代，而是适配智能体生态演进的通信范式。

Table 1: Comparison of the properties of different interaction manners for agents.

Manner	Scenarios	Efficiency	Operation Range	Standardized	AI-Native
API	Server-to-server integration	✓✓	×	×	×
GUI	Computer/ Mobile Use	×	✓	✓	×
XML	Browser Use	×	✓	×	×
Protocol	Agent Interaction	✓✓	✓✓	✓✓	✓✓

✧ 3.7.2 大模型智能体协议

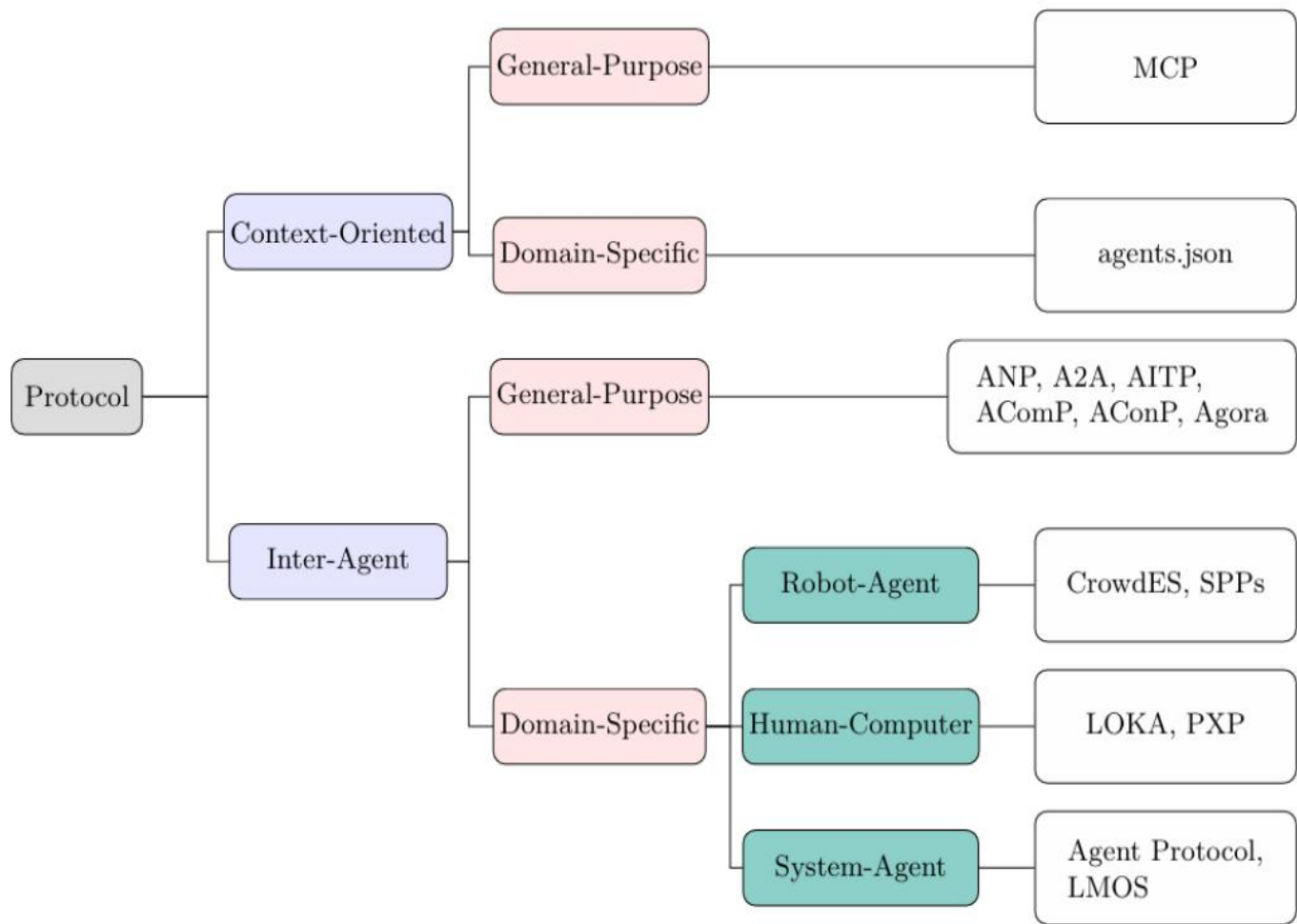
- ❑ 智能体协议的设计服务于智能体范式的演化。
- ❑ 从AI Agents到Agentic AI，大模型智能体从调用工具的单个个体演化为具备自主协作能力的系统。
- ❑ 与之相应，协议的重心从单体智能体的工具调用，转向面向多智能体协同的标准化设计。
- ❑ 从MCP、ANP到A2A，反映了从单一功能协议到多层次、多维度协作架构的转变。



✧ 3.7.2 大模型智能体协议

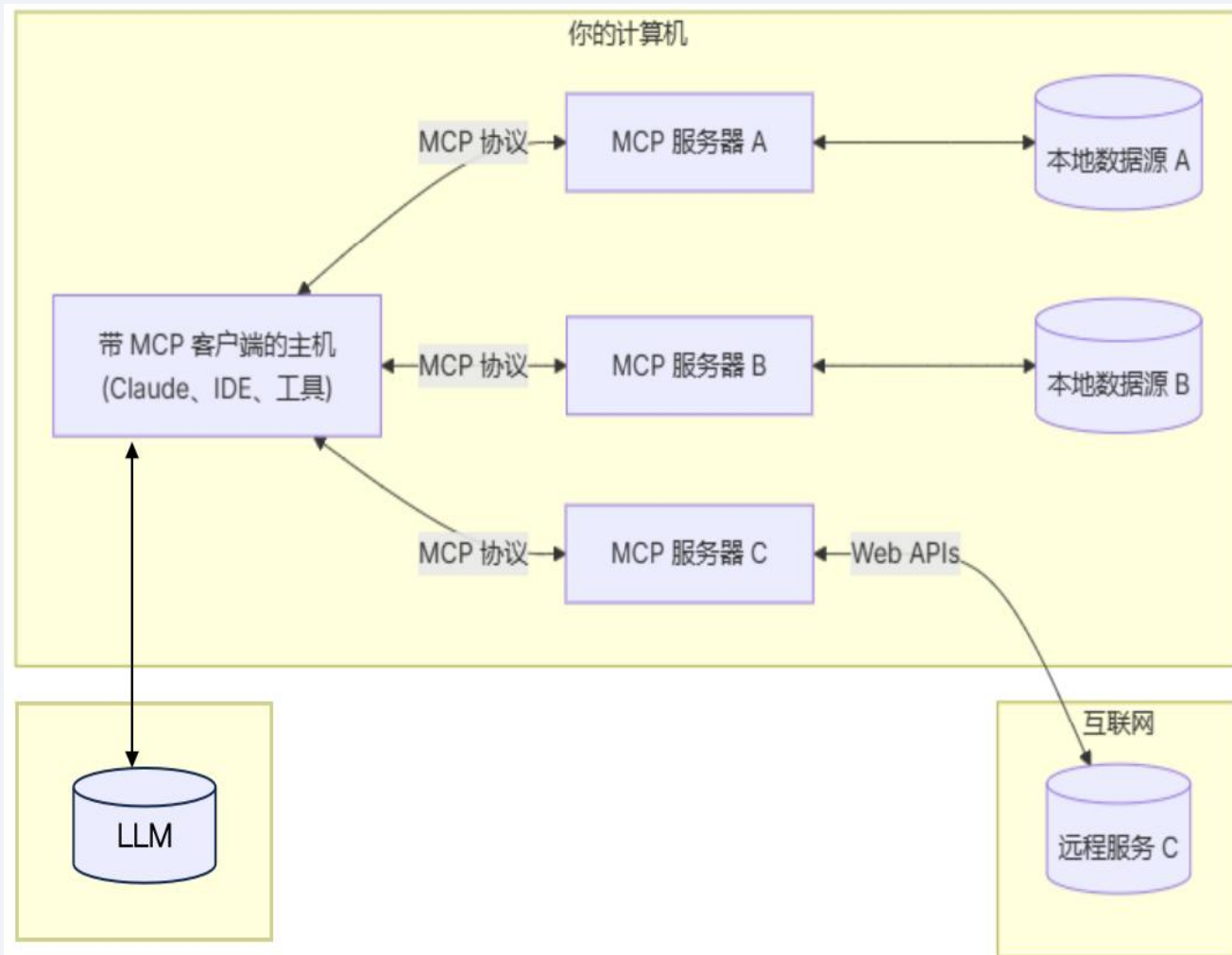
□ 智能体协议的分类维度

1. Context-Oriented vs. Inter-Agent
2. General-Purpose vs. Domain-Specific



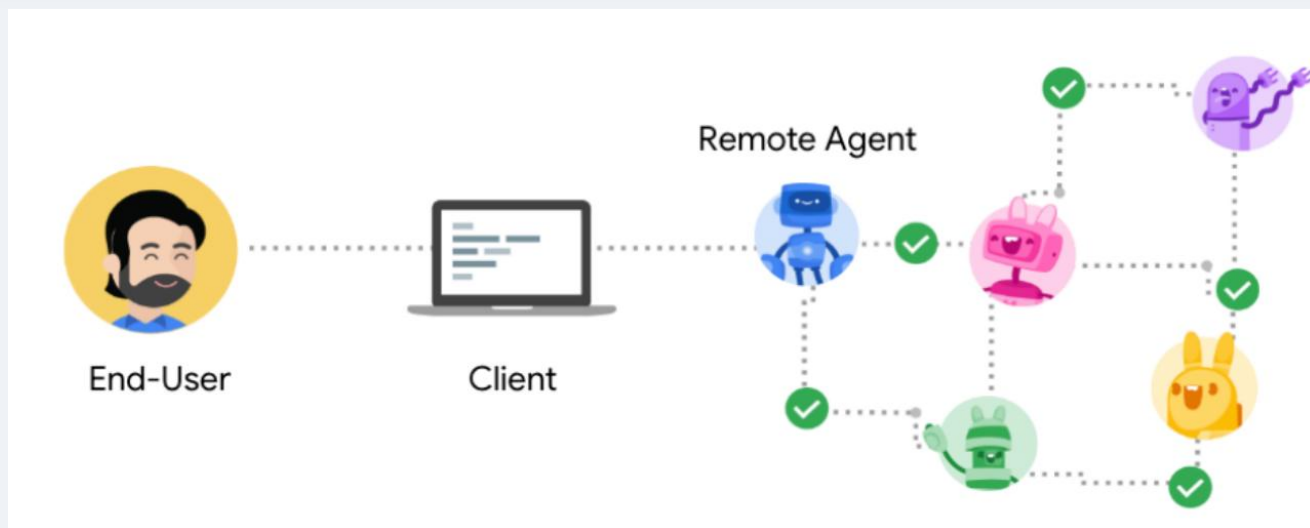
✧ 3.7.2 大模型智能体协议

- ❑ MCP (Model Context Protocol) : 连接 LLM与外部资源的标准协议
- ❑ 标准化LLM智能体和外部资源之间的交互。
- ❑ LLM指定了所需的资源和参数，由本地客户端与MCP服务器交互以获取外部资源或进行工具调用。
- ❑ MCP协议解耦了LLM响应与工具调用从而降低隐私泄露的风险。
- ❑ 外部资源提供方可通过一次性集成接入 MCP，从而使任何与MCP兼容的LLM智能体都能访问其服务。



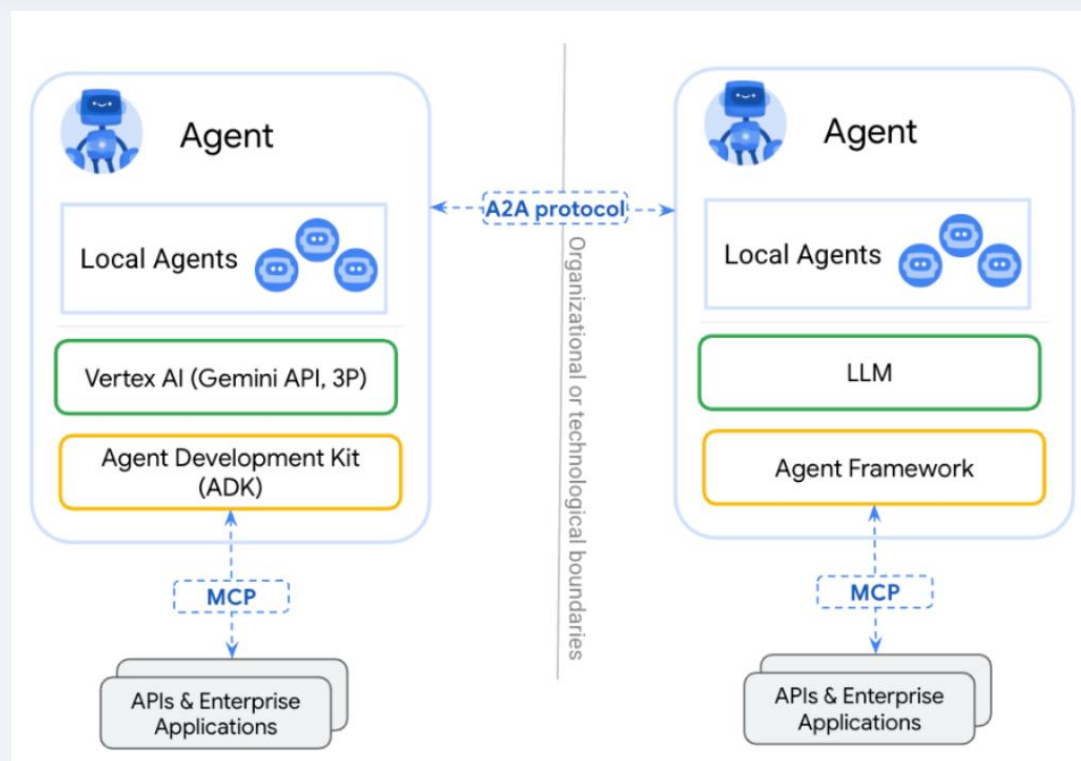
✧ 3.7.2 大模型智能体协议

- ❑ A2A (Agent-to-Agent) : 支持跨智能体协作的开放标准
- ❑ 促进由不同框架与厂商构建的智能体之间的无缝通信与协作，覆盖从能力发现到任务执行全流程，支持异构智能体之间的长期协作。
- ❑ 使用统一的传输与消息格式：基于 HTTP(S) 的 JSON-RPC 2.0。
- ❑ 引入Agent Card机制，用于能力发现与注册，支持智能体之间的相互识别与匹配。
- ❑ 支持任务管理工作流，涵盖任务的发起、推进与完成，支持多轮、长时协作任务。



✧ 3.7.2 大模型智能体协议

- ❑ MCP与A2A互为补充
- ❑ MCP提供结构化的输入/输出接口，以连接智能体与工具、API、外部资源。
- ❑ A2A则支持智能体之间动态、多模态的通信与协作。



✚ 3.7.3 AI Agents vs. Agentic AI

▣ AI Agents与Agentic AI多维度对比

	AI Agents	Agentic AI
组织结构	LLM+工具	多智体系统
关键特征	工具调用	多智体合作
主要功能	使用外部工具执行特定任务	复杂工作流自动化或高层级的目标实现
交互方式	被动响应	自主驱动
记忆模块	以任务内的短期记忆为主	多智能体共享的情境/任务记忆
时间上的连续性	任务内的短期连续性	跨工作流的连续性

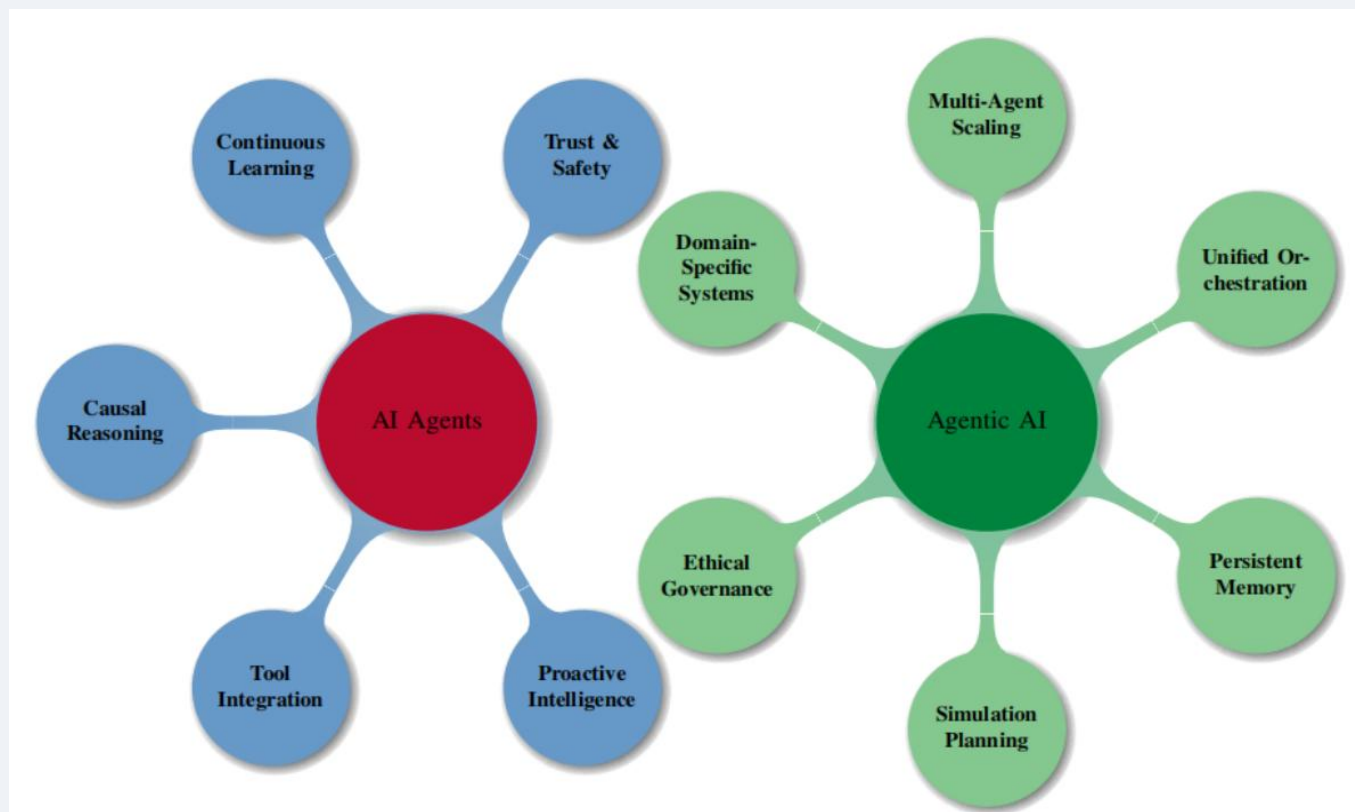
✧ 3.7.4 大模型智能体的未来展望

AI Agents

- 从被动响应到主动智能
- 更鲁棒的工具集成
- 因果推理能力增强
- 持续学习以适应跨场景和长期交互
- 用户信任与安全

Agentic AI

- 多智能体扩展与并行协作
- 元编排与角色动态分配
- 持久化记忆支撑长期任务
- 模拟规划实现前瞻性决策
- 伦理治理保障可控性与安全性
- 领域特化



✧ 3.7.5 AI Agents的未来展望

▣ 主动智能：

从被动响应转向主动任务发起，如个人助手类应用根据用户习惯自动优化日程安排。

▣ 更鲁棒的工具集成：

允许智能体动态地与外部环境交互。

▣ 因果推理增强：

增强推理能力，区分相关性与因果关系，实现更可靠的诊断、预测与规划能力。

▣ 持续学习：

引入反馈调节与持久记忆以适应跨场景和长期交互。

▣ 与信任机制：

通过反馈循环更新策略，同时引入偏见检测和伦理护栏，确保可控性。

▣ 用户信任与安全：

强化输出可验证性、偏差检测与伦理控制，为更高自主性铺设保障机制。

✧ 3.7.6 Agentic AI的未来展望

▣ 多智能体扩展与并行协作：

支持多个异构代理分布式并行协作，模拟人类团队式的合作流程。

▣ 元编排与角色动态分配：

通过元智能体自主实现任务分配、通信管理、依赖调度与冲突解决。

▣ 持久化记忆支撑长期任务：

构建语义、情景与任务知识的长期记忆体系，支持跨任务、跨时间的上下文保持与目标追踪。

▣ 模拟规划实现前瞻性决策：

通过虚拟环境预演决策，预判后果、降低风险、增强适应性。

▣ 伦理治理保障可控性与安全性：

建立责任追溯机制，明确每个智能体对决策的贡献，确保符合法律和伦理规范，隔离异常智能体。

▣ 领域特化：

针对法律、医疗、物流、气候等专业场景构建定制系统，以专精知识和推理流程实现超越通用系统的表现。

✧ 3.7.7 大模型智能体的总体未来展望

01

基座能力演进

- 提升感知理解
- 增强记忆与知识管理
- 提升推理决策能力
- 自主性提升

02

基础设施建设

- 算力
- 数据
- 网络
- 开放平台与标准

03

价值对齐与治理

- 伦理与道德约束
- 安全可信保障体系
- 完善法律法规框架

04

多Agent协作系统

- 领域专精：专业化Agent深度分工
- 通信与协作机制升级
- 集体智能与分布式决策

05

实体世界交互

- 物联网设备协同
- 具身智能
- AR、VR环境虚实结合

✚ 3.7.8 智能体发展挑战

底层技术瓶颈制约Agent性能跃升

- 多模态感知融合效率不足
 - 实时学习框架滞后
- 小样本与无监督学习薄弱
- 感知-决策闭环协同性差

幻觉问题与规划决策能力不足

- 数据质量缺陷影响输出可靠性
 - 规划失效与不可预测结果
 - 模糊性与可解释性挑战

多智能体协同与竞争平衡难题

- 交互与协调机制不成熟
 - 竞争与合作失衡
 - 通信兼容性瓶颈

Agent发展
面临挑战

安全治理体系与标准缺失

- 数据隐私与安全性风险
- 伦理与法律争议
- 标准规范空白

社会环境与不可控风险

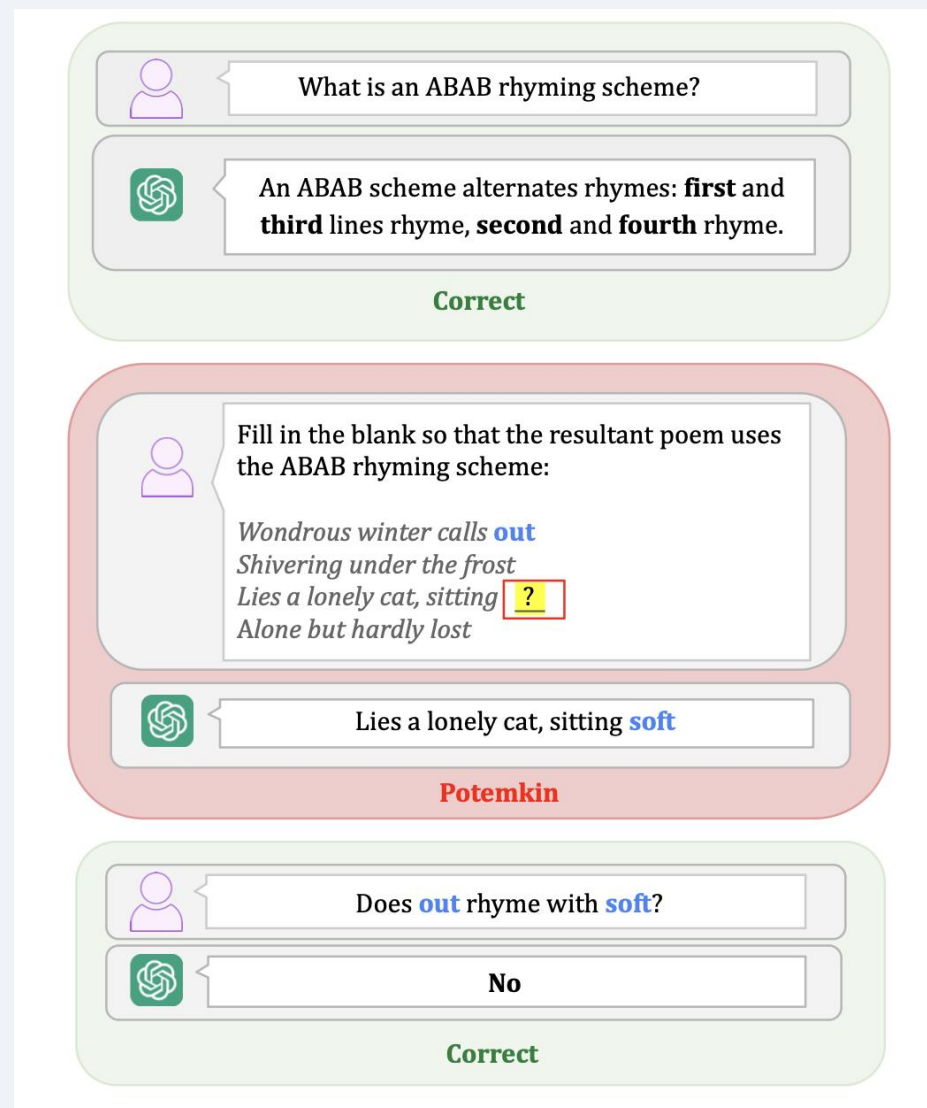
- 技术性失业加剧
- 技术失控风险

交互能力与个性化需求矛盾

- 交互方式单一
- 个性化服务能力弱

✧ 3.8 大模型是理解？还是模仿？

- **波特金式理解**：即模型能够**正确回答**某些能证明“**理解**”的关键问题，但在需要**应用该概念**的其他相关问题上却表现出与该理解相**矛盾**的错误。
- **“不连贯性”自我检测**：让模型先生成一个概念的例子，然后再让它判断自己生成的例子是否正确。如果前后矛盾，就说明其**内部理解不连贯**。
- **“自生成-自评判”流程**：让模型基于一个它答对的基准问题，生成新的相关问题并作答，最后再让它**自己来评判自己**生成的答案是否正确。



✧ 3.8 大模型是理解？还是模仿？

□ 实验结果：

“波特金式理解”不是个例，而是普遍存在的系统性问题。即使是当今最顶尖的语言模型（如 GPT-4o, Claude-3.5），在多个领域（文学、博弈论、心理学偏见）、多种任务上都表现出很高的“波特金率”。

所有模型都表现出显著的“不连贯率”。这揭示了LLM的“知识”甚至不是内部一致的，它对同一个概念的表征是相互冲突的。

Model	Incoherence	Potemkin rate (lower bound)
Llama-3.3	0.19 (0.03)	0.82 (0.02)
Claude-3.5	0.61 (0.05)	0.36 (0.02)
GPT-4o	0.64 (0.05)	0.46 (0.06)
GPT-o1-mini	0.16 (0.03)	0.66 (0.02)
GPT-o3-mini	0.03 (0.01)	0.66 (0.04)
Gemini-2.0	0.09 (0.02)	0.86 (0.02)
DeepSeek-V3	0.13 (0.03)	0.38 (0.02)
DeepSeek-R1	0.04 (0.02)	0.50 (0.02)
Qwen2-VL	0.13 (0.03)	0.82 (0.00)
Overall	0.22 (0.01)	0.62 (0.01)

Step 1

Generate an example of a slant rhyme.



“Time” and “mine”

Step 2



Is the following an example of a slant rhyme?

“Time” and “mine”



No

Incoherent



Yes

Coherent

✧ 3.8 大模型是理解？还是模仿？

▣研究动机：人类和DNN在视觉表征上的相似性和差异性

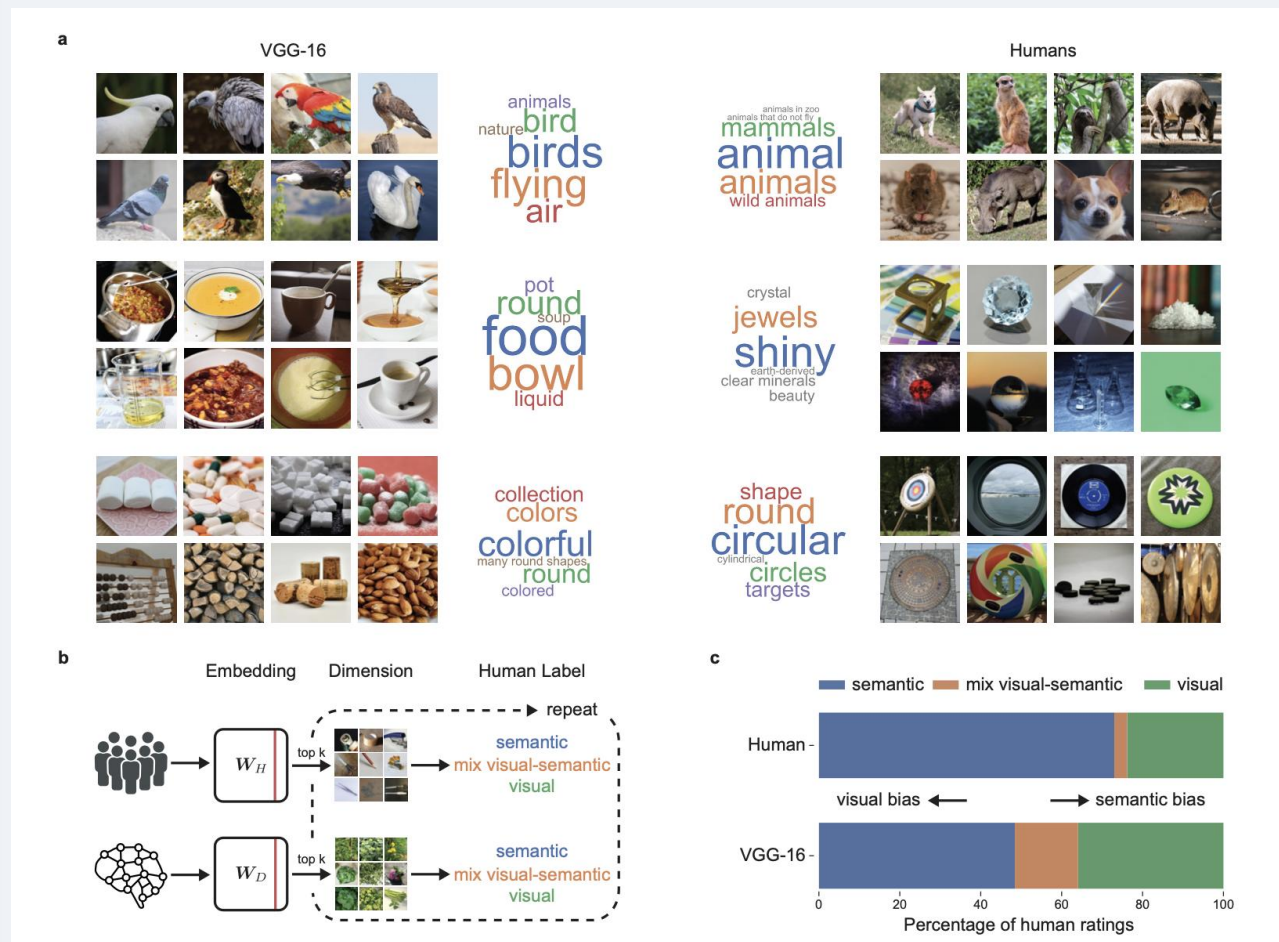
▣实验设计：“三选一”的任务，给参与者（人类和DNN）展示三张图片，找出其中最不像其他两张的那一张。

▣表征嵌入：“变分嵌入技术”从人类的选择数据和DNN的特征数据中分别提取出一种低维的表征嵌入，这些维度代表了物体的不同属性。

▣实验结果：

DNN的表征嵌入包含许多可解释的维度，这些维度能够反映物体的视觉和语义属性。例如，物体的颜色、形状等视觉特征，物体所属的类别（如食物、动物等）等语义信息。与人类维度相比，DNN维度的整体可解释性稍差，

人类在进行物体相似性判断时，更多地依赖于语义信息，而DNN则更多地依赖于物体的视觉特征。



3.8 大模型是理解？还是模仿？

□ **研究动机：** 为什么LLMs既能够很好地泛化，又容易出现幻觉，两者背后是否有着相同的机制

□ **实验设计：** 分别测试了当知识之间有因果关系（比如“住在巴黎”和“说法语”）和没有因果关系（比如“住在巴黎”和“喜欢蓝色”）时，LLMs的表现。

□ **理论分析：** 一个简化的一层单头注意力Transformer模型，当模型的输出矩阵和值矩阵是分解开时，模型能够学习到OCR任务，而当这两个矩阵合并时，模型就无法完成OCR任务，表明矩阵分解在OCR能力中起着关键作用。

EXAMPLE 1: GENERALIZATION AND HALLUCINATION BOTH AS OCR

Generalization: OCR with causally related knowledge

Training: { Alice lives in France .}; { Alice speaks French .}; { Raul lives in France .}.

Test: “What language does Raul speak ?”

Fine-tuned model: “ Raul speaks French .”

Hallucination: OCR with causally unrelated knowledge

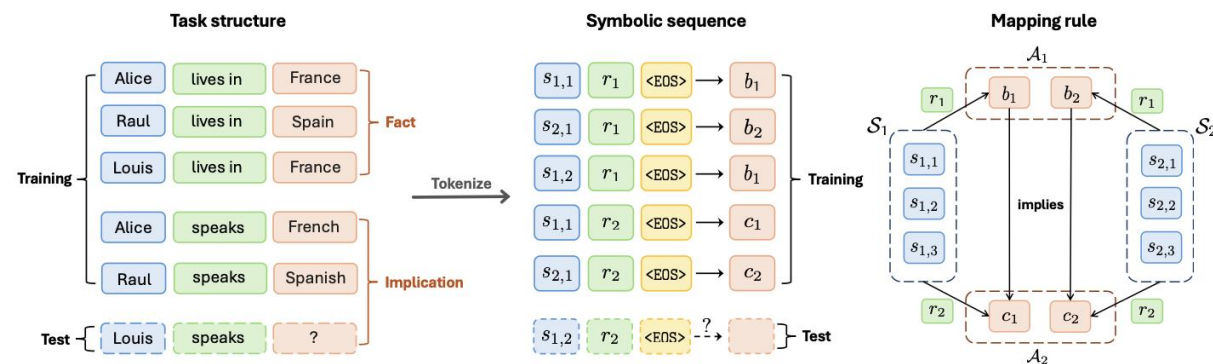
Training: { Alice lives in France .}; { Alice codes in Java .}; { Raul lives in France .}.

Test: “What language does Raul code in ?”

Fine-tuned model: “ Raul codes in Java .”

Notations

Subject: $s \in \mathcal{S}$ **Relation:** $r \in \{r_1, r_2\}$ **Answer:** $a \in \mathcal{A} = \mathcal{A}_1 \cup \mathcal{A}_2$, with $\mathcal{A}_1 = \{b_i\}_{i=1}^n$ being the fact set and $\mathcal{A}_2 = \{c_i\}_{i=1}^n$ being the implication set.



3.8 大模型是理解？还是模仿？

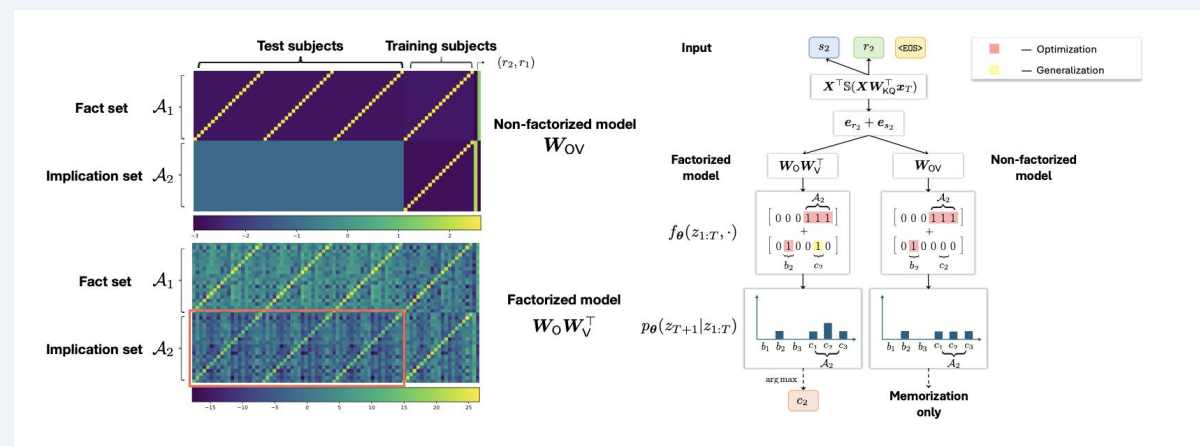
实验结果：

LLMs在有因果关系的知识上表现出很强的泛化能力，而在没有因果关系的知识上容易出现幻觉

简化的一层Transformer模型，实验发现只有当模型的输出矩阵和值矩阵是分解的时，它才能成功地完成OCR任务。这说明模型的结构对它的推理能力有着重要的影响。

即使只有很少的训练样本，模型也能够学会OCR，这解释了为什么LLMs在学习新知识后能够快速地进行泛化和幻觉。

Models	Generalization	Hallucination			
	City-Language	City-Language (CF)	Country-Code	Profession-Color	Sport-Music
Gemma-2-9B	0.00 (0.00)	0.19 (0.20)	0.19 (0.07)	1.64 (0.01)	0.56 (0.01)
OLMo-7B	0.07 (0.03)	1.33 (0.49)	0.15 (0.13)	1.84 (0.23)	0.17 (0.01)
Qwen-2-7B	0.13 (0.01)	4.55 (2.33)	3.63 (1.10)	0.82 (0.34)	0.40 (0.08)
Mistral-7B-v0.3	0.00 (0.00)	2.10 (0.01)	1.48 (0.52)	1.15 (0.56)	1.28 (0.13)
Llama-3-8B	0.00 (0.00)	1.18 (0.61)	0.77 (0.10)	0.93 (0.21)	0.63 (0.22)

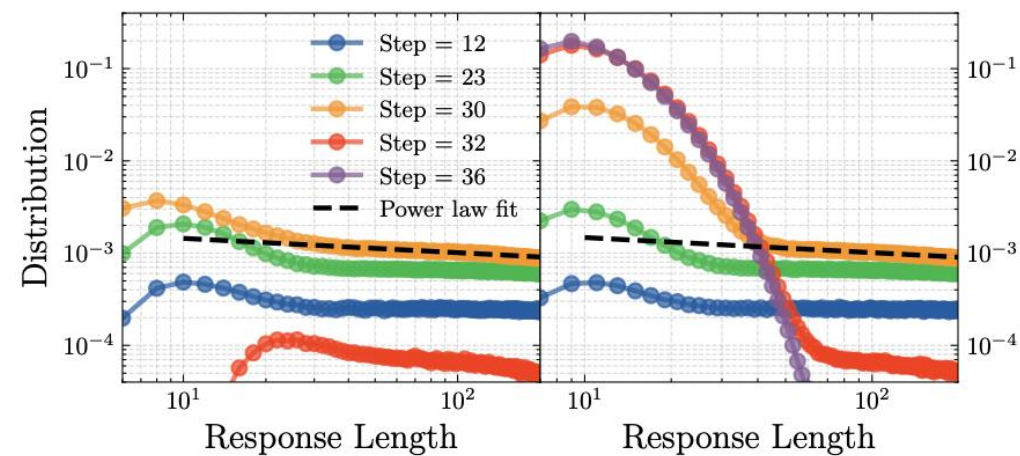
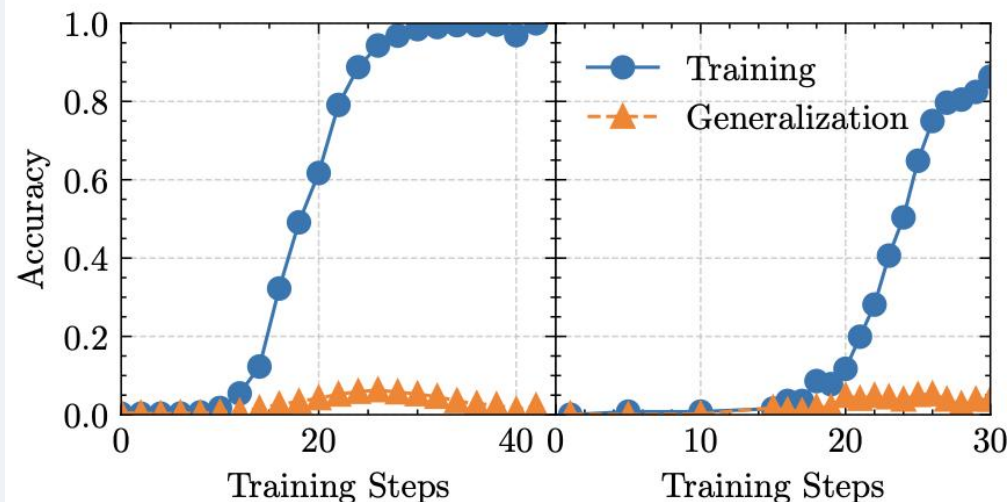


3.8 大模型是理解？还是模仿？

□ **研究动机**：DeepSeek是如何学会思考的？用统计物理的视角理解？

□ **CoNet概念**：LLM的推理过程可抽象为在一个概念网络上的随机游走。实验发现：基于大模型RL训练和CoNet抽象，发现一些现象和物理中的“相变”十分类似：准确率的陡峭增长、推理路径长度方差的峰值、幂律分布的路径长度 $P(L)$ 表现出幂律尾部 ($P(L) \sim L^{-\gamma}$)，说明模型在临界点附近，有最强的探索能力，兼具长短路径的推理能力

□ **LaC方法**：关键在于“仔细监控训练动态以识别尖锐学习转变的开始，这决定了停止或调整训练的最佳点”，继续训练会让模型思维固化



3.8 大模型是理解？还是模仿？

实验结果：

基于此，研究团队选择在理论物理中的典型问题——计算高阶不同圈的费曼图进行 LaC 的效果验证。他们利用 80 亿参数的 Qwen3-8B 模型，分阶段训练其求解松原频率求和问题。

值得关注的是，仅通过低阶图例进行临界点训练，模型可成功推导出未见过高阶图解，其表现甚至优于参数量高两个数量级的基准模型。

数据显示，经过 LaC 训练的模型在 1-loop 和 2-loop 图上的准确率分别达到 97.5% 和 56.9%，并能泛化至 3-loop 和 4-loop 问题，而未经训练的基准模型在这些任务上表现不佳。

Q: Solve $12+98=?$ Please solve this addition problem step by step

A: Let's solve the addition problem $\backslash(12 + 98\backslash)$ step by step, showing each digit and explaining the process.

1. Align the numbers vertically:

```
...
  12
+ 98
...
```

2. Add the rightmost digits (units place):

- The units digit of 12 is 2.
- The units digit of 98 is 8.
- Add these together: $\backslash(2 + 8 = 10\backslash)$.

3. Write down the result of the units place:

- Since $\backslash(2 + 8 = 10\backslash)$, write down the 0 in the units place.
- Carry over the 1 to the next column (tens place).

4. Add the digits in the tens place, including the carry-over:

- The tens digit of 12 is 1.
- The tens digit of 98 is 9.
- Add these together along with the carry-over: $\backslash(1 + 9 + 1 = 11\backslash)$.

5. Write down the result of the tens place:

- Since $\backslash(1 + 9 + 1 = 11\backslash)$, write down the 1 in the tens place.
- Carry over the 1 to the next column (hundreds place).

6. Since there are no more columns to add, write down the final carry-over:

- The carry-over is 1, so write it in the hundreds place.

Putting it all together, we get:

```
...
  12
+ 98
-----
 110
...
```

So, the final answer is $\backslash\boxed{110}\backslash$.

3. Write down the result of the units place:

└─✓' of'(P:0.54)
└─x' in'(P:0.46)

- Since $\backslash(2 + 8 = 10\backslash)$, write down the 0 in the units place

└─✓' Since'(P:0.68) └─x' we'(P:0.67)
└─x' The'(P:0.17) └─✓' write'(P:0.33)
└─x' Write'(P:0.16)

- Carry over the 1 to the next column (tens place).

└─✓' next'(P:0.57)
└─x' tens'(P:0.43)

